

SOFTWARE REQUIREMENTS & DESIGN SPECIFICATION

Digital Banking Platform

Software Development Life Cycle

Planning · Analysis · Design · Implementation · Testing

ABSTRACT

This dossier specifies a secure, compliant, event-driven core-banking platform delivered as three products — a **backend API**, a **customer mobile app** (iOS + Android), and an **admin dashboard** — covering customer accounts, wallets, a double-entry transaction ledger, wallet-to-wallet transfers, KYC, an immutable audit trail, and role-based access control. Enterprise concerns — JWT with rotating refresh tokens, a multi-role authorization model, an API gateway with rate limiting, a transactional-outbox event backbone, and full observability via Prometheus, Grafana, and distributed tracing — are treated as first-class, launch-blocking requirements rather than later add-ons. The document walks the five SDLC phases end to end and includes an entity-relationship diagram plus mobile and dashboard flow diagrams that trace every requirement to a concrete design decision.

DOCUMENT	VERSION	DATE	CLASSIFICATION	GRADE
SDLC-DBP-001	1.0	June 12, 2026	Confidential	Enterprise Fintech

Contents

#	PHASE	DOCUMENT	PURPOSE
1	Planning	docs/01-planning.md	Vision, scope, stakeholders, risk, cost, schedule, feasibility.
2	Analysis	docs/02-analysis.md	Functional + non-functional requirements, use cases, data needs.
3	Design	docs/03-design.md	Architecture, services, data models, APIs, security, observability.
3	Diagrams	docs/04-diagrams.md	ER diagram + mobile and dashboard flow diagrams (rendered images).
4	Implement	docs/05-implementation-plan.md	Backlog, epics, milestones, sprint-by-sprint delivery plan.
5	Testing	docs/06-testing-strategy.md	Test pyramid, test cases, NFR tests, CI gates, exit criteria.
7	Tech Stack	docs/07-tech-stack.md	Spring Boot / React / React Native; event-driven vs microservices.
8	Infra	docs/08-deployment-infra.md	Monolith deployment, cloud architecture, orchestration, scaling.
9	Backend	docs/09-backend-api.md	Spring Boot feature docs + code: auth, RBAC, ledger, outbox, etc.
10	Mobile	docs/10-mobile-app.md	React Native feature docs + code: secure auth, KYC, transfers, push.
11	Dashboard	docs/11-admin-dashboard.md	React feature docs + code: RBAC UI, search, operator actions.
12	GitOps	docs/12-gitops-deployment.md	EKS + Argo CD: Dockerfile, manifests, Kustomize, CI/CD, secrets.

Repository layout

```

banking-sdlc/
├── README.md                # this index
├── Digital-Banking-Platform-SDLC.pdf # the compiled deliverable
├── docs/                    # human-readable chapters (Markdown)
│   ├── 01-planning.md ... 12-gitops-deployment.md
│   └── diagrams/           # rendered .png / .svg
│       └── src/            # mermaid (.mmd) sources
└── tools/                  # build machinery (not for reading)
    ├── build.sh            # one-command build
    ├── build_pdf.py        # markdown → self-contained HTML
    ├── render_pdf.js       # HTML → PDF with page-number footer
    └── fonts/DMSans-VF.ttf # embedded font

```

To rebuild the PDF: `tools/build.sh` (renders diagrams, embeds them, prints the PDF to the repo root).

Product at a glance

Deliverables

- Backend API (core-banking services + gateway)
- Customer mobile app (iOS + Android)
- Admin dashboard (operator console)

Core features

- Customer accounts
- Wallets
- Transactions
- Transfers
- KYC (Know Your Customer)
- Audit logs
- Admin dashboard
- RBAC (Role-Based Access Control)

Enterprise features

- JWT + Refresh Token authentication
- Multi-role system
- Audit Trail
- Event-driven architecture
- API Gateway

- Rate Limiting
- Prometheus metrics
- Grafana dashboards
- Distributed tracing

How to read this

Start with Planning to understand *why* and *what for*, move to Analysis for *what exactly*, Design for *how*, then Implementation for *how it ships* and Testing for *how it's proven*. Each document is self-contained but cross-references the others.

Rating target: enterprise / regulated-fintech grade.

Phase 1 — Planning

Digital Banking Platform · SDLC Phase 1 of 3 Status: Baseline · Owner: Product + Engineering Leadership

1.1 Purpose & Vision

Build a **secure, compliant, event-driven digital banking platform** that lets customers open accounts, hold money in wallets, move funds via transactions and transfers, and pass identity verification (KYC) — while giving operators a full audit trail, an admin dashboard, and fine-grained role-based access.

Vision statement

Provide a regulator-ready core-banking backbone that any fintech or neobank can deploy, observe, and scale — with security, traceability, and operational visibility built in from day one, not bolted on later.

Why now

- Demand for embedded finance and neobanking continues to grow.
- Regulators (AML/KYC, PSD2-style open banking, data residency) require

auditable, traceable systems — retrofitting compliance is expensive.

- Event-driven + observability-first architecture reduces incident cost and

enables horizontal scale.

1.2 Business Objectives

ID	OBJECTIVE	SUCCESS METRIC
O1	Enable end-to-end customer onboarding	< 5 min from signup to verified (KYC) account
O2	Process money movement reliably	99.99% transaction durability, zero double-spend
O3	Meet audit & compliance requirements	100% of state changes captured in audit trail
O4	Operate securely	0 critical vulns in prod; all access via RBAC
O5	Provide operational visibility	MTTD < 5 min via metrics/traces/alerts
O6	Scale horizontally	10x load with linear cost, no schema rewrite

1.3 Scope

In scope (this initiative)

- Customer account lifecycle (create, suspend, close).
- Wallet management (one or more wallets per customer, multi-currency-ready).
- Transactions (credits, debits, holds, reversals) with ledger integrity.
- Transfers (intra-platform wallet-to-wallet; rails-ready abstraction for

external).

- KYC workflow (document submission, verification states, decisioning hooks).
- Audit logs / immutable audit trail.
- Admin dashboard (operator console).
- **Customer mobile app (iOS + Android)** — primary end-user client for

onboarding, wallets, transfers, and KYC.

- RBAC with a multi-role system.
- Security: JWT access tokens + refresh tokens.
- Platform: API Gateway, rate limiting, event-driven backbone.
- Observability: Prometheus metrics, Grafana dashboards, distributed tracing.

Out of scope (explicitly, for v1)

- Card issuing / physical cards.
- Direct integration with national payment rails (abstracted, not built).
- Lending, credit scoring, interest accrual.
- Customer **web** app (mobile-first for v1; web is a later phase — the API

already supports it).

- ML-based fraud scoring (event hooks provided; model not included).

Assumptions

- Cloud-native deployment (containers + orchestrator).
- Single primary region at launch; multi-region is a later phase.
- A licensed banking/EMI partner or sandbox provides regulatory cover.

Constraints

- Must satisfy AML/KYC and data-protection obligations from day one.
- Money movement must be **strongly consistent** within the ledger.
- All privileged actions must be authenticated, authorized, and audited.

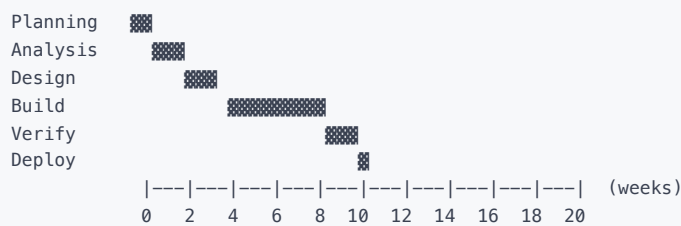
1.4 Stakeholders

STAKEHOLDER	INTEREST / CONCERN	INFLUENCE
Customers	Fast onboarding, safe funds, reliable transfers	High
Compliance / AML team	KYC correctness, audit trail completeness	High
Operations / Support	Admin dashboard, ability to investigate & remediate	High
Engineering	Maintainability, clear contracts, observability	High
Security	AuthN/Z, secrets, least privilege, attack surface	High
Finance / Treasury	Ledger accuracy, reconciliation	Medium
Executives / Investors	Time to market, cost, regulatory risk	High
Regulators / Auditors	Traceability, controls, data handling	High

1.5 Deliverables & Milestones

MILESTONE	DELIVERABLE	PHASE
M0	Approved plan, scope, budget (this document)	Planning
M1	Requirements baseline + use-case catalog	Analysis
M2	Architecture & detailed design sign-off	Design
M3	Core ledger + accounts/wallets MVP	Build
M4	KYC + RBAC + audit trail	Build
M5	Gateway, rate limiting, observability stack	Build
M6	Security review, pen test, compliance review	Verify
M7	Production launch (single region)	Deploy

1.6 High-Level Schedule (indicative)



Estimate: ~18–20 weeks to first production launch with a small, senior team.

1.7 Team & Roles (RACI summary)

FUNCTION	ROLE ON PROJECT
Product Owner	Prioritization, scope, sign-off (A)
Tech Lead / Architect	Architecture, design authority (R/A)
Backend Engineers	Services, ledger, APIs (R)
Platform / DevOps	Gateway, CI/CD, observability (R)
Security Engineer	Threat model, auth, review (R/C)
Compliance Officer	KYC/AML rules, audit requirements (C/A)
QA Engineer	Test strategy, automation (R)
SRE	SLOs, on-call, dashboards (C)

1.8 Budget / Cost Drivers (qualitative)

- **People:** senior backend + platform + security (largest line item).
- **Infrastructure:** orchestrated compute, managed DB (with PITR), message

broker, observability stack (Prometheus/Grafana/tracing backend).

- **Third parties:** KYC/identity verification vendor, sanctions/PEP screening.
- **Compliance:** legal, audit, pen testing.
- **Contingency:** 15–20% for regulatory and security unknowns.

1.9 Risk Register

ID	RISK	LIKELIHOOD	IMPACT	MITIGATION
R1	Ledger inconsistency / double-spend	Low	Critical	Double-entry ledger, DB transactions, idempotency keys, tests
R2	KYC vendor downtime / poor accuracy	Medium	High	Vendor abstraction, async retries, manual-review fallback
R3	Security breach / token theft	Low	Critical	Short-lived JWT, rotating refresh tokens, RBAC, secrets mgmt
R4	Compliance gaps surface late	Medium	High	Compliance officer embedded from Analysis; audit trail first
R5	Scope creep (cards, lending, multi-region)	High	Medium	Strict scope gate; backlog for later phases
R6	Observability added too late to debug incidents	Medium	High	Metrics/tracing are launch blockers, not nice-to-haves
R7	Event ordering / duplicate processing	Medium	High	Idempotent consumers, outbox pattern, dedup keys
R8	Rate-limit misconfiguration (DoS or false block)	Medium	Medium	Tunable policies, monitoring, gradual rollout

1.10 Feasibility Assessment

DIMENSION	ASSESSMENT
Technical	Feasible with mature, well-understood tech (containers, RDBMS, broker).
Operational	Feasible; requires SRE discipline and a staffed compliance function.
Economic	Justified if onboarding + transaction volume targets are met.
Legal/Regulatory	Feasible only with KYC/AML controls and a licensed partner/sandbox.
Schedule	~18–20 weeks realistic for MVP with a senior team; aggressive but doable.

Go / No-Go recommendation: GO, conditional on (a) a confirmed licensed/EMI partner, (b) a named compliance owner, and (c) treating audit trail + observability as launch-blocking, not optional.

1.11 Success Criteria (exit gate for Planning)

- Vision, scope (in/out), and assumptions documented and agreed.
- Stakeholders identified with interests and influence.
- Milestones, indicative schedule, and team roles defined.
- Risk register with mitigations established.
- Feasibility assessed; Go/No-Go decision recorded.

➡ Proceed to **Phase 2 — Analysis** ([02-analysis.md](#)).

Phase 2 — Analysis

Digital Banking Platform · SDLC Phase 2 of 3 Status: Requirements Baseline · Owner: Product + Tech Lead + Compliance

2.1 Purpose

Translate the approved plan into precise, testable requirements: **what** the system must do (functional), **how well** it must do it (non-functional), the actors involved, the use cases they perform, the data the system holds, and the business rules that govern money and identity.

2.2 Actors

ACTOR	DESCRIPTION
Customer	End user; owns accounts and wallets, initiates transfers.
Admin / Operator	Staff using the admin dashboard to investigate and remediate.
Compliance Officer	Reviews KYC, sanctions hits, suspicious activity.
Support Agent	Limited operator; reads data, opens tickets, cannot move money.
System / Scheduler	Automated jobs (settlement, KYC polling, audit export).
External KYC Vendor	Verifies identity documents and returns a decision.
External Rail (abstracted)	Future external transfer destination.

2.3 Functional Requirements

FR-A · Customer Accounts

- FR-A1: Register a customer with unique email/phone.
- FR-A2: Authenticate via credentials, receive JWT access + refresh token.
- FR-A3: View and update profile (subject to KYC lock on key fields).
- FR-A4: Account lifecycle states: `PENDING → ACTIVE → SUSPENDED → CLOSED`.
- FR-A5: Admin can suspend/close accounts (audited, RBAC-gated).

FR-B · Wallets

- FR-B1: Each customer has one or more wallets.
- FR-B2: Wallet holds a balance in a single currency (multi-currency-ready).
- FR-B3: Balance is **derived from the ledger**, never edited directly.
- FR-B4: Wallet states: `ACTIVE`, `FROZEN`, `CLOSED`.
- FR-B5: Freezing a wallet blocks debits but allows reconciliation reads.

FR-C · Transactions

- FR-C1: Every money movement creates immutable double-entry ledger records.
- FR-C2: Transaction types: `CREDIT`, `DEBIT`, `HOLD`, `RELEASE`, `REVERSAL`.
- FR-C3: Transactions carry an **idempotency key** to prevent duplicates.
- FR-C4: A transaction is atomic: either fully applied or not at all.
- FR-C5: Insufficient funds → transaction rejected, no partial state.
- FR-C6: Reversal references the original transaction and is itself audited.

FR-D · Transfers

- FR-D1: Wallet-to-wallet transfer between customers (intra-platform).
- FR-D2: Transfer = paired debit + credit, committed atomically.
- FR-D3: Transfer is rejected if source frozen, KYC-incomplete, or low balance.
- FR-D4: Transfer emits a domain event (`TransferCompleted` / `TransferFailed`).
- FR-D5: External-rail transfers are modeled but stubbed (out of scope v1).

FR-E · KYC

- FR-E1: Customer submits identity documents.
- FR-E2: KYC states: `NOT_STARTED` → `PENDING` → `VERIFIED` → `REJECTED` .
- FR-E3: Verification is delegated to an external vendor (async).
- FR-E4: Money-out actions require `VERIFIED` ; configurable limits below it.
- FR-E5: Sanctions/PEP screening hook on verification.
- FR-E6: Compliance officer can override decision (audited, RBAC-gated).

FR-F • Audit Logs / Trail

- FR-F1: Every state change records actor, action, before/after, timestamp.
- FR-F2: Audit records are **append-only and immutable**.
- FR-F3: Audit trail is queryable by entity, actor, time range, action.
- FR-F4: Privileged/admin actions are always audited.
- FR-F5: Audit export for regulators (signed, tamper-evident).

FR-G • Admin Dashboard

- FR-G1: Search customers, wallets, transactions, KYC cases.
- FR-G2: View audit trail per entity.
- FR-G3: Perform RBAC-gated operations (suspend, freeze, reverse, override).
- FR-G4: View platform health (links to Grafana / metrics).

FR-H • RBAC & Multi-Role

- FR-H1: Roles: `customer` , `support` , `operator` , `compliance` , `admin` , `auditor` (read-only).
- FR-H2: Permissions are attached to roles; users may hold multiple roles.
- FR-H3: Every protected endpoint enforces a required permission.
- FR-H4: Privilege escalation attempts are denied and audited.

FR-I • Platform / Enterprise

- FR-I1: All external traffic enters via an **API Gateway**.
- FR-I2: **Rate limiting** per client/identity/endpoint, configurable.
- FR-I3: Domain changes publish **events** to a broker (event-driven).
- FR-I4: Consumers are **idempotent** (handle duplicates safely).

- FR-15: Services expose **Prometheus metrics**.
- FR-16: **Distributed tracing** spans gateway → services → DB/broker.
- FR-17: **Grafana dashboards** visualize golden signals + business KPIs.

2.4 Non-Functional Requirements (NFRs)

ID	CATEGORY	REQUIREMENT
NFR1	Security	JWT access (short TTL) + rotating refresh tokens; TLS everywhere; secrets in a vault.
NFR2	Consistency	Ledger operations strongly consistent; no double-spend; idempotent writes.
NFR3	Availability	99.9%+ for read APIs; money-movement path designed for graceful degradation.
NFR4	Performance	p95 API latency < 300 ms for core reads; transfers < 800 ms p95.
NFR5	Scalability	Stateless services scale horizontally; broker absorbs spikes.
NFR6	Auditability	100% of state changes captured; audit immutable and exportable.
NFR7	Observability	Metrics, logs, traces correlated by trace/request ID.
NFR8	Compliance	KYC/AML enforced; data residency & retention configurable.
NFR9	Privacy	PII encrypted at rest; least-privilege access; right-to-erasure workflow.
NFR10	Reliability	At-least-once events + idempotency = effectively-once processing.
NFR11	Maintainability	Clear service boundaries, versioned APIs, documented contracts.
NFR12	Recoverability	DB PITR; RPO ≤ 5 min, RTO ≤ 1 hr for core services.

2.5 Key Use Cases

UC-1 · Customer Onboarding (with KYC)

Actor: Customer

Pre: None

Flow:

1. Customer registers (FR-A1).
2. System creates account in `PENDING`, issues JWT (FR-A2).
3. Customer submits KYC documents (FR-E1).
4. System sends to vendor; state `PENDING` (FR-E3).
5. Vendor returns decision; sanctions hook runs (FR-E5).
6. On `VERIFIED`, account → `ACTIVE`, default wallet created (FR-B1).

Post: Customer can transact within limits.

Alt: `REJECTED` → compliance review queue (FR-E6).

UC-2 · Wallet-to-Wallet Transfer

Actor: Customer

Pre: Source `VERIFIED`, `ACTIVE`, sufficient balance.

Flow:

1. Customer requests transfer with idempotency key (FR-C3).
2. System validates KYC, wallet state, balance (FR-D3).
3. Ledger writes paired debit+credit in one DB transaction (FR-D2).
4. Emits `TransferCompleted` event (FR-D4).
5. Audit record written (FR-F1).

Post: Balances updated; event consumers notified.

Alt: Validation fails → reject, emit `TransferFailed`, audit, no balance change.

UC-3 · Admin Reverses a Transaction

Actor: Operator (RBAC `transaction:reverse`)

Flow:

1. Operator locates transaction in dashboard (FR-G1).
2. Requests reversal; RBAC checked (FR-H3).
3. System writes compensating ledger entries referencing original (FR-C6).

4. Audit record with actor + reason (FR-F4).

Post: Net effect reversed; full trail preserved.

UC-4 · Compliance Reviews KYC Case

Actor: Compliance Officer

Flow:

1. Officer opens pending/rejected case.
2. Reviews documents, sanctions hits.
3. Approves or rejects (override) — audited (FR-E6).

Post: Account KYC state updated; downstream limits adjusted.

UC-5 · Auditor Exports Trail

Actor: Auditor (read-only)

Flow:

1. Auditor queries by entity/time/action (FR-F3).
2. Requests signed export (FR-F5).

Post: Tamper-evident export delivered; export action itself audited.

2.6 Business Rules

- BR1: **A wallet balance can never be negative** (except explicit overdraft

products, out of scope v1).

- BR2: **Debits require sufficient available balance** (balance minus holds).
- BR3: **Money-out requires KYC = VERIFIED** (sub-limits configurable for lower tiers).

- BR4: **Every ledger entry is immutable**; corrections are new compensating entries, never edits.

- BR5: **Idempotency key** uniquely identifies a money-movement request for a

configurable window.

- BR6: **Sum of all ledger entries per closed transaction = 0** (double-entry invariant).

- BR7: **Frozen/suspended** entities reject debits but remain readable.
- BR8: **Refresh token rotation**: using a refresh token invalidates it and issues a new pair; reuse of a consumed token revokes the session.

2.7 Data Requirements (conceptual)

Core entities and key attributes (refined into schema in Design):

- **Customer** — id, contact, status, kyc_status, created_at.
- **Wallet** — id, customer_id, currency, status, created_at (balance derived).
- **LedgerEntry** — id, transaction_id, wallet_id, direction, amount, created_at.
- **Transaction** — id, type, status, idempotency_key, metadata, created_at.
- **Transfer** — id, source_wallet, dest_wallet, amount, status, event_ref.
- **KycCase** — id, customer_id, status, vendor_ref, decided_by, decided_at.
- **AuditRecord** — id, actor_id, action, entity, before, after, ts, trace_id.
- **Role / Permission / UserRole** — RBAC mapping tables.
- **RefreshToken** — id, user_id, hash, status, expires_at, rotated_from.

See [03-design.md](#) §3.4 for the physical data model and relationships.

2.8 Traceability (requirements → use cases)

REQUIREMENT GROUP	COVERED BY USE CASES
FR-A, FR-E	UC-1, UC-4
FR-B, FR-C, FR-D	UC-2, UC-3
FR-F	UC-3, UC-5
FR-H (RBAC)	UC-3, UC-4, UC-5
FR-I (platform)	Cross-cutting (all)

2.9 Success Criteria (exit gate for Analysis)

- Functional requirements enumerated and grouped.
- NFRs defined with measurable targets.
- Actors, use cases, and business rules documented.
- Conceptual data model and requirement→use-case traceability established.

➡ Proceed to **Phase 3 — Design** ([03-design.md](#)).

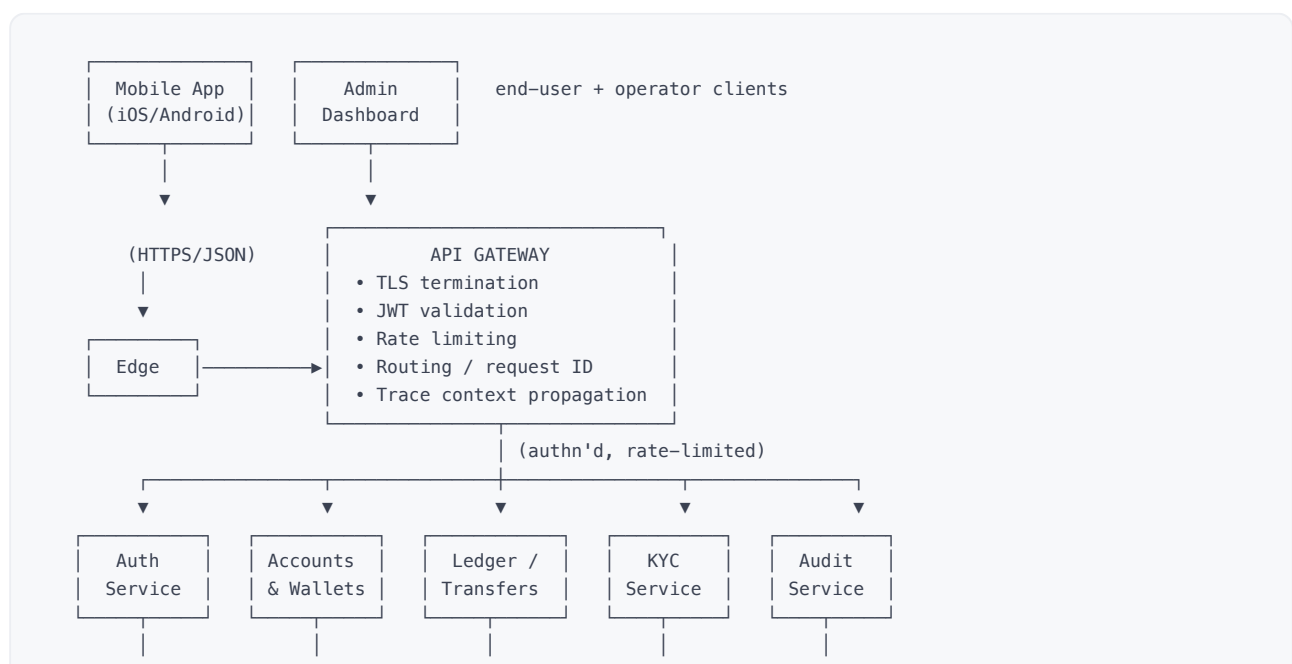
Phase 3 — Design

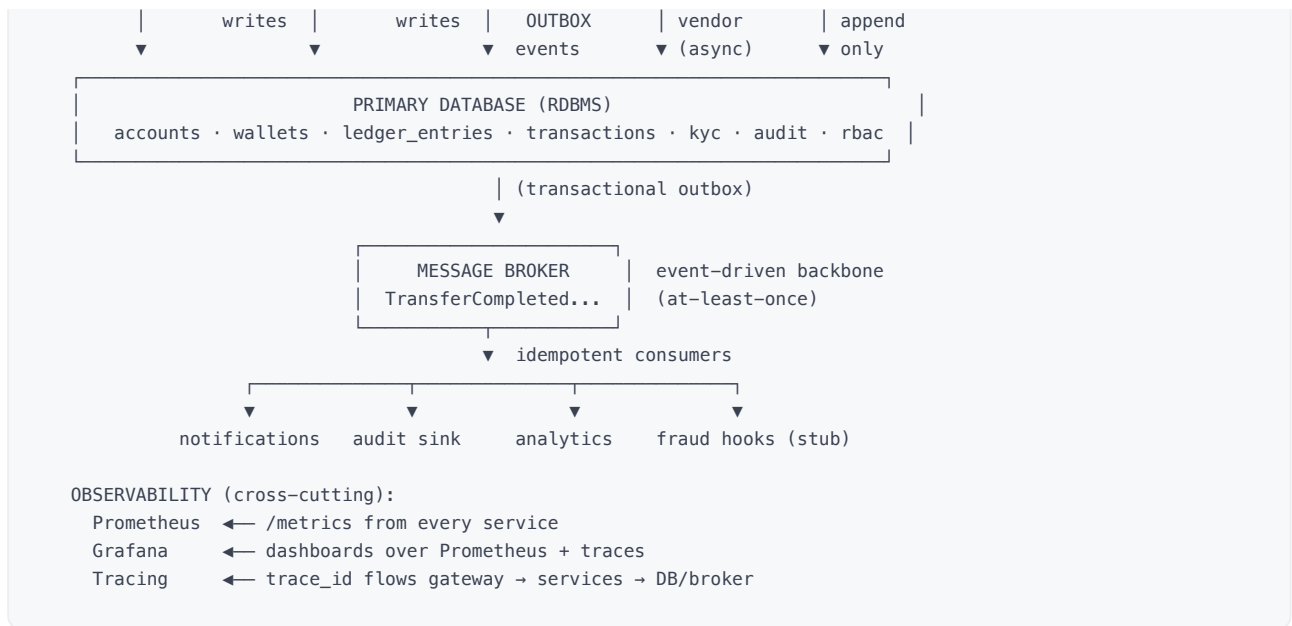
Digital Banking Platform · SDLC Phase 3 of 3 Status: Architecture & Detailed Design · Owner:
Architect + Security + Platform

3.1 Design Goals

- **Correctness of money** above all: double-entry ledger, atomic writes, idempotency, no double-spend.
- **Security & compliance by construction:** JWT + refresh rotation, RBAC, immutable audit trail, encrypted PII.
- **Operability:** every request is traceable; every service is measured.
- **Evolvability:** clear service boundaries, event-driven decoupling, versioned APIs.

3.2 High-Level Architecture





Style: service-oriented + event-driven. Services own their logic and share one strongly-consistent ledger database for money correctness; non-critical fan-out (notifications, analytics, fraud) happens asynchronously via events.

3.3 Services / Components

SERVICE	RESPONSIBILITY	KEY NFRS
API Gateway	TLS, JWT validation, routing, rate limiting, trace propagation	NFR1, NFR4
Auth Service	Login, JWT issue, refresh-token rotation, session revocation	NFR1
Accounts & Wallets	Customer lifecycle, wallet lifecycle, balance reads (from ledger)	NFR2, NFR3
Ledger & Transfers	Double-entry postings, transactions, transfers, idempotency, outbox	NFR2, NFR10
KYC Service	Document intake, vendor orchestration, decisioning, screening hooks	NFR8
Audit Service	Append-only audit records, query, signed export	NFR6
RBAC (lib + store)	Roles, permissions, enforcement middleware	NFR1
Admin Dashboard	Operator UI over the above (RBAC-gated)	NFR1, NFR7
Mobile App (client)	iOS/Android customer app: onboarding, KYC capture, wallets, transfers	NFR1, NFR4
Event Consumers	Notifications, analytics, fraud hooks (idempotent)	NFR10

3.3.1 Mobile App (customer client)

The mobile app is the primary end-user surface and a pure consumer of the

[/v1](#) REST API — it holds no business logic of its own.

- **Platforms:** iOS + Android (native or a single cross-platform codebase such as React Native — implementation choice deferred to the team).
- **Auth:** stores the **refresh token in the OS secure enclave** (iOS Keychain / Android Keystore), never in plain storage; short-lived access JWT held in memory. Silent refresh + rotation per BR8.
- **Key flows:** registration, KYC document/selfie capture and upload, wallet balances, transfer initiation with an **idempotency key generated on-device** (so retries over flaky mobile networks never double-charge — BR5), and push

notifications driven by `TransferCompleted` / `KycVerified` events.

- **Security:** certificate pinning to the gateway, biometric unlock

(Face/Touch ID), jailbreak/root signal, no PII cached beyond what's needed.

- **Resilience:** offline-tolerant read cache; all writes are idempotent and

retried with backoff.

3.4 Data Model (physical, relational)

```
customers
  id          UUID PK
  email       CITEXT UNIQUE
  phone       TEXT UNIQUE
  status      ENUM(PENDING, ACTIVE, SUSPENDED, CLOSED)
  kyc_status  ENUM(NOT_STARTED, PENDING, VERIFIED, REJECTED)
  created_at  TIMESTAMPTZ

wallets
  id          UUID PK
  customer_id UUID FK → customers.id
  currency    CHAR(3)          -- ISO 4217
  status      ENUM(ACTIVE, FROZEN, CLOSED)
  created_at  TIMESTAMPTZ
  -- balance is DERIVED: SUM(ledger_entries) – never stored as source of truth

transactions
  id          UUID PK
  type        ENUM(CREDIT, DEBIT, HOLD, RELEASE, REVERSAL)
  status      ENUM(PENDING, POSTED, FAILED, REVERSED)
  idempotency_key TEXT UNIQUE   -- (key, scope) dedup
  reverses_id UUID FK → transactions.id NULL
  metadata    JSONB
  created_at  TIMESTAMPTZ

ledger_entries          -- immutable, append-only
  id          UUID PK
  transaction_id UUID FK → transactions.id
  wallet_id   UUID FK → wallets.id
  direction   ENUM(DEBIT, CREDIT)
  amount      NUMERIC(20,4)    -- minor-unit safe, never float
  created_at  TIMESTAMPTZ
  CONSTRAINT: per transaction, SUM(credits) = SUM(debits) -- double-entry

transfers
  id          UUID PK
  source_wallet UUID FK → wallets.id
  dest_wallet  UUID FK → wallets.id
  amount      NUMERIC(20,4)
  status      ENUM(PENDING, COMPLETED, FAILED)
  transaction_id UUID FK → transactions.id
  created_at  TIMESTAMPTZ

kyc_cases
  id          UUID PK
  customer_id UUID FK → customers.id
```

```

status          ENUM(PENDING,VERIFIED,REJECTED)
vendor_ref      TEXT
decided_by      UUID NULL          -- compliance officer (override)
decided_at      TIMESTAMPTZ NULL

audit_records   -- immutable, append-only
  id            UUID PK
  actor_id      UUID
  action        TEXT
  entity_type   TEXT
  entity_id     UUID
  before        JSONB
  after         JSONB
  trace_id      TEXT              -- correlate with tracing
  created_at    TIMESTAMPTZ

roles(id, name)
permissions(id, name)          -- e.g. transaction:reverse, kyc:override
role_permissions(role_id, permission_id)
user_roles(user_id, role_id)   -- multi-role: many rows per user

refresh_tokens
  id            UUID PK
  user_id       UUID
  token_hash    TEXT            -- store hash, never raw
  status        ENUM(ACTIVE,ROTATED,REVOKED)
  rotated_from  UUID NULL
  expires_at    TIMESTAMPTZ

outbox          -- transactional outbox for events
  id            UUID PK
  aggregate_type TEXT
  event_type    TEXT
  payload       JSONB
  published     BOOLEAN DEFAULT false
  created_at    TIMESTAMPTZ

```

Money correctness invariants

- Balance = `SUM(credit) - SUM(debit)` over a wallet's ledger entries.
- Available balance = balance - active holds.
- Ledger entries are never updated or deleted; reversals add new entries.
- `idempotency_key` unique constraint prevents duplicate postings.

3.5 Security Design

Authentication — JWT + Refresh Token

- **Access token (JWT):** short TTL (e.g. 10–15 min), signed (asymmetric, RS256/EdDSA). Carries `sub`, `roles`, `permissions` (or a claims ref), `exp`, `jti`. Validated at the **gateway** and re-checked in services.

- **Refresh token:** long-lived, opaque, stored **hashed**. On use →

rotation: old token marked **ROTATED** , new pair issued. **Reuse of a consumed token revokes the whole session** (theft detection — BR8).

- Logout / compromise → revoke all **ACTIVE** refresh tokens for the user.

Authorization — RBAC / Multi-Role

- Roles: **customer, support, operator, compliance, admin, auditor** .
- Users hold **multiple roles**; effective permissions = union of role perms.
- Enforcement middleware on every protected route requires a named permission

(e.g. **transaction:reverse**). Default deny.

- Sensitive actions (**kyc:override** , **transaction:reverse** , **account:suspend**)

always produce an audit record (FR-F4).

Data protection

- PII encrypted at rest; secrets in a vault (never in env files in repos).
- TLS in transit end to end, including service-to-service.
- Principle of least privilege for service DB accounts.

Threat-model highlights (STRIDE-lite)

THREAT	CONTROL
Spoofing	Signed JWT, gateway validation, mTLS option internally
Tampering	Immutable ledger + audit; signed audit export
Repudiation	Full audit trail with actor + trace_id
Info disclosure	Encryption at rest/in transit, least privilege
DoS	Gateway rate limiting + broker backpressure
Elevation	RBAC default-deny, audited privilege checks

3.6 API Design (representative, versioned `/v1`)

```
POST /v1/auth/register      → create customer (PENDING)
POST /v1/auth/login         → { access_token, refresh_token }
POST /v1/auth/refresh       → rotate; new token pair
POST /v1/auth/logout        → revoke session

GET  /v1/customers/{id}      (perm: customer:read | self)
POST /v1/customers/{id}/suspend (perm: account:suspend)

GET  /v1/wallets/{id}        (perm: wallet:read | owner)
POST /v1/wallets/{id}/freeze (perm: wallet:freeze)

POST /v1/transfers           (perm: transfer:create | owner)
  headers: Idempotency-Key: <uuid>
  body: { source_wallet, dest_wallet, amount, currency }
GET  /v1/transactions/{id}   (perm: transaction:read)
POST /v1/transactions/{id}/reverse (perm: transaction:reverse)

POST /v1/kyc/{customerId}/documents (perm: kyc:submit | self)
POST /v1/kyc/{caseId}/decision      (perm: kyc:override)

GET /v1/audit?entity=&actor=&from=&to= (perm: audit:read)
GET /v1/audit/export                  (perm: audit:export)

GET /metrics                         (Prometheus scrape; internal)
```

API conventions: JSON; idempotency on all money-moving POSTs; pagination on lists; standard error envelope `{ code, message, trace_id }`; every response carries the `trace_id` for support.

3.7 Event-Driven Design

3.7.1 Plain-language overview

The idea. When something important happens, the part of the system that did the work records a short note saying *"this happened"* and immediately moves on. Other parts of the system that care about that fact pick up the note and react to it on their own time. The part that did the work never waits for them.

Three terms cover the whole model:

- **Event** — a note that a fact occurred, written in the past tense

(`TransferCompleted` , `KycVerified`). It is plain data: who, what, when,

amounts, and a `trace_id` .

- **Producer** — the component that emits the event (here, the service that just changed state, e.g. Ledger & Transfers).
- **Consumer** — a component that reacts to an event (notifications, audit, analytics, fraud scoring). Each consumer works independently of the others.
- **Broker** — the infrastructure that holds events and delivers them to the interested consumers.

The standard pattern (publish/subscribe with a log). A producer publishes one event; any number of consumers subscribe and each receives its own copy. This is the same shape used across the industry — order systems emitting `OrderPlaced` , streaming platforms emitting `VideoWatched` , logistics systems emitting `ShipmentDispatched` . The producer does not know or care who is listening, which is exactly the property that keeps it decoupled.

What is and is not event-driven here (important). The money movement itself is **not** event-driven. A transfer is a synchronous, immediate database transaction: validate → debit + credit → commit → return the result to the caller. Events are used only for the **after-effects** of a committed fact — notify, audit, analyze, score. We never move money by emitting an event and hoping a consumer handles it.

3.7.2 Why this architecture (rationale)

REASON	WHAT IT BUYS US
Speed / isolation	The core action commits in tens of ms and is not slowed or failed by a downstream (push provider, analytics) being slow or down.
Decoupling / extensibility	New side-effects (receipts, loyalty, reconciliation) are added as new consumers without modifying or redeploying the producer.
Failure isolation	A broken or slow consumer cannot break the core action; its work simply queues until it recovers.
Burst absorption	Traffic spikes queue in the broker instead of overwhelming services; producers and consumers scale independently.
Built-in history	The append-only event stream <i>is</i> an ordered record of what happened — directly useful for the audit/compliance requirement.

3.7.3 The trade-offs (honest costs)

- **Eventual consistency:** anything fed by events (audit view, dashboards, analytics) lags the source of truth by milliseconds-to-seconds. This is acceptable for those surfaces. ****Balances are never read from a lagging projection**** — they come straight from the money database, synchronously.
- **Operational surface:** a broker to run and monitor; consumers must be idempotent; dead-letter handling and replay must exist.
- **Harder debugging:** there is no single stack trace across an async boundary, which is why a `trace_id` is propagated through every event and log.
- **When it would be overkill:** if there were a single side-effect and no audit requirement, a direct in-process call would be correct and a broker would be over-engineering. The pattern is justified here by **multiple independent consumers + a hard audit requirement + bursty load + a core path that must stay fast**.
- **Minimum-viable form:** the outbox table + a simple relay + one or two consumers already delivers the decoupling and the history. A full

high-throughput broker is adopted only when consumer count and volume demand it — not on day one.

3.7.4 Mechanics

- **Transactional outbox:** domain changes and their events are written in the **same DB transaction** (outbox table). A relay publishes to the broker → no

lost events, no dual-write inconsistency.

- **Events (examples):** `CustomerRegistered` , `KycVerified` , `KycRejected` , `TransferCompleted` , `TransferFailed` , `TransactionReversed` , `AccountSuspended` .
 - **Delivery:** at-least-once. **Consumers are idempotent** (dedup by event id)
- effectively-once processing (NFR10).
- **Consumers:** notifications, audit sink, analytics, fraud hooks (stub).
 - **Ordering:** per-aggregate ordering via partition key (e.g. wallet id).

3.8 API Gateway & Rate Limiting

- Single ingress: TLS, JWT validation, routing, request/trace ID injection.
- **Rate limiting** strategies (configurable, layered):
- Per IP (coarse, anti-abuse).
- Per authenticated identity (fair use).
- Per endpoint (protect expensive/money-moving routes).
- Algorithm: token bucket / sliding window; limits tunable per policy.
- On limit breach → `429 Too Many Requests` with `Retry-After` ; event/metric

emitted for monitoring (avoid false-positive lockouts — R8).

3.9 Observability Design

PILLAR	IMPLEMENTATION
Metrics	Each service exposes <code>/metrics</code> ; Prometheus scrapes. Golden signals (latency, traffic, errors, saturation) + business KPIs (transfers/min, KYC pass rate, balance totals).
Dashboards	Grafana boards: platform health, money-movement, KYC funnel, auth/refresh anomalies, rate-limit hits.
Tracing	Distributed tracing : trace context created at the gateway, propagated through every service to DB/broker; spans correlate with audit <code>trace_id</code> and logs.
Logging	Structured JSON logs, correlated by <code>trace_id</code> / <code>request_id</code> .
Alerting	SLO-based alerts (error budget burn, latency, failed transfers, dead-letter growth).

Correlation: one `trace_id` ties together gateway log → service spans → audit record → metric exemplar. MTTR target < 5 min (O5).

3.10 Deployment & Environments

- Containerized services on an orchestrator; stateless services autoscale.
- Environments: `dev → staging → prod` ; identical config shape, secrets per env.
- Database: managed RDBMS with **PITR**; read replicas for reporting.
- CI/CD: build → test (unit, ledger-invariant, contract) → security scan →

deploy with health gates and easy rollback.

- DR: RPO ≤ 5 min, RTO ≤ 1 hr for core services (NFR12).

3.11 Key Design Decisions (ADR summary)

#	DECISION	RATIONALE
1	Double-entry ledger, balance derived	Auditability + no balance-edit bugs (BR4, BR6)
2	Idempotency keys on money-moving POSTs	Prevent duplicate charges on retries (BR5)
3	Transactional outbox for events	Avoid dual-write loss/inconsistency (NFR10)
4	Short JWT + rotating refresh tokens	Limit blast radius of token theft (BR8)
5	RBAC default-deny + audited privileged ops	Least privilege + traceability (FR-H, FR-F)
6	Shared ledger DB, async fan-out via events	Strong money consistency, loose coupling elsewhere
7	NUMERIC money, never float	Avoid rounding errors in financial math
8	Trace_id stitched into audit records	One-click incident correlation (O5)

3.12 Success Criteria (exit gate for Design)

- System architecture and service boundaries defined.
- Physical data model with money-correctness invariants specified.
- Security design (JWT+refresh, RBAC, threat model) complete.
- API contracts, event design, gateway/rate-limiting designed.
- Observability (Prometheus, Grafana, tracing) designed and correlated.
- Key decisions recorded as ADRs.

➡ Design baseline ready for **Build (M3+)**.

Diagrams

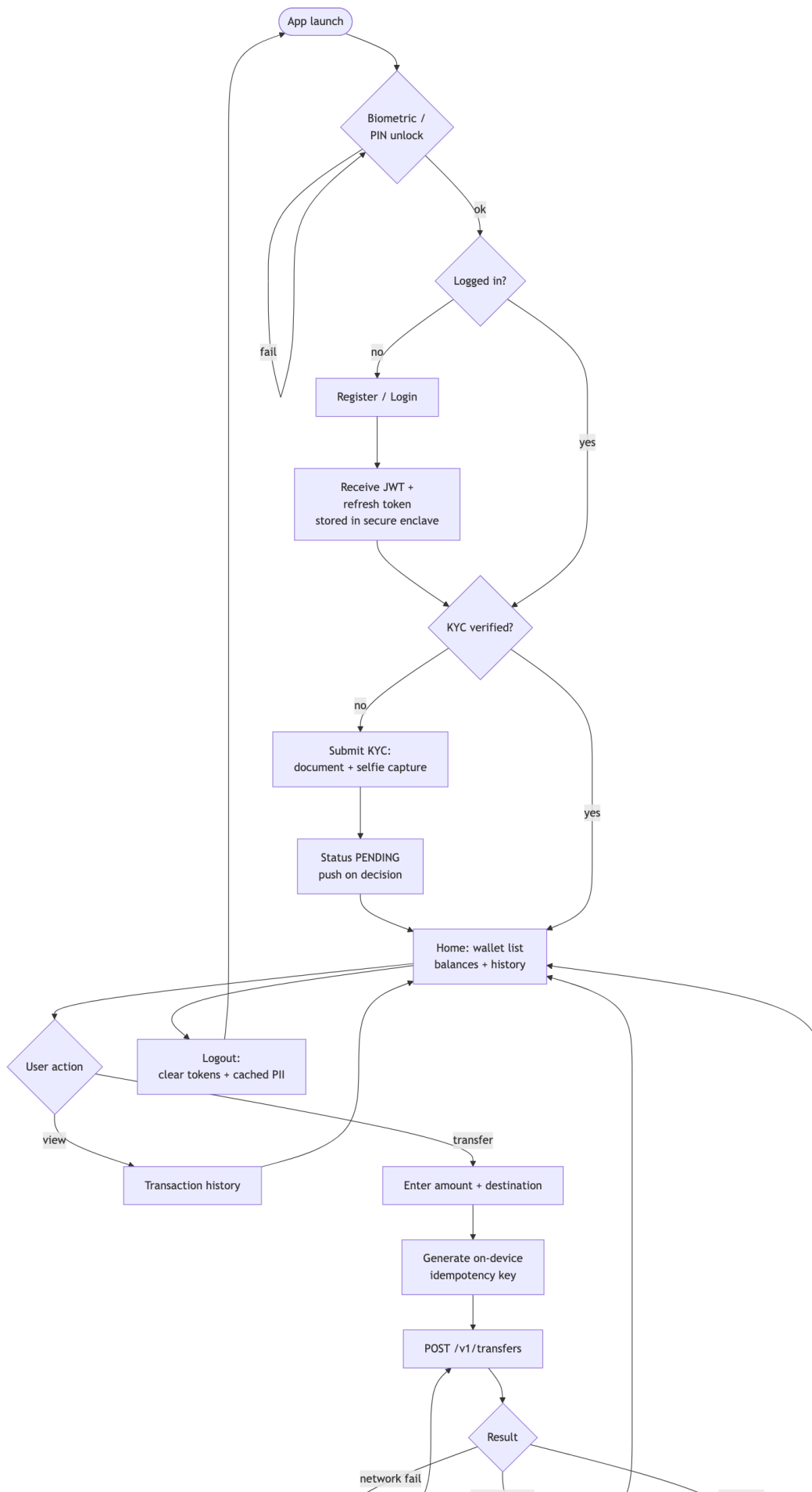
Digital Banking Platform · ERD + Mobile & Dashboard Flows Rendered images live in [diagrams/](#) .
Mermaid source kept below each image.

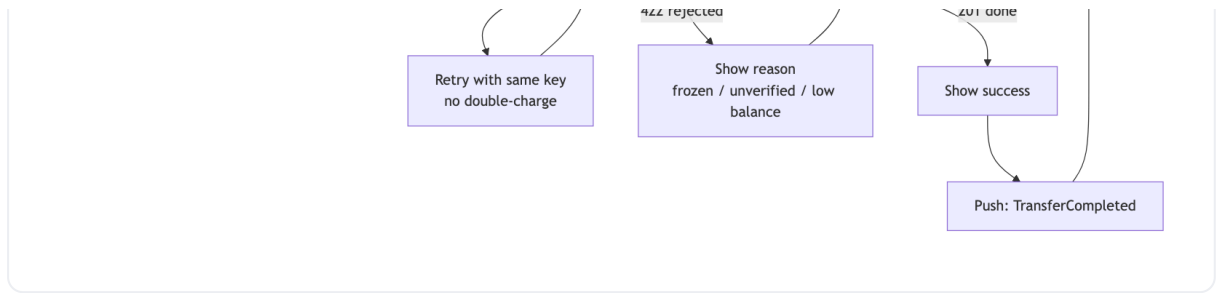
To re-render after editing any `.mmd` :

```
npx -y @mermaid-js/mermaid-cli -i diagrams/src/erd.mmd -o diagrams/erd.png -b white -s 2
```

A. Mobile App Flow

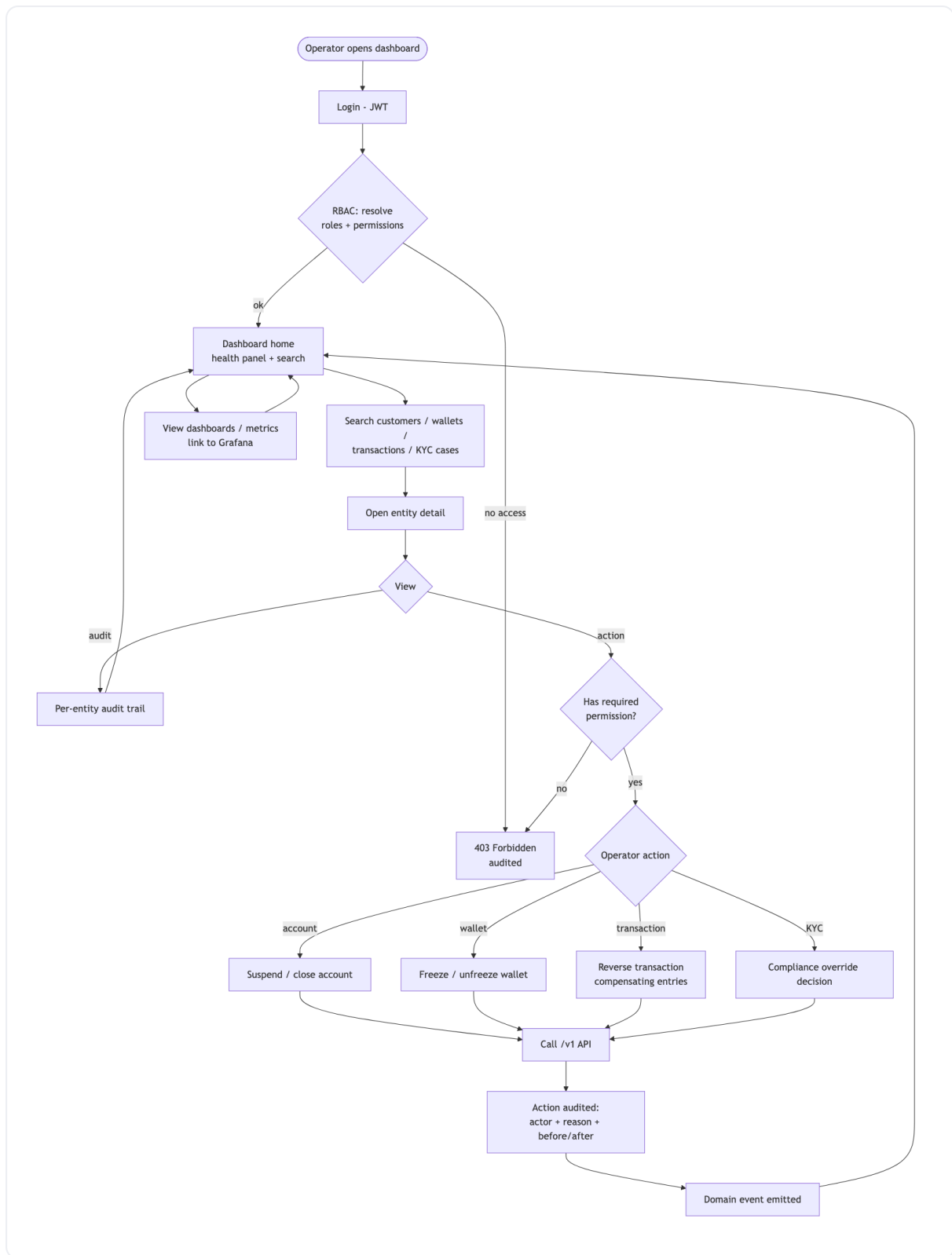
End-user journey through the React Native app — unlock, onboarding, KYC, wallets, and the transfer flow with on-device idempotency.



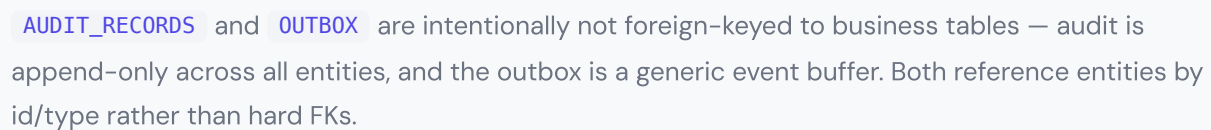


B. Admin Dashboard Flow

Operator journey through the React console — RBAC-gated search, entity views, and audited operator actions.



C. Entity Relationship Diagram (ERD)



Phase 4 — Implementation / Sprint Plan

Digital Banking Platform · SDLC Phase 4 of N Status: Delivery Plan · Owner: Tech Lead + Product Owner + Delivery Lead

4.1 Purpose

Turn the approved Design (Phase 3) into an executable delivery plan: a prioritized product backlog, a milestone map, a sprint-by-sprint breakdown, the team's working agreements (Definition of Ready / Done), and the tracking metrics that tell us whether we're on course. The goal of M3–M7 from Planning is a production launch in a single region in ~12–14 build weeks (6–7 two-week sprints) with a small senior team.

4.2 Delivery Approach

- **Cadence:** 2-week sprints, Scrum-lite (planning, daily sync, review, retro).
- **Branching:** trunk-based with short-lived feature branches; PR + 1 review + green CI to merge. No direct pushes to `main`.
- **Vertical slices:** each story delivers an end-to-end, demoable increment (API + persistence + tests + metrics), never a horizontal layer alone.
- **Walking skeleton first:** stand up gateway → service → DB → CI/CD on day one so every later story plugs into a working pipeline.
- **Compliance/observability are acceptance criteria**, not separate stories — a money-moving story isn't "done" without audit + metrics + tests.

4.3 Estimation Scale

Story points use a modified Fibonacci scale: **1, 2, 3, 5, 8, 13**.

- 1–2: trivial, well understood.
- 3–5: normal story, some unknowns.
- 8: large or risky; consider splitting.
- 13: too big — **must** be split before entering a sprint.

Indicative team velocity assumption: **~24–28 pts/sprint** for a squad of

~4 engineers. Treat as a forecast, recalibrate after Sprint 2.

4.4 Epics → Milestones

EPIC	TITLE	MAPS TO MILESTONE	MAPS TO REQUIREMENTS
E0	Platform skeleton & CI/CD	M3	NFR7, NFR11
E1	AuthN/AuthZ (JWT + refresh + RBAC)	M4	FR-A2, FR-H, NFR1
E2	Accounts & Wallets	M3	FR-A, FR-B
E3	Ledger, Transactions & Transfers	M3	FR-C, FR-D, NFR2, NFR10
E4	KYC workflow	M4	FR-E, NFR8
E5	Audit trail	M4	FR-F, NFR6
E6	Event backbone (outbox + consumers)	M5	FR-I3, FR-I4, NFR10
E7	API Gateway & rate limiting	M5	FR-I1, FR-I2
E8	Observability (metrics/Grafana/tracing)	M5	FR-I5, FR-I6, FR-I7, NFR7
E9	Admin dashboard	M4/M5	FR-G
E11	Customer mobile app (iOS + Android)	M4/M5	FR-A, FR-B, FR-D, FR-E, NFR1
E10	Hardening, security review, launch	M6/M7	NFR1, NFR3, NFR12

4.5 Product Backlog (prioritized)

Legend — Priority: PO (must, blocks others) · P1 (core) · P2 (important) ·

P3 (nice). Pts = story points.

E0 · Platform skeleton & CI/CD

ID	STORY	PRI	PTS
S-001	Repo, service scaffolding, containerization	PO	3
S-002	CI pipeline: build, lint, unit tests, security scan	PO	5
S-003	CD to staging with health gates + rollback	PO	5
S-004	Managed RDBMS provisioned with PITR; migration tooling	PO	3
S-005	Walking skeleton: gateway → sample service → DB round-trip	PO	3

E1 · AuthN/AuthZ

ID	STORY	PRI	PTS
S-010	User credential store + registration endpoint	PO	3
S-011	Login issues signed JWT (short TTL)	PO	5
S-012	Refresh token issue + rotation + hashed storage	PO	5
S-013	Refresh-reuse detection → session revocation (BR8)	P1	3
S-014	RBAC model: roles, permissions, user-roles (multi-role)	PO	5
S-015	Authorization middleware (default-deny, per-permission)	PO	5
S-016	Logout / revoke-all-sessions	P2	2

E2 · Accounts & Wallets

ID	STORY	PRI	PTS
S-020	Customer entity + lifecycle states	P0	3
S-021	Create/read customer; profile update (KYC-locked fields)	P1	3
S-022	Wallet entity + lifecycle (ACTIVE/FROZEN/CLOSED)	P0	3
S-023	Balance read derived from ledger	P0	5
S-024	Admin suspend/close account (RBAC + audit)	P1	3
S-025	Freeze/unfreeze wallet (RBAC + audit)	P1	2

E3 · Ledger, Transactions & Transfers

ID	STORY	PRI	PTS
S-030	Double-entry ledger schema + append-only invariant	P0	5
S-031	Post transaction atomically (DB tx, debit+credit sum=0)	P0	8
S-032	Idempotency-key enforcement on money moves (BR5)	P0	5
S-033	Insufficient-funds + available-balance (holds) checks	P0	5
S-034	Wallet-to-wallet transfer (paired debit/credit)	P0	8
S-035	Transaction reversal via compensating entries (RBAC + audit)	P1	5
S-036	Holds: HOLD / RELEASE lifecycle	P2	5

E4 · KYC

ID	STORY	PRI	PTS
S-040	KYC case entity + state machine	P0	3
S-041	Document submission endpoint	P1	3
S-042	Vendor integration (async submit + webhook/poll)	P1	8
S-043	Sanctions/PEP screening hook	P1	5
S-044	KYC-gated money-out + tiered limits (BR3)	P0	5
S-045	Compliance override decision (RBAC + audit)	P2	3

E5 • Audit trail

ID	STORY	PRI	PTS
S-050	Append-only audit store (actor, action, before/after, trace_id)	P0	5
S-051	Auto-audit on all privileged/state-changing actions	P0	5
S-052	Audit query API (by entity/actor/time/action)	P1	3
S-053	Signed, tamper-evident export	P2	5

E6 • Event backbone

ID	STORY	PRI	PTS
S-060	Broker provisioned; topic/partition design (per-aggregate order)	P0	3
S-061	Transactional outbox table + relay publisher	P0	8
S-062	Idempotent consumers (dedup by event id)	P0	5
S-063	Notification consumer	P2	3
S-064	Dead-letter handling + replay	P1	5

E7 • Gateway & rate limiting

ID	STORY	PRI	PTS
S-070	API gateway: TLS, routing, JWT validation at edge	PO	5
S-071	Request/trace-ID injection + propagation	PO	3
S-072	Rate limiting (per IP / identity / endpoint), 429 + Retry-After	P1	5
S-073	Rate-limit metrics + tunable policy config	P2	3

E8 · Observability

ID	STORY	PRI	PTS
S-080	<code>/metrics</code> on every service (golden signals)	PO	5
S-081	Prometheus scrape + retention	PO	3
S-082	Grafana dashboards (platform + money + KYC funnel + auth anomalies)	P1	5
S-083	Distributed tracing gateway → services → DB/broker	P1	8
S-084	SLO alerts (error budget, latency, failed transfers, DLQ growth)	P1	5

E9 · Admin dashboard

ID	STORY	PRI	PTS
S-090	Dashboard shell + auth (RBAC-aware UI)	P1	5
S-091	Search customers/wallets/transactions/KYC	P1	5
S-092	Per-entity audit trail view	P2	3
S-093	Operator actions (suspend/freeze/reverse/override) wired to APIs	P1	5
S-094	Health panel linking Grafana	P3	2

E11 · Customer mobile app (iOS + Android)

ID	STORY	PRI	PTS
S-110	App scaffold, navigation, design system, CI build (both platforms)	PO	5
S-111	Auth: login + secure refresh-token storage (Keychain/Keystore)	PO	5
S-112	Registration + onboarding flow	P1	5
S-113	KYC document + selfie capture and upload	P1	8
S-114	Wallet list + balance + transaction history	P1	5
S-115	Transfer flow with on-device idempotency key + retry	PO	8
S-116	Push notifications (TransferCompleted / KycVerified)	P2	5
S-117	Biometric unlock + certificate pinning + root/jailbreak signal	P1	5
S-118	Offline-tolerant read cache + backoff on writes	P2	5
S-119	App store submission + release pipeline	P1	3

E10 · Hardening & launch

ID	STORY	PRI	PTS
S-100	Threat model review + fixes	PO	5
S-101	Penetration test + remediation	PO	8
S-102	Load/soak test to 10x; tune autoscaling	P1	5
S-103	DR drill: PITR restore, validate RPO≤5m / RTO≤1h	P1	5
S-104	Runbooks, on-call, alert routing	P1	3
S-105	Compliance sign-off + audit-export validation	PO	3
S-106	Production cutover + smoke tests	PO	3

Backlog total: ~70 stories, ≈ 324 points (incl. the mobile track, E11).

4.6 Sprint Breakdown (forecast)

Forecast only — re-baseline after Sprint 2 actuals. Each sprint ends with a demoable increment.

Sprint 1 — Walking skeleton + identity foundation

Goal: a request can flow gateway → service → DB in CI/CD; users can register and log in.

Stories: S-001, S-002, S-003, S-004, S-005, S-010, S-011 · ~27 pts

Demo: deploy to staging; register + login returns a JWT.

Sprint 2 — Accounts, wallets, RBAC

Goal: customers and wallets exist; access is permission-gated.

Stories: S-012, S-014, S-015, S-020, S-022, S-023 · ~26 pts

Demo: create customer + wallet; protected endpoint enforces RBAC.

Sprint 3 — The ledger (core money)

Goal: money can be posted correctly and idempotently.

Stories: S-030, S-031, S-032, S-033, S-050 · ~28 pts

Demo: post transactions; double-entry invariant + idempotency proven by tests.

Sprint 4 — Transfers, audit, events

Goal: end-to-end transfer with audit and an event emitted via outbox.

Stories: S-034, S-035, S-051, S-060, S-061 · ~29 pts

Demo: wallet-to-wallet transfer → balances move, audit written, event published.

Sprint 5 — KYC + gateway + consumers

Goal: onboarding gated by KYC; traffic enters via gateway; events consumed.

Stories: S-040, S-042, S-044, S-070, S-062 · ~29 pts

Demo: unverified user blocked from money-out; verified user transfers; consumer reacts.

Sprint 6 — Observability + admin + rate limiting

Goal: the platform is observable and operable.

Stories: S-080, S-082, S-083, S-072, S-090, S-091 · **~30 pts** (*split if needed*)

Demo: Grafana shows golden signals + a traced transfer; admin searches entities.

Sprint 7 — Hardening + launch

Goal: production-ready.

Stories: S-100, S-101, S-102, S-103, S-105, S-106 · **~27 pts**

Demo: pen-test clean, load target met, DR drill passes, compliance sign-off, cutover.

Parallel track — Mobile app (E11)

The mobile app runs as a **parallel workstream** alongside the backend, staffed

by a dedicated mobile sub-squad. It consumes the API contracts as they

stabilize, so it trails the backend by ~1 sprint per capability:

- **MT-1 (≈ backend Sprint 2–3):** S-110 scaffold/CI, S-111 secure auth — built

against the auth API once it's live.

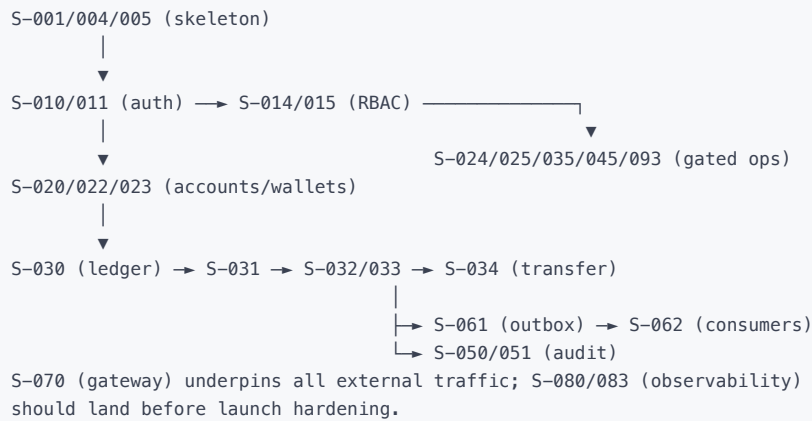
- **MT-2 (≈ Sprint 4):** S-112 onboarding, S-114 wallets/history.
- **MT-3 (≈ Sprint 5):** S-115 transfers (on-device idempotency), S-113 KYC

capture — follow the transfer + KYC APIs.

- **MT-4 (≈ Sprint 6):** S-116 push, S-117 biometric/pinning, S-118 offline.
- **MT-5 (≈ Sprint 7):** S-119 app-store submission, hardening, store review.

Deferred to fast-follow (post-launch): S-013, S-016, S-021, S-024, S-025, S-036, S-041, S-043, S-045, S-052, S-053, S-063, S-064, S-071, S-073, S-081, S-084, S-092, S-093, S-094, S-104. Pull forward by priority as velocity allows.

4.7 Critical Path & Dependencies



Hard rule: nothing money-moving (S-031+) merges before the ledger invariant

tests (double-entry sum=0, idempotency) are green.

4.8 Definition of Ready (DoR)

A story may enter a sprint only when:

- Clear, testable acceptance criteria.
- Dependencies identified and unblocked (or stubbed).
- API contract / schema change sketched.
- Security & audit implications noted.
- Estimable and ≤ 8 pts (split if larger).

4.9 Definition of Done (DoD)

A story is done only when:

- Code merged via PR with ≥ 1 review and green CI.
- Unit + integration tests written and passing.
- Money-path stories: ledger-invariant + idempotency tests included.
- AuthZ enforced; privileged actions emit audit records.
- `/metrics` updated; traces span the new path.
- API docs / contract updated; versioned.
- Deployed to staging and demoed.

- ¶ No new critical/high security findings.

4.10 Roles & Ceremonies

CEREMONY	CADENCE	OUTPUT
Sprint Planning	Start of sprint	Committed sprint backlog
Daily Sync	Daily, 15 min	Blockers surfaced
Backlog Refinement	Mid-sprint	Next sprint's stories meet DoR
Sprint Review	End of sprint	Demoed increment, stakeholder feedback
Retrospective	End of sprint	1–3 process improvements

4.11 Tracking Metrics

METRIC	PURPOSE	HEALTHY SIGNAL
Velocity (pts/sprint)	Forecasting	Stable $\pm 15\%$ after Sprint 2
Sprint goal hit rate	Predictability	$\geq 80\%$
Burndown	In-sprint progress	Trends to zero, no cliff
Escaped defects	Quality	Near zero on money paths
PR cycle time	Flow efficiency	< 1 day median
CI pass rate	Pipeline health	$> 90\%$ on first run
Test coverage (core)	Safety on ledger/auth	High on money + auth modules

4.12 Delivery Risks (delta to Planning register)

ID	RISK	MITIGATION
DR1	Ledger story (S-031) underestimated	Spike early in Sprint 2; pair on it; invariant tests
DR2	KYC vendor integration latency (S-042)	Start vendor onboarding in Sprint 1; mock first
DR3	Velocity unknown until Sprint 2	Forecast, don't commit dates downstream until S2
DR4	Observability deferred → blind during hardening	S-080/083 are PO/P1, scheduled before Sprint 7
DR5	Scope creep from deferred list	Strict DoR gate; product owner guards the backlog

4.13 Success Criteria (exit gate for Phase 4 planning)

- ☑ Epics mapped to milestones and requirements.
- ☑ Prioritized backlog with estimates.
- ☑ Sprint-by-sprint forecast with goals and demos.
- ☑ Critical path, dependencies, DoR/DoD, ceremonies, and metrics defined.
- ☑ Delivery-specific risks captured.

➡ Execution begins at **Sprint 1**; re-baseline the forecast after Sprint 2.

Phase 5 — Testing Strategy & Quality Assurance

Digital Banking Platform · SDLC Phase 5 of N Status: Test Plan · Owner: QA Lead + Tech Lead + Security

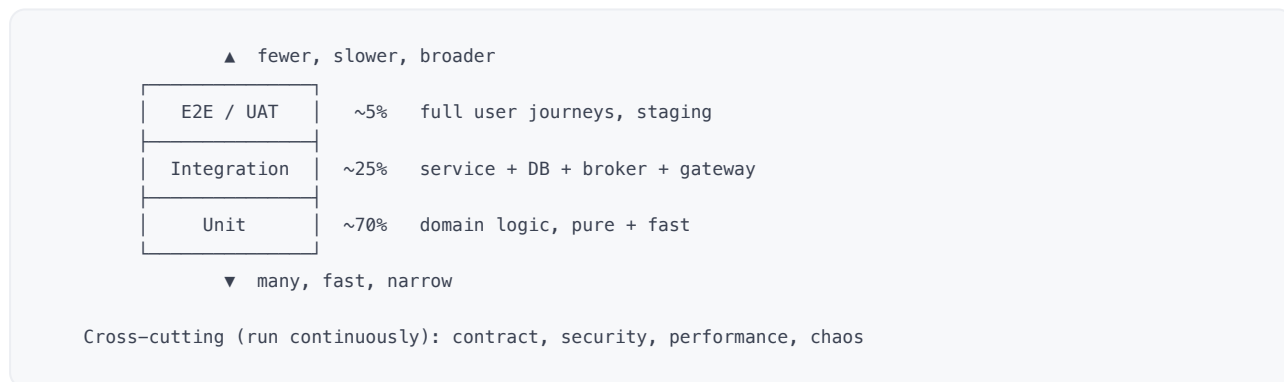
5.1 Purpose

Define how we prove the platform is correct, secure, and operable before it touches real money. In a regulated fintech the cost of a defect is asymmetric — a single ledger bug can mint or destroy funds — so testing is treated as a first-class deliverable with hard gates, not an afterthought. This document sets the test pyramid, environments, coverage targets, concrete test cases for every critical flow, non-functional test plans, and the exit criteria that block a release.

5.2 Quality Objectives

ID	OBJECTIVE	TARGET / GATE
Q1	Money correctness	0 ledger-invariant violations, ever
Q2	No duplicate money movement	Idempotency proven under concurrency
Q3	Access control enforced	0 unauthorized-access escapes
Q4	Audit completeness	100% of state changes audited
Q5	Performance within SLO	Meets NFR4 latency under target load
Q6	Security posture	0 critical/high vulns at launch
Q7	Resilience	Survives broker/DB/vendor outage tests
Q8	Regression safety	Core paths covered, CI green to merge

5.3 Test Pyramid & Mix



- **Unit (~70%):** domain rules in isolation — balance math, state machines,

RBAC permission resolution, token rotation logic. Milliseconds, no I/O.

- **Integration (~25%):** real DB + broker + gateway wired together; the

money-movement and event paths live here.

- **E2E (~5%):** a handful of full journeys against a deployed staging stack.
- **Continuous:** contract tests (API/event schemas), security scans,

performance and chaos suites on a schedule + pre-release.

5.4 Test Types & Ownership

TYPE	WHAT IT PROVES	OWNER	WHEN
Unit	Domain logic correctness	Engineers	Every commit/CI
Integration	Services interact correctly with infra	Engineers/QA	Every CI run
Contract	API + event schemas stay backward-compatible	Engineers	Every CI run
Property-based	Invariants hold over random inputs	Engineers	Every CI run
E2E	User journeys work end to end	QA	Per merge to main / nightly
Performance	Latency + throughput meet SLO	QA/SRE	Pre-release + weekly
Security	No exploitable vulns; authz holds	Security/QA	CI + pre-release
Chaos/Resilience	Graceful degradation under failure	SRE	Pre-release
UAT	Stakeholders accept the behavior	Product	Before launch
DR	RPO/RTO met on restore	SRE	Pre-release

5.5 Test Environments & Data

ENV	PURPOSE	DATA
Local	Dev inner loop, unit + container	Ephemeral, seeded fixtures
CI	Automated unit/integration/contract	Ephemeral containers, factory data
Staging	E2E, performance, UAT, security	Synthetic, prod-like volume, no real PII
Prod	Live; smoke tests only	Real (read-only verification)

Data rules

- No production PII in lower environments — use synthetic/anonymized data.
- KYC vendor and external rails are **mocked** in CI/local; a vendor **sandbox**

is used in staging.

- Every test that moves money runs in an isolated DB transaction or a fresh schema; tests must be independent and order-agnostic.

5.6 Critical Test Cases

Format: ID · Pre-condition → Action → Expected. PO cases are release blockers.

TC-A · Ledger & Money Correctness (PO)

ID	SCENARIO
TC-A1	Post transaction → SUM(credits) == SUM(debits) for that transaction (=0).
TC-A2	Debit > available balance → rejected; no ledger rows written.
TC-A3	Wallet balance == SUM(credit) - SUM(debit) over its entries, always.
TC-A4	Ledger entry is immutable: UPDATE/DELETE attempt is rejected/blocked.
TC-A5	Available balance excludes active holds (balance - holds).
TC-A6	Reversal posts compensating entries; net effect == 0; original intact.
TC-A7	Money uses fixed-precision decimal; 0.1+0.2 style rounding never drifts.
TC-A8	Negative or zero amount transfer → rejected by validation.

TC-B · Idempotency & Concurrency (PO)

ID	SCENARIO
TC-B1	Same Idempotency-Key twice → exactly one posting; 2nd returns 1st result.
TC-B2	100 concurrent transfers from one wallet → no overspend, no lost update.
TC-B3	Two transfers racing on the same balance → serialized, balance correct.
TC-B4	Retry after timeout (client unsure) → no double charge.
TC-B5	Idempotency key expires after window → new key required for replay.

TC-C · AuthN / Tokens (PO/P1)

ID	SCENARIO
TC-C1	Login with valid creds → access JWT + refresh token issued.
TC-C2	Expired access token → 401; refresh succeeds → new pair.
TC-C3	Refresh rotation: old refresh token marked ROTATED, unusable.
TC-C4	Reuse of a consumed refresh token → entire session revoked (BR8).
TC-C5	Tampered/invalid JWT signature → 401, no access.
TC-C6	Logout → all active refresh tokens for the user revoked.

TC-D · Authorization / RBAC (PO)

ID	SCENARIO
TC-D1	Customer calls admin-only endpoint → 403, action audited.
TC-D2	Operator without <code>transaction:reverse</code> → 403 on reversal.
TC-D3	Multi-role user gets union of permissions; each enforced.
TC-D4	Default-deny: endpoint with no granted permission → 403.
TC-D5	Auditor (read-only) cannot mutate any resource.
TC-D6	Privilege-escalation attempt (forged role claim) → denied + audited.

TC-E · KYC (PO/P1)

ID	SCENARIO
TC-E1	New customer is NOT_STARTED; money-out blocked (BR3).
TC-E2	Submit docs → PENDING; vendor VERIFIED → account ACTIVE, wallet created.
TC-E3	Vendor REJECTED → routed to compliance queue; money-out stays blocked.
TC-E4	Sub-limit tier: unverified can transact up to configured cap only.
TC-E5	Sanctions/PEP hit → flagged, blocked, compliance review.
TC-E6	Compliance override → state changes, fully audited with actor + reason.

TC-F · Audit Trail (PO)

ID	SCENARIO
TC-F1	Every state change writes actor, action, before/after, timestamp.
TC-F2	Audit record is append-only; edit/delete attempt rejected.
TC-F3	Audit record carries the request trace_id (correlation works).
TC-F4	Privileged actions (suspend/freeze/reverse/override) always audited.
TC-F5	Audit export is signed/tamper-evident; tampering detectable.

TC-G · Transfers End-to-End (PO)

ID	SCENARIO
TC-G1	Verified A → verified B transfer: balances move, event emitted, audited.
TC-G2	Source wallet FROZEN → transfer rejected, no balance change.
TC-G3	Source KYC not VERIFIED → rejected (BR3).
TC-G4	Insufficient funds → 422, TransferFailed event + audit, no movement.
TC-G5	Transfer to CLOSED/nonexistent destination → rejected.

TC-H · Events / Outbox (P0/P1)

ID	SCENARIO
TC-H1	Outbox row written in the same DB tx as the ledger change (atomic).
TC-H2	DB commits but relay crashes → event still published on recovery.
TC-H3	Duplicate delivery → idempotent consumer processes effectively once.
TC-H4	Poison message → dead-lettered after retries; replayable.
TC-H5	Per-aggregate ordering preserved (same wallet's events in order).

TC-I · Gateway & Rate Limiting (P1)

ID	SCENARIO
TC-I1	Unauthenticated request to protected route → 401 at the gateway.
TC-I2	Over-limit client → 429 with Retry-After; metric incremented.
TC-I3	Limit is per-identity/endpoint; one abuser doesn't block others.
TC-I4	trace_id injected at edge and propagated to downstream services.

TC-M · Mobile App (P0/P1)

ID	SCENARIO
TC-M1	Refresh token stored in Keychain/Keystore, never in plain storage.
TC-M2	Transfer retried over flaky network reuses on-device idempotency key → no double charge.
TC-M3	Access token expiry → silent refresh + rotation; user not logged out.
TC-M4	Biometric unlock gates app open; failure blocks access to balances.
TC-M5	Certificate pinning: MITM/proxy with bad cert → connection refused.
TC-M6	KYC capture uploads documents; PENDING state reflected in UI.
TC-M7	Offline → cached balances shown read-only; writes queued + retried.
TC-M8	Push on TransferCompleted / KycVerified delivered and deep-links.
TC-M9	Logout clears tokens + cached PII from the device.

5.7 Non-Functional Test Plans

Performance / Load (NFR4, NFR5)

- **Load test:** sustained target RPS; assert p95 core reads < 300 ms, transfers < 800 ms p95.
- **Stress test:** ramp to 10x; find the knee, confirm graceful degradation (429s, queueing) rather than data corruption or crashes.
- **Soak test:** target load for 8–24h; watch for memory leaks, connection-pool exhaustion, DLQ growth.
- **Spike test:** sudden burst (e.g. campaign) → broker absorbs, no lost events.

Security (NFR1, NFR9, Q6)

- **SAST** on every CI run; **dependency/SCA** scan for known CVEs.
- **DAST** against staging; **penetration test** before launch (S-101).

- AuthN/AuthZ abuse cases: token theft/replay, IDOR (access another user's wallet), privilege escalation, JWT tampering, injection.
- Secrets scanning in CI (no keys in repo); verify PII encrypted at rest.

Resilience / Chaos (NFR3, NFR10, Q7)

- Kill a service instance mid-transfer → no partial postings, retry succeeds.
- Broker down → outbox buffers; events flush on recovery (TC-H2).
- KYC vendor timeout/outage → async retry, manual-review fallback, no hang.
- DB failover → in-flight tx rolls back cleanly; balances consistent after.

Disaster Recovery (NFR12)

- PITR restore drill → validate **RPO ≤ 5 min, RTO ≤ 1 hr**; reconcile ledger post-restore (sum invariants still hold).

Observability validation (NFR7)

- Assert `/metrics` exposes golden signals; a traced transfer shows a complete span chain gateway → service → DB/broker; alerts fire on injected failures.

5.8 Test Data & Fixtures

- **Factories/builders** for customers, wallets, transactions — no shared mutable global state.
 - **Golden ledger fixtures:** known opening balances + a sequence of operations with a precomputed expected end state (regression anchor).
 - **Mocks/stubs:** KYC vendor, sanctions API, external rails — with both happy-path and failure responses.
 - **Seed personas:** unverified customer, verified customer, frozen wallet, compliance officer, operator, auditor, admin — one per RBAC role.
-

5.9 Automation & CI Gates

```
PR opened → lint → unit → integration → contract → SAST/SCA → coverage gate
                                                    |
all green + 1 review —————→ merge
merge to main → deploy staging → E2E + smoke → nightly: perf + security + chaos
release candidate → pen test + load + DR drill + UAT + compliance sign-off → prod
```

Hard merge gates (CI blocks the PR if any fail):

- All unit + integration + contract tests pass.
- Coverage thresholds met on **money + auth modules** (high bar there even if global bar is lower).
- No new critical/high SAST or dependency findings.
- Money-path changes include ledger-invariant + idempotency tests (enforced by review checklist + targeted required tests).

5.10 Coverage Targets

AREA	TARGET	RATIONALE
Ledger / transactions / transfers	≥ 95% lines + branches	Money correctness is non-negotiable
Auth / RBAC	≥ 95%	Security-critical
KYC / audit	≥ 90%	Compliance-critical
Gateway / rate limiting	≥ 85%	Important, lower blast radius
Admin dashboard / UI	≥ 70%	Lower risk, E2E covers journeys
Global	≥ 80%	Baseline

Coverage is a floor, not a goal — a green % with weak assertions is worse than useless. Mutation testing on the ledger module is recommended to verify the tests actually catch faults.

5.11 Defect Management

SEVERITY	DEFINITION	ACTION
S1 Critical	Money loss/creation, breach, data loss	Stop-the-line; hotfix; no launch
S2 High	Core flow broken, no safe workaround	Fix before release
S3 Medium	Degraded but workaroundable	Schedule into a sprint
S4 Low	Cosmetic / minor	Backlog

Every S1/S2 gets a regression test added before closure (no silent fixes).

5.12 Traceability (requirement/use-case → tests)

REQUIREMENT / RULE	COVERED BY
BR1/BR2 balance + sufficiency	TC-A2, TC-A3, TC-G4
BR4 immutable ledger	TC-A4, TC-F2
BR5 idempotency	TC-B1, TC-B4, TC-B5
BR6 double-entry sum=0	TC-A1, TC-A6
BR8 refresh rotation/theft	TC-C3, TC-C4
FR-H RBAC	TC-D1...TC-D6
FR-E KYC + BR3 gating	TC-E1...TC-E6, TC-G3
FR-F audit	TC-F1...TC-F5
FR-D transfers (UC-2)	TC-G1...TC-G5
FR-I3/4 events/outbox	TC-H1...TC-H5
FR-I1/2 gateway + rate limit	TC-I1...TC-I4
Mobile app (E11)	TC-M1...TC-M9
NFR4/5 performance	§5.7 Performance
NFR1/9 security	§5.7 Security
NFR3/10 resilience	§5.7 Resilience, TC-H2
NFR12 DR	§5.7 DR

5.13 Release Exit Criteria (Go/No-Go for launch)

- All PO test cases pass; no open S1/S2 defects.
- Coverage targets met on money + auth + KYC + audit modules.
- Performance SLOs met at target load; stress knee understood.
- Penetration test complete; no open critical/high findings.

- ☐ Chaos suite passed (service/broker/vendor/DB failure scenarios).
- ☐ DR drill passed (RPO ≤ 5 min, RTO ≤ 1 hr); ledger reconciles post-restore.
- ☐ Audit completeness verified; signed export validated.
- ☐ Observability validated (metrics, traces, alerts firing correctly).
- ☐ UAT signed off by Product; compliance sign-off recorded.

➡ When all boxes are checked, proceed to **production cutover (S-106)**.

Tech Stack & Architecture Style

Digital Banking Platform · Technology Decisions Status: Baseline · Owner: Architect + Tech Lead

7.1 Chosen Technologies

LAYER	TECHNOLOGY	ROLE
Backend API	Java + Spring Boot	Core-banking services, REST API, business logic, ledger.
Admin Dashboard	React	Operator web console (RBAC-gated).
Mobile App	React Native	Customer iOS + Android app (single codebase).
Database	PostgreSQL (recommended)	System of record for the money core.
Message broker	RabbitMQ or Kafka	Event delivery (start with RabbitMQ; Kafka if scale needs).
Auth	JWT (access) + refresh tokens	Stateless authentication.
Observability	Prometheus + Grafana + tracing	Metrics, dashboards, distributed tracing.

These three client/runtime choices (Spring Boot, React, React Native) are fixed.

The database, broker, and observability tools are recommendations consistent with the design and can be swapped for equivalents.

7.2 The Real Question: Microservices or Not?

A common confusion is to treat **"event-driven"** and **"microservices"** as the

same decision. They are not. They are two independent choices:

- **Event-driven** = *how components talk* — by emitting/reacting to events

(asynchronous) instead of direct synchronous calls.

- **Microservices** = *how the code is deployed* — many small, independently

deployed services, each owning its own database.

You can mix and match all four combinations. The most important point:

You can be fully event-driven inside a single application. Microservices are **not** required to get the benefits of event-driven design.

The two honest options for this project

Option A — Modular Monolith (recommended to start).

One Spring Boot application, internally split into clean modules (auth, accounts, wallets, ledger, transfers, kyc, audit). Modules talk to each other through well-defined interfaces, and **events flow internally** through the outbox + broker exactly as designed. One deployable unit, one database for the money core.

Option B — Microservices.

Each capability (auth, accounts, ledger, kyc, etc.) is its own separately deployed Spring Boot service with its own database, communicating over the network and via events.

7.3 Recommendation: Start with a Modular Monolith

For this platform — a first build, a team new to event-driven design, and a hard requirement for money correctness — ****start with Option A (modular monolith)**** and extract services later only where a real need appears.

Why a monolith first (honest reasoning)

FACTOR	WHY IT FAVORS THE MONOLITH
Money correctness	A transfer must atomically write debit + credit + outbox in one database transaction. In one app + one DB this is a single <code>@Transactional</code> commit. Across microservices it becomes a distributed saga with compensations — far more code and far more ways to corrupt balances.
Team experience	Event-driven is new to the team. Learning it inside one codebase (no network failures, no service discovery, no distributed tracing just to debug) is dramatically simpler.
Operational cost	One app to deploy, log, and monitor — not 6+ services, their pipelines, and the failure modes between them.
Speed to launch	Less infrastructure plumbing = more time on actual features.
Reversibility	Clean module boundaries mean you can extract a module into its own service later with low effort. Going the other way (merging premature microservices back) is painful.

Why NOT microservices yet (the trap to avoid)

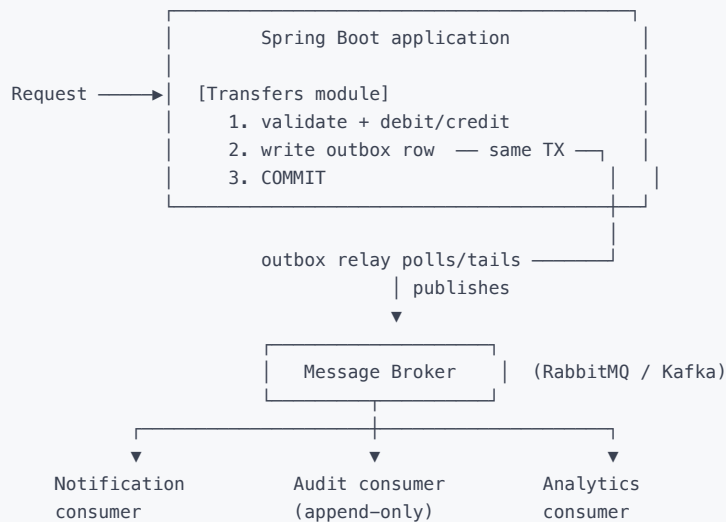
Microservices solve problems you do **not** have at launch: independent scaling of specific capabilities, large teams stepping on each other, polyglot stacks.

Adopting them on day one means paying their full cost — distributed transactions, network failures, eventual consistency *everywhere*, harder debugging — to solve problems that aren't present. That is over-engineering, the same mistake as reaching for heavy infrastructure when a simpler design works.

Rule of thumb: don't split into a service until a concrete pressure forces it (a module needs to scale independently, a separate team owns it, or it needs isolation/compliance boundaries). Split for a reason, not for fashion.

7.4 How Event-Driven Works Inside the Monolith

Even as a single Spring Boot app, the system is genuinely event-driven:



- Spring's `@Transactional` makes step 1–3 a single atomic commit.
- The outbox relay (a scheduled poller, or Debezium CDC if using Kafka) reads committed outbox rows and publishes them to the broker.
- Consumers can run as `@RabbitListener` / Kafka listeners ****inside the same app**** to start, and be moved to separate processes later without changing the event contracts.
- Spring Boot also supports in-process events (`ApplicationEventPublisher`) for the simplest cases, but the **outbox + broker** is what gives durability and the ability to extract services later — so use the outbox for anything that must not be lost (all money-related events).

7.5 Spring Boot Module Layout (suggested)

com.bank	
├ config/	app configuration only (security, JWT filter, broker)
├ shared/	cross-cutting: exception/, entity/ (BaseAuditEntity), utils/ (Money)
└ feature/	
├ auth/	JWT issue/verify, refresh rotation
├ rbac/	roles, permissions, enforcement
├ accounts/	customer lifecycle
├ wallets/	wallet lifecycle, balance reads
├ ledger/	double-entry postings, transactions ← money core
├ transfers/	transfer orchestration ← money core
├ kyc/	vendor orchestration, decisioning

└─ audit/	append-only audit consumer
└─ events/	outbox table, relay, event contracts

Each feature under `feature/` is internally layered into `web/` (controllers + DTOs), `domain/` (a service *interface* + its `Default*` implementation and business logic), and `persistence/` (repositories + JPA entities), with dependencies pointing inward toward `domain/`. The `ledger` + `transfers` + `events(outbox)` modules share the money database and one transaction boundary. `audit`, `kyc`, notifications, and analytics consume events and can later become their own services.

7.6 Per-Stack Notes

Backend — Java / Spring Boot

- **Spring Web** (REST), **Spring Security** (JWT + RBAC), **Spring Data JPA** (PostgreSQL), **Spring AMQP / Spring for Kafka** (broker), **Flyway** (DB migrations), **Micrometer** (Prometheus metrics), **Spring Boot Actuator** (`/metrics`, health), **OpenTelemetry** (tracing).
- Money amounts use `BigDecimal` (never `double` / `float`).
- Idempotency keys enforced with a unique DB constraint.

Dashboard — React

- Talks only to the `/v1` REST API; holds no business logic.
- RBAC-aware UI: hide/disable actions the role lacks, ****but the server still enforces**** every permission (UI hiding is convenience, not security).
- Auth via the same JWT; access token in memory, refresh handled carefully (httpOnly cookie or secure storage per the team's security review).

Mobile — React Native

- Single JavaScript/TypeScript codebase → iOS + Android.
- Refresh token in the **secure enclave** (iOS Keychain / Android Keystore).
- Transfer requests carry an **on-device idempotency key** so retries over a

flaky mobile network never double-charge.

- Certificate pinning, biometric unlock, push notifications fed by events.

7.7 Decision Summary

DECISION	CHOICE
Backend	Java + Spring Boot
Dashboard	React
Mobile	React Native
Communication style	Event-driven for side-effects (outbox + broker)
Money path	Synchronous, ACID (single DB transaction)
Deployment style	Modular monolith first , extract services on demand
Database (money core)	One shared PostgreSQL, single transaction boundary
Derived stores (audit/analytics)	Separate, event-fed, eventually consistent

Bottom line: Use event-driven design from day one (via the outbox), but deploy as a modular monolith — not microservices — until a concrete need justifies splitting. This gets the decoupling and audit-log benefits without the distributed-systems cost, and keeps money correctness simple.

Deployment & Cloud Infrastructure

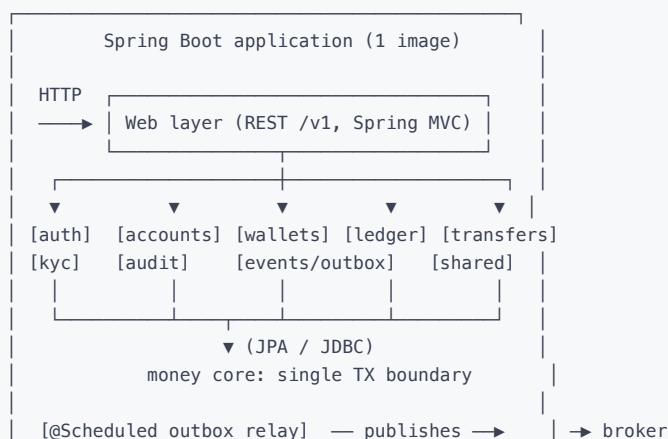
Digital Banking Platform · Monolith Architecture + Cloud Setup Status: Baseline · Owner: Architect + Platform / DevOps

8.1 Purpose

Describe, concretely, how the **modular-monolith** backend (Java / Spring Boot), the **React** dashboard, and the **React Native** mobile app are packaged, run, and scaled in the cloud — including the event-driven pieces (outbox + broker), the database, observability, networking, environments, and a realistic launch-size footprint. This is the bridge between the design (what) and operations (where it runs).

8.2 The Monolith, Concretely

One deployable Spring Boot application, internally split into modules. It is a **modular monolith**: one process, one codebase, clean internal boundaries — not many services.





Key properties:

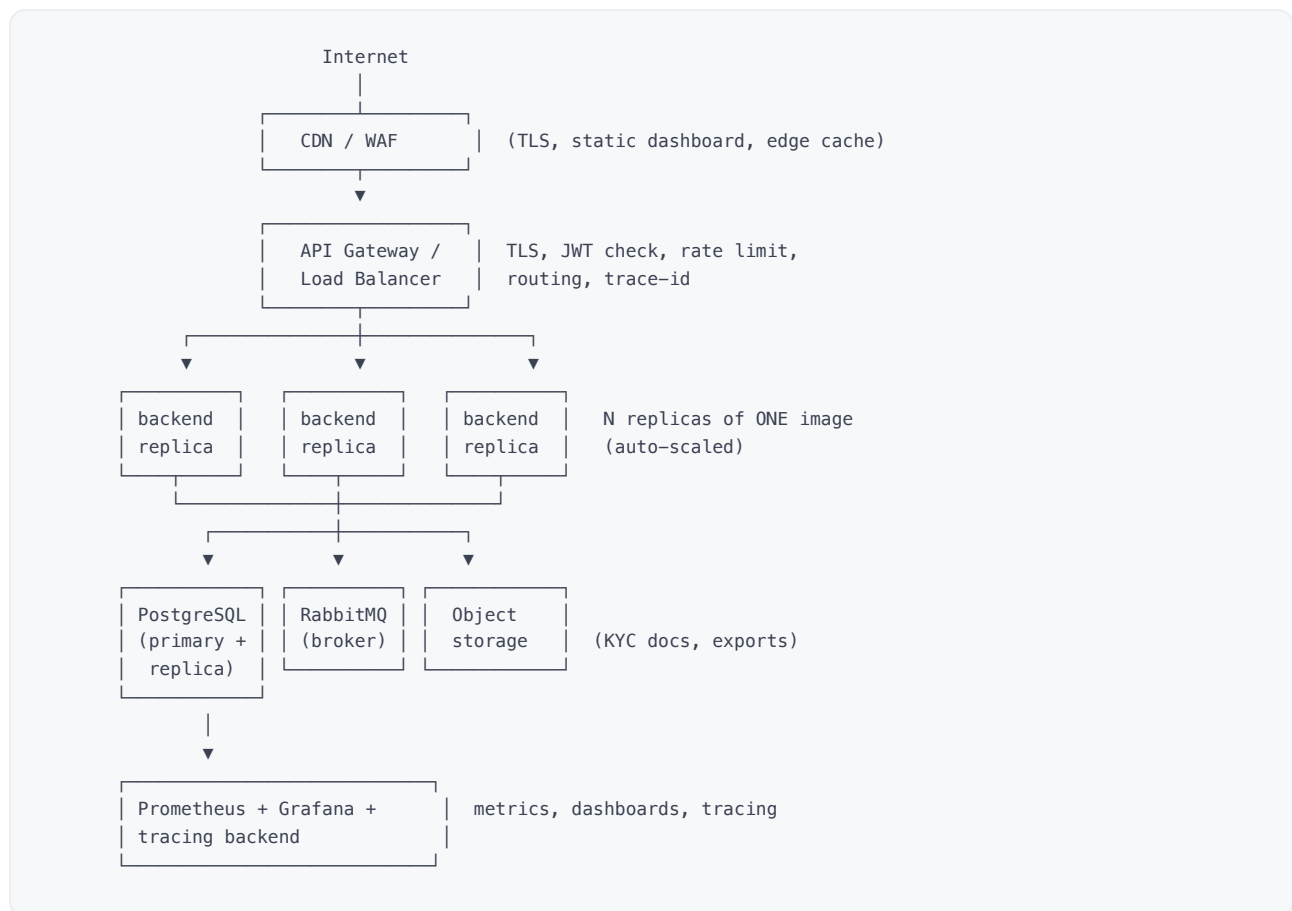
- **Stateless app:** no session state in memory; all state lives in the database. This is what lets us run **N identical replicas** behind a load balancer.
- **One money database, one transaction boundary:** `ledger` + `transfers` + `outbox` commit together (`@Transactional`).
- **Event-driven inside the app:** the relay and consumers run on background threads; the broker carries events out and back (see §3.7).

8.3 Deployable Units (what actually gets shipped)

UNIT	BUILT WITH	PACKAGED AS	RUNS AS
Backend API	Spring Boot (JAR)	Docker image (<code>backend: <ver></code>)	N replicas behind a load balancer
Admin Dashboard	React (static build)	Static files (Nginx/CDN)	Served by CDN or an Nginx container
Mobile App	React Native	<code>.ipa</code> / <code>.apk</code> (app stores)	On user devices
Outbox relay	(part of backend)	Same backend image	1 replica (or leader-elected)
Event consumers	(part of backend)	Same backend image	Inside backend replicas

The dashboard is just static files talking to the API — it needs no server of its own beyond a CDN/Nginx. The mobile app ships to the App Store / Play Store. Only the **backend** needs orchestration.

8.4 Cloud Architecture (target)



Components

- **CDN / WAF:** terminates TLS at the edge, serves the React static bundle, filters malicious traffic, caches static assets.
- **API Gateway / Load Balancer:** single ingress to the backend; validates JWT, applies rate limiting, injects `trace_id`, load-balances across replicas.
- **Backend replicas:** N copies of the one Spring Boot image; horizontally auto-scaled.
- **PostgreSQL:** managed, with a primary for writes, a read replica, automated backups, and **point-in-time recovery (PITR)**. This is the money system of record.
- **RabbitMQ:** the event broker (lightweight; see sizing).
- **Object storage:** KYC documents, signed audit exports (e.g. S3-compatible).

- **Observability stack:** Prometheus scrapes `/metrics` ; Grafana dashboards;

a tracing backend collects spans.

8.5 Orchestration

Dev / small staging — Docker Compose

A single `docker-compose.yml` brings up the whole stack with one command:

`backend` , `postgres` , `rabbitmq` , `prometheus` , `grafana` . Ideal for local

development and a small staging box.

Production — Kubernetes (or a managed container service)

- **Deployment:** declares the desired replica count of the backend image; the orchestrator keeps that many alive and replaces failed ones.
- **Service + Ingress:** load-balances across replicas; TLS at the edge.
- **Liveness/Readiness probes:** wired to Actuator `/health` ; a hung replica is restarted automatically; a not-ready replica receives no traffic.
- **Horizontal Pod Autoscaler (HPA):** add replicas when CPU/latency crosses a threshold (e.g. a payment surge), remove them after — pay for capacity only when used.
- **Rolling updates:** replace replicas one at a time → zero-downtime deploys and instant rollback to the previous image.
- **Secrets/config:** DB credentials, JWT signing keys, vendor API keys in a secrets manager — never in the image.

Two stateful constraints (important)

- **Database:** one primary writer (plus read replicas / failover). You do not run N independent writers — state has a single home.

- **Outbox relay:** exactly **one active publisher** (single replica or leader-elected) so events aren't double-published. Consumers are idempotent, so a rare duplicate is survivable, but one relay is cleaner.

8.6 Environments & Promotion

ENV	PURPOSE	INFRA
Local	Dev inner loop	Docker Compose on the developer machine
CI	Automated tests on every push	Ephemeral containers in the pipeline
Staging	E2E, perf, UAT, security	Prod-like, smaller replica counts, no real PII
Prod	Live	Full HA: multi-AZ, autoscaling, backups

Promotion: the same immutable image (`backend:<ver>`) is built once, tested in CI, deployed to staging, then promoted to prod — identical artifact, only config/secrets differ per environment.

8.7 CI/CD Pipeline

```

git push
|
▼
[CI] build JAR → unit + integration tests → contract tests → SAST/SCA
    |
    → build Docker image → push to registry (tag = commit/version)
    ▼
[CD] deploy image to STAGING → run E2E + smoke
    |
    → manual/auto gate
    ▼
    promote SAME image to PROD → rolling update → health-gated → done
                                | on failure
                                ▼ automatic rollback to previous image
  
```

- One image, built once, promoted across environments (no rebuild per env).
- Health-gated rollouts; rollback is just "redeploy previous tag."

8.8 Launch-Size Footprint (realistic, not a "big server")

A modular monolith + event-driven runs on a modest footprint:

COMPONENT	LAUNCH SIZING	NOTES
Backend replicas	2–3 × (1 vCPU, 1–2 GB) on a 2–4 vCPU node	Start at 2 for HA; autoscale up
PostgreSQL	Small managed tier (2 vCPU, 4–8 GB)	Primary + 1 read replica, PITR on
RabbitMQ	512 MB–1 GB	Lightweight; can co-locate at first
Prometheus+Grafana	Small (1 vCPU, 1–2 GB)	Scales with retention
Object storage	Pay-per-use	KYC docs, exports

This fits comfortably on a single small cloud VM (Docker Compose) for early staging, and a small Kubernetes node pool for production. You **scale horizontally in cheap increments** — add a backend replica when traffic demands — rather than buying a big server upfront. Microservices, by contrast, would multiply JVM overhead and infra (per-service DBs, discovery, mesh) and cost **more** for the same load.

8.9 Data Durability & Disaster Recovery

The single most important rule: **a replica is not a backup**. A replica keeps the system *available*; backups let you *recover*. Money data needs both, in layers, because no single mechanism covers every failure.

8.9.1 Don't self-host the production database in Docker

Running PostgreSQL in a plain Docker container on one host is fine for ****local**

dev **and small staging** — **but** not for production money data^{**}. If that

container or its host dies and the data lived only there, it is gone.

For production, use a **managed PostgreSQL** service (AWS RDS, Google Cloud SQL,

Azure Database for PostgreSQL, DigitalOcean Managed Databases, etc.). The provider runs the database and the durability machinery (replication, continuous backup, PITR, cross-region copies); the app just connects to it.

The split: Docker/Kubernetes orchestrate the **stateless app**. The **stateful database** lives outside that, as a managed service. Never put the only copy of money data inside an ephemeral container.

8.9.2 Why "primary + replica" alone is not enough

A replica protects against exactly **one** failure: the primary crashing. It does **not** protect against the failures that destroy data, because a replica faithfully copies whatever the primary does — including mistakes:

FAILURE	DOES A REPLICA SAVE YOU?
Primary server crashes	✅ Yes — promote replica
Bad deploy / bug DELETes or corrupts rows	❌ No — replicated instantly
Accidental <code>DROP TABLE</code>	❌ No — replicated instantly
Ransomware / account compromise	❌ No — hits both
Whole datacenter / region outage	❌ No (if same location)

A replica is for **availability**; it is **not** a recovery tool. This is exactly why "what if both are down?" is the right question — replicas alone do not answer it.

8.9.3 The layered protection (defense in depth)

Money data is protected by a **stack** of independent mechanisms, not one:

1. **Replica (high availability).** If the primary fails, the replica is promoted in seconds-to-minutes. Place it in a **different availability zone** so one datacenter failure can't take both.
1. **Continuous backup + Point-In-Time Recovery (PITR) — the real safety net.**

The database continuously ships its write-ahead log (WAL) to ****separate storage (e.g. object storage / S3). This lets you restore to any moment****

— e.g. "the exact state at 14:32:05, one second before the bad `DELETE`."

Even if both primary and replica are lost or corrupted, the database is rebuilt from these backups. This is what delivers the **RPO ≤ 5 min** guarantee: at most ~5 minutes of data is ever at risk.

1. **Periodic snapshots.** Automated daily full backups, retained for weeks-to-months, stored separately from the live database.

1. **Offsite / cross-region copies.** Backups copied to a ****different geographic region****. If an entire region is lost, the data still exists elsewhere.

8.9.4 "Both database servers down" — what actually happens

SCENARIO	OUTCOME
Primary crashes	Replica promoted → keep running. No data loss.
Primary and replica both crash	App down briefly, but data is safe in backups . Restore from PITR → back online with ≤ RPO loss.
Someone runs <code>DROP TABLE</code> / bad migration	Replica copied it too — but PITR restores to the instant before.
Whole region gone	Restore from the cross-region backup in another region.
Backups also lost	← the only true "lose everything." Prevented by storing backups separately, cross-region, with retention + tested restores.

So: primary **and** replica both down does **not** mean data lost — the backups live in separate storage (and another region), independent of both database instances. You restore from them and lose at most the RPO window, not everything.

8.9.5 The 3-2-1 rule (industry standard)

Keep **3** copies of the data, on **2** different storage types, with **1** copy offsite.

Our mapping: **primary (1) + replica (2) + **backup** in object storage in another region (3, offsite)**. That is precisely why "both database servers down" never equals "data lost."

8.9.6 A backup you have never restored is not a backup

Backups silently fail (wrong config, corrupted archive, missing tables). The only proof they work is a **restore drill**: periodically restore a backup into a clean environment and verify the data — including that the **ledger invariants still hold** (debits = credits, balances reconcile). This validates both the **RPO ≤ 5 min** and the **RTO ≤ 1 hr** (time to be back online) targets.

8.9.7 Design-level recovery path (specific to this platform)

Beyond database backups, the **data model itself** adds a recovery path:

- The **ledger is append-only and double-entry** — it is a complete, reconstructable history of every money movement.
- There is an **immutable audit trail** and an **event stream** of everything that happened.

So even in a catastrophic partial-data scenario, balances can be **reconciled/replayed** from the ledger, audit, and events — a recovery layer on top of the database backups. This is a deliberate property of the money model, not an accident.

8.9.8 Other reliability measures

- **High availability**: app replicas and DB standby spread across availability zones; broker persistence enabled so queued events survive a broker restart.
- **Graceful degradation**: if a consumer or external vendor is down, events

queue and retry; the money path keeps working.

- **Dead-letter queue:** poison events are parked and replayable.
- **Backups of the broker/object storage:** KYC documents and signed audit

exports in object storage are themselves versioned and replicated.

8.9.9 Recovery targets (summary)

TARGET	VALUE	MEANING
RPO	≤ 5 min	Maximum data loss window (covered by continuous WAL/PITR)
RTO	≤ 1 hr	Maximum time to restore core services
Backup retention	weeks–months	How far back you can restore
Restore drill	periodic	Proves backups actually work
Copies (3–2–1)	3 / 2 / 1	3 copies, 2 media, 1 offsite/cross-region

8.10 Security of the Infrastructure

- TLS everywhere (edge and service-to-service).
- Secrets in a managed vault; signing keys rotated.
- Least-privilege network policies: only the backend reaches the DB/broker; the

DB is never public.

- PII encrypted at rest (DB + object storage); KYC docs access-controlled.
- WAF + rate limiting at the edge; gateway enforces JWT before anything reaches

a service.

8.11 Scaling Path (when you outgrow the monolith)

You do **not** start with microservices, but the design leaves the door open:

1. **First:** scale the monolith horizontally (more replicas) — handles a large range of growth cheaply.

1. **Then:** offload reads to DB read replicas; cache hot reads.

2. **Later, only if a specific module needs independent scale or ownership:**

extract that one module into its own service. Because modules already communicate via events and clean interfaces, extraction is low-effort and incremental — pull out `kyc` or `notifications` first (they're already event-fed), leave the money core intact.

Extract a service for a concrete reason (independent scale, separate team, compliance isolation) — not by default. The monolith carries you a long way.

8.12 Reference Implementation on AWS (EKS + GitOps)

8.12.1 Components → AWS services

NEED	AWS SERVICE
Kubernetes cluster	EKS (managed Kubernetes)
Database (PostgreSQL)	RDS PostgreSQL (Multi-AZ)
Message broker	Amazon MQ (RabbitMQ)
Object storage	S3 (KYC docs, exports)
Container registry	ECR
Load balancer / ingress	ALB (AWS LB Controller)
Dashboard (React)	S3 + CloudFront
TLS / CDN / firewall	CloudFront + WAF + ACM
Secrets	Secrets Manager
Metrics / logs	CloudWatch + Prometheus/Grafana

Runs inside the cluster (GitOps-managed): backend monolith

(Deployment + Service + HPA), outbox relay (1 replica), ingress, Prometheus + Grafana, the GitOps controller.

Stays outside as managed services: RDS, Amazon MQ, S3, Secrets Manager — the database is **never** run inside Kubernetes (same durability reasoning as §8.9).

8.12.2 Why EKS (not ECS) here

GitOps tooling (Argo CD, Flux) is **Kubernetes-native**. Choosing GitOps is the reason to use **EKS** rather than ECS Fargate — the cluster's entire desired state is declarative YAML a controller can reconcile from Git.

8.12.3 What GitOps is

Git is the **single source of truth** for what runs in the cluster:

1. Desired state (Deployments, replicas, config) lives as YAML in a Git repo.
2. A controller in the cluster (**Argo CD** or **Flux**) watches that repo.

3. On a Git change, the controller makes the cluster match Git automatically.
4. Manual cluster drift is detected and reverted back to Git.

To change production you open a **pull request**; merge → the cluster updates

itself. Git history is the deployment history; **rollback** = `git revert`. Every

production change is a reviewed commit — a compliance win for a regulated system.

8.12.4 The two-repo flow



- **App repo:** Spring Boot / React / React Native source. CI builds and pushes the image to ECR.

- **Config repo (the GitOps repo):** Kubernetes manifests Argo CD watches.

Promotion to prod = a commit bumping the image tag in the prod overlay.

8.12.5 Tooling

- **Controller:** Argo CD (has a UI showing sync status/diffs — friendlier to learn) or Flux (lighter, CLI/CRD-driven).

- **Manifests:** Kustomize (base + `dev` / `staging` / `prod` overlays) or Helm.

Kustomize is simpler to start.

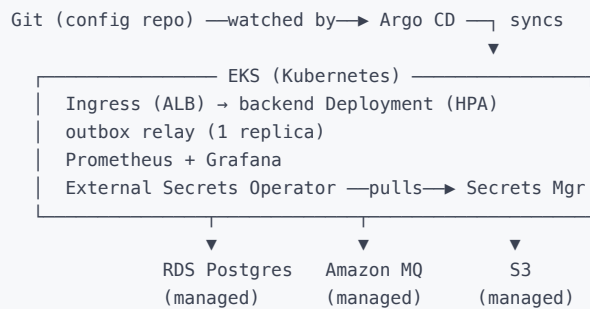
- **Environments:** one Argo CD `Application` per env; promote by bumping the image tag up the overlays via PR.

- **Secrets:** never commit raw secrets. Use **External Secrets Operator**

(pulls from AWS Secrets Manager into the cluster) or **Sealed Secrets**

(encrypted, safe to commit).

8.12.6 Picture



8.12.7 Adoption order (don't learn it all at once)

1. Monolith on EKS with plain manifests — get cluster basics working.
2. Add Argo CD pointing at the config repo.
3. Add Kustomize overlays per environment.
4. Add External Secrets for credentials.

Caveat: GitOps + EKS is powerful (auditable, self-healing, declarative) but is a real learning curve. Sequence it after the cluster basics are comfortable — don't adopt Kubernetes, Argo CD, and event-driven design all in the same week.

8.13 Summary

DECISION	CHOICE
Backend packaging	One Docker image (Spring Boot), N stateless replicas
Dashboard hosting	React static build on S3 + CloudFront
Mobile distribution	React Native apps via App Store / Play Store
Orchestration (dev)	Docker Compose
Orchestration (prod)	EKS (Kubernetes): Deployment + Service + HPA + probes
Deployment model	GitOps (Argo CD): Git is the source of truth
Database	RDS PostgreSQL, Multi-AZ, PITR (managed, outside k8s)
Broker	Amazon MQ (RabbitMQ), managed
Event relay	Single active replica (leader-elected)
Secrets	Secrets Manager via External Secrets Operator
Footprint	Small node pool; scale horizontally on demand
Microservices	Deferred — extract a module only when a concrete need arises

Backend API — Features & Implementation

Digital Banking Platform · Backend Feature Documentation Status: Baseline · Owner: Backend Lead
Runtime: Java 17 · Spring Boot 3 · PostgreSQL · RabbitMQ

Overview

The backend is a single deployable **modular monolith** built on Java 17 and Spring Boot 3. It is internally divided into clean modules — `auth`, `accounts`, `wallets`, `ledger`, `transfers`, `kyc`, `audit`, and `events` — that talk to each other through well-defined service interfaces. There is **one** PostgreSQL database that is the system-of-record for money.

Two rules drive every design decision in this document:

1. **Money moves synchronously, inside one ACID transaction.** A money operation either commits completely or not at all. Events never move money.
1. **Side-effects are event-driven via the transactional outbox.** Notifications, projections, and downstream fan-out are published *after* the money transaction commits, derived from rows written in that same transaction.

Each feature below is documented as: **(a) what** it is, **(b) why** it is designed this way, **(c) how** it works, and **(d) real code**.

1. Project Structure & Module Layout

What. A **package-by-feature** layout inside one Gradle/Maven project. Each feature is a package holding its own web, domain, and persistence layers

(controller + DTOs, service interface + implementation, entity + repository).

This is the opposite of **package-by-layer** (one big `controller/`, `service/`, `repository/`), which scatters a single feature across the codebase.

Why package-by-feature. A change to transfers stays in the `transfers` package instead of touching four layer-packages; related code sits together; and a future extraction of a module into its own service is a matter of moving one package rather than untangling shared layers.

Layered inside each feature. A feature package is **not** flat — it is split into three explicit layers, each a sub-package with one responsibility:

LAYER	HOLDS	DEPENDS ON
<code>web/</code>	<code>*Controller</code> , request/response DTOs	<code>domain/</code> (interface)
<code>domain/</code>	service interface + business logic	nothing outward
<code>persistence/</code>	<code>Repository</code> , <i>JPA</i> , <code>Entity</code>	implements ports the domain owns

The dependency arrow always points **inward** toward `domain/`: `web/` and `persistence/` know about the domain, never the reverse. This is the structural expression of the Dependency Inversion Principle — the core logic depends on abstractions, and the delivery (web) and infrastructure (DB) layers plug into it.

Interface + implementation by default. Every service is declared as an **interface** in `domain/` with a `Default*` implementation beside it — `OrderService` + `DefaultOrderService`, `TransferService` + `DefaultTransferService`, and so on:

WHY INTERFACE-FIRST	PAY-OFF
Callers depend on the port , not the class	Dependency Inversion is uniform, not selective
Tests mock the interface, no Spring context needed	Fast, isolated unit tests for every service
A second implementation never forces a refactor	Open/Closed: add an impl, callers untouched
One predictable shape across all features	Lower cognitive load — every feature reads the same

The `Default*` prefix marks the production implementation; a test stub or an alternate strategy is just another class against the same interface.

How — the shape of one feature package. Each module is layered into

`web/` , `domain/` , and `persistence/` :

```

transfers/
├── web/
│   ├── TransferController.java    # @RestController /v1/transfers
│   └── dto/
│       ├── TransferRequest.java   # request record (validated)
│       └── TransferResult.java    # response record
├── domain/
│   ├── TransferService.java       # INTERFACE – the port callers depend on
│   └── DefaultTransferService.java # @Service @Transactional debit+credit+outbox
└── persistence/
    ├── Transfer.java              # @Entity
    ├── Transaction.java           # @Entity
    ├── TransferRepository.java    # extends JpaRepository
    └── TransactionRepository.java

```

Full package tree

```

com.bank
├── BankApplication.java           # @SpringBootApplication entry point
├── shared/                        # DRY: cross-cutting, no business logic
│   ├── exception/
│   │   ├── ApiError.java         # {code, message, traceId} envelope
│   │   ├── GlobalExceptionHandler.java # @RestControllerAdvice
│   │   └── DomainException.java   # base for typed business errors
│   ├── entity/
│   │   └── BaseAuditEntity.java   # id, createdAt, updatedAt, version
│   └── utils/
│       ├── Money.java            # BigDecimal helper, minor-unit safe
│       ├── CurrentUser.java      # authenticated principal accessor
│       └── PageResponse.java      # pagination wrapper
├── config/                        # SRP: app configuration only
│   ├── SecurityConfig.java
│   ├── JwtAuthFilter.java        # OncePerRequestFilter
│   └── RabbitTopologyConfig.java  # exchanges, queues, bindings

```

```

└─ feature/
  └─ auth/
    └─ web/
      └─ AuthController.java      # POST /v1/auth/{register,login,refresh,logout}
      └─ dto/ (LoginRequest, RegisterRequest, TokenPair)
    └─ domain/
      └─ AuthService.java         # INTERFACE
      └─ DefaultAuthService.java  # @Service
      └─ JwtService.java          # INTERFACE – RS256 sign/verify
      └─ DefaultJwtService.java   # @Service
    └─ persistence/
      └─ User.java RefreshToken.java # @Entity (refresh: hashed, sessionId family)
      └─ UserRepository.java RefreshTokenRepository.java

  └─ rbac/
    └─ domain/
      └─ PermissionService.java    # INTERFACE – resolve effective permissions
      └─ DefaultPermissionService.java # @Service
      └─ PermissionEvaluator.java   # backs @PreAuthorize("hasPermission(...)")
    └─ persistence/
      └─ Role.java Permission.java UserRole.java # @Entity
      └─ RoleRepository.java PermissionRepository.java

  └─ accounts/
    └─ web/ AccountController.java # /v1/accounts + dto/
    └─ domain/ AccountService.java + DefaultAccountService.java # @Service
    └─ persistence/ Account.java AccountStatus.java AccountRepository.java

  └─ wallets/
    └─ web/ WalletController.java # /v1/wallets + dto/
    └─ domain/ WalletService.java + DefaultWalletService.java # balance derived from ledger
    └─ persistence/ Wallet.java WalletStatus.java WalletRepository.java

  └─ ledger/ # append-only money truth
    └─ domain/
      └─ LedgerService.java # INTERFACE – post(), balanceOf()
      └─ DefaultLedgerService.java # @Service
    └─ persistence/
      └─ LedgerEntry.java # @Entity, immutable
      └─ Direction.java # DEBIT | CREDIT
      └─ LedgerEntryRepository.java

  └─ transfers/ # the money core
    └─ web/ TransferController.java # /v1/transfers, /v1/transactions + dto/
    └─ domain/ TransferService.java + DefaultTransferService.java # @Transactional

debit+credit+outbox
└─ persistence/ Transfer.java Transaction.java TransferRepository.java TransactionRepository.java

└─ kyc/
  └─ web/ KycController.java # /v1/kyc + dto/
  └─ domain/
    └─ KycService.java + DefaultKycService.java # case state machine
    └─ KycVendorClient.java # INTERFACE – vendor port
    └─ HttpKycVendorClient.java # real impl (test stub is the alternate)
  └─ persistence/ KycCase.java KycStatus.java KycCaseRepository.java

└─ audit/
  └─ web/ AuditController.java # /v1/audit + dto/
  └─ domain/
    └─ AuditService.java + DefaultAuditService.java # append(), query()
    └─ AuditAspect.java # @Audited AOP
  └─ persistence/ AuditRecord.java AuditRecordRepository.java # @Entity, append-only

└─ events/ # the event backbone
  └─ domain/
    └─ DomainEvent.java
    └─ EventPublisher.java # INTERFACE – outbox port
    └─ OutboxEventPublisher.java # real impl (in-memory is the test alternate)
    └─ OutboxRelay.java # @Scheduled publisher → RabbitMQ

```

```

├─ persistence/ OutboxEvent.java OutboxEventRepository.java
└─ web/      NotificationConsumer.java # @RabbitListener, idempotent

```

Every service is an **interface + Default* implementation** in its feature's **domain/** layer. **KycVendorClient** and **EventPublisher** additionally have a *second* production-relevant implementation (test stub / in-memory), but the interface-first rule is uniform — callers always depend on the port, never the class.

Resources & build layout

```

src/main/resources
├─ application.yml           # base config
├─ application-dev.yml       # profile overrides
├─ application-prod.yml
├─ db/migration/             # Flyway, ordered, immutable once shipped
│   ├── V1__core.sql         # users, accounts, wallets, rbac
│   ├── V2__ledger.sql       # ledger_entries, transactions, transfers
│   ├── V3__kyc.sql
│   ├── V4__audit.sql
│   └── V5__outbox.sql
└─ logback-spring.xml        # structured JSON logs with traceId

src/test/java/com/bank/feature
├─ transfers/ DefaultTransferServiceTest.java # double-entry + idempotency (mocks the port)
├─ auth/      RefreshRotationTest.java
└─ support/   PostgresTestContainer.java, TestDataFactory.java

```

Tests live in the same package as the feature they exercise, mirroring **src/main**.

Because each service is an interface, unit tests target the **Default*** impl and mock collaborators through their ports — no Spring context needed.

Conventions

- Every feature is split into **web/** (controllers + DTOs), **domain/** (service **interface** + **Default*** impl + business logic), and **persistence/** (repositories + JPA entities). Dependencies point inward toward **domain/**.
- Every service is an **interface + Default* implementation**; callers depend on the interface, never the concrete class.
- DTOs are Java **record**s under **web/dto/**; entities stay in **persistence/**.
- All money-moving logic is confined to **ledger/** + **transfers/**; no other feature writes **ledger_entries**.

- `shared/` holds only cross-cutting plumbing — `exception/` (error envelope + handler), `entity/` (`BaseAuditEntity`), `utils/` (`Money` , paging) — and `config/` holds app-wide configuration only. Neither holds business logic.

How the structure follows SOLID / DRY / KISS / YAGNI

The feature-based, layered structure isn't arbitrary — each principle maps to a concrete structural decision:

PRINCIPLE	IN THIS STRUCTURE
S — Single Responsibility	Each layer has one job: <code>web/</code> (HTTP edge + DTOs), <code>domain/</code> (business logic behind a service interface), <code>persistence/</code> (repositories + entities). <code>LedgerService</code> is the only writer of <code>ledger_entries</code> — money correctness has one owner.
O — Open/Closed	New side-effects are added as new event consumers in <code>events/</code> ; the producer (<code>DefaultTransferService</code>) never changes. A new behaviour is a new implementation of an existing service interface, not an edit to a caller.
L — Liskov Substitution	Every <code>Default*</code> impl is fully substitutable for its interface, and alternates (<code>HttpKycVendorClient</code> vs. a test stub, outbox vs. in-memory publisher) honor the same contract the caller relies on.
I — Interface Segregation	Service interfaces are per-feature and narrow; DTOs are small per-use-case <code>record</code> s (<code>TransferRequest</code> , <code>TransferResult</code>), not one fat shared object. Callers depend only on the operations they need.
D — Dependency Inversion	High-level logic depends on abstractions it owns : <code>web/</code> → <code>domain/</code> service <i>interface</i> , and <code>domain/</code> → repository/port — never on a concrete class or RabbitMQ directly. The <code>web/</code> → <code>domain/</code> ← <code>persistence/</code> arrows all point inward. All wiring is constructor injection .
DRY	One <code>shared/</code> source of truth per cross-cutting decision: <code>exception/</code> (one error shape via <code>ApiError</code> + <code>GlobalExceptionHandler</code>), <code>entity/BaseAuditEntity</code> (one audit-column set), <code>utils/Money</code> (one money type).
KISS	A uniform three-layer shape (<code>web/</code> / <code>domain/</code> / <code>persistence/</code>) every feature follows, so any module reads the same way; derived balance (<code>SUM(ledger_entries)</code>) instead of a cached column + invalidation; a single <code>@Transactional</code> money path instead of a saga.
YAGNI	The structure stops at three layers and one interface per service — no <code>api/internal/impl</code> micro-layers, no microservices, no generic base-service, no premature caching until a measured need appears.

The through-line: give each class and each layer one job behind an explicit interface (SOLID), inject collaborators through the ports they own (DIP), and keep one source of truth for each cross-cutting concern in `shared/` (DRY) — with a single, predictable feature shape rather than ad-hoc structure.

2. Authentication

What. Registration, password login, stateless **JWT access tokens** signed with **RS256** (short TTL), and **rotating refresh tokens** that are stored hashed. Re-use of an already-rotated refresh token is treated as theft and revokes the entire session family.

Why.

- **Stateless JWT access tokens** mean the API does not hit the database to validate every request — the signature and claims are enough. Short TTL (e.g. 10 minutes) limits the blast radius of a leaked token.
- **RS256 (asymmetric)** lets the auth module hold the private key while every other module (or a future separate service) verifies with the public key. No shared secret to leak.
- **Rotating refresh tokens** give long sessions without long-lived bearer credentials. Each refresh issues a new refresh token and invalidates the old.
- **Reuse detection** is the key security property: if an old (already rotated) token is presented, either the client is buggy or an attacker replayed a stolen token. We cannot tell which, so we revoke the whole session family — fail safe.

How. Refresh tokens are persisted as a linked chain (`previousId`) within a `sessionId` family. We store only a SHA-256 hash of the secret. On refresh we look up by hash; if the matched token is already `rotated` or `revoked` , we revoke every token sharing that `sessionId` .

SecurityConfig

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity // enables @PreAuthorize
public class SecurityConfig {
```

```

private final JwtAuthFilter jwtAuthFilter;

public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
    this.jwtAuthFilter = jwtAuthFilter;
}

@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable) // stateless API, no cookies
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/v1/auth/register", "/v1/auth/login",
                "/v1/auth/refresh").permitAll()
            .requestMatchers("/actuator/health", "/actuator/info").permitAll()
            .anyRequest().authenticated() // default=deny everything else
        )
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
        .exceptionHandling(e -> e
            .authenticationEntryPoint((req, res, ex) ->
                res.sendError(HttpServletResponse.SC_UNAUTHORIZED)));
    return http.build();
}

@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}
}

```

JwtService (RS256 issue + verify)

```

@Service
public class DefaultJwtService implements JwtService {

    private final RSAPrivateKey privateKey;
    private final RSAPublicKey publicKey;
    private final Duration accessTtl;

    public JwtService(KeyProvider keys,
        @Value("${security.jwt.access-ttl:PT10M}") Duration accessTtl) {
        this.privateKey = keys.privateKey();
        this.publicKey = keys.publicKey();
        this.accessTtl = accessTtl;
    }

    public String issueAccessToken(User user, Set<String> permissions) {
        Instant now = Instant.now();
        return JWT.create()
            .withSubject(user.getId().toString())
            .withIssuer("bank-core")
            .withClaim("email", user.getEmail())
            .withArrayClaim("perms", permissions.toArray(String[]::new))
            .withIssuedAt(now)
            .withExpiresAt(now.plus(accessTtl))
            .withJWTId(UUID.randomUUID().toString())
            .sign(Algorithm.RSA256(publicKey, privateKey));
    }

    public DecodedJWT verify(String token) {
        return JWT.require(Algorithm.RSA256(publicKey, null))
            .withIssuer("bank-core")
            .build()
            .verify(token); // throws JWTVerificationException on tamper/expiry
    }
}

```

```

    }
}

```

JwtAuthFilter

```

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    public JwtAuthFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader(HttpHeaders.AUTHORIZATION);
        if (header != null && header.startsWith("Bearer ")) {
            try {
                DecodedJWT jwt = jwtService.verify(header.substring(7));
                List<SimpleGrantedAuthority> authorities = jwt.getClaim("perms")
                    .asList(String.class).stream()
                    .map(SimpleGrantedAuthority::new)
                    .toList();
                var auth = new UsernamePasswordAuthenticationToken(
                    jwt.getSubject(), null, authorities);
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JWTVerificationException ex) {
                SecurityContextHolder.clearContext(); // invalid → stays anonymous
            }
        }
        chain.doFilter(req, res);
    }
}

```

RefreshToken entity + repository

```

@Entity
@Table(name = "refresh_tokens",
        indexes = @Index(name = "ix_refresh_hash", columnList = "tokenHash", unique = true))
public class RefreshToken {

    @Id @GeneratedValue
    private UUID id;

    @Column(nullable = false)
    private UUID userId;

    @Column(nullable = false)
    private UUID sessionId; // family id; reuse revokes the whole family

    @Column(nullable = false, length = 64)
    private String tokenHash; // SHA-256 hex of the secret, never the secret

    private UUID previousId; // chain link to the token this rotated from

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TokenStatus status; // ACTIVE, ROTATED, REVOKED
}

```

```

@Column(nullable = false)
private Instant expiresAt;

// getters / setters / protected no-arg ctor omitted
}

```

```

public interface RefreshTokenRepository extends JpaRepository<RefreshToken, UUID> {
    Optional<RefreshToken> findByTokenHash(String tokenHash);

    @Modifying
    @Query("update RefreshToken t set t.status = 'REVOKED' where t.sessionId = :sessionId")
    void revokeSession(@Param("sessionId") UUID sessionId);
}

```

AuthController

```

@RestController
@RequestMapping("/v1/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    public UserView register(@Valid @RequestBody RegisterRequest req) {
        return authService.register(req);
    }

    @PostMapping("/login")
    public TokenPair login(@Valid @RequestBody LoginRequest req) {
        return authService.login(req.email(), req.password());
    }

    @PostMapping("/refresh")
    public TokenPair refresh(@Valid @RequestBody RefreshRequest req) {
        return authService.refresh(req.refreshToken());
    }
}

```

AuthService (rotation + reuse detection)

```

@Service
public class DefaultAuthService implements AuthService {

    private final UserRepository users;
    private final RefreshTokenRepository refreshTokens;
    private final PermissionService rbac;
    private final JwtService jwtService;
    private final PasswordEncoder encoder;
    private final Duration refreshTtl;

    // constructor injection omitted for brevity
}

```

```

@Override
@Transactional
public TokenPair login(String email, String rawPassword) {
    User user = users.findByEmail(email)
        .filter(u -> encoder.matches(rawPassword, u.getPasswordHash()))
        .orElseThrow(() -> new ApiException("AUTH_INVALID", "Invalid credentials", 401));
    return issuePair(user, UUID.randomUUID()); // new session family
}

@Override
@Transactional
public TokenPair refresh(String presentedSecret) {
    String hash = Hashing.sha256Hex(presentedSecret);
    RefreshToken token = refreshTokens.findByTokenHash(hash)
        .orElseThrow(() -> new ApiException("AUTH_REFRESH_UNKNOWN", "Unknown token", 401));

    // REUSE DETECTION: an already-rotated/revoked token means replay → kill the family
    if (token.getStatus() != TokenStatus.ACTIVE || token.getExpiresAt().isBefore(Instant.now())) {
        refreshTokens.revokeSession(token.getSessionId());
        throw new ApiException("AUTH_REFRESH_REUSE", "Refresh token reuse detected", 401);
    }

    token.setStatus(TokenStatus.ROTATED); // single-use
    User user = users.findById(token.getUserId()).orElseThrow();
    return issuePair(user, token.getSessionId(), token.getId());
}

private TokenPair issuePair(User user, UUID sessionId) {
    return issuePair(user, sessionId, null);
}

private TokenPair issuePair(User user, UUID sessionId, UUID previousId) {
    Set<String> perms = rbac.permissionsFor(user.getId());
    String access = jwtService.issueAccessToken(user, perms);

    String secret = SecureRandoms.urlToken(48); // opaque high-entropy string
    RefreshToken rt = new RefreshToken();
    rt.setUserId(user.getId());
    rt.setSessionId(sessionId);
    rt.setPreviousId(previousId);
    rt.setTokenHash(Hashing.sha256Hex(secret));
    rt.setStatus(TokenStatus.ACTIVE);
    rt.setExpiresAt(Instant.now().plus(refreshTtl));
    refreshTokens.save(rt);

    return new TokenPair(access, secret);
}
}

```

3. Authorization / RBAC

What. Role-based access control with fine-grained permissions. Roles are

`{customer, support, operator, compliance, admin, auditor}`. A user can hold

several roles; effective permissions are the union of all roles' permissions.

Enforcement is **default-deny**: nothing is allowed unless a permission grants

it. Privileged actions are audited.

Why. Roles are convenient for assignment but coarse for enforcement.

Binding endpoints to *permissions* (e.g. `transfer:create`, `kyc:review`) rather than role names means we can re-shape roles without touching controllers. Default-deny is the only safe posture for a bank: a missing rule must mean "no", never "yes".

How. Permissions are baked into the JWT `perms` claim at login (see §2), so enforcement is a stateless claim check. We use Spring method security with `@PreAuthorize("hasAuthority('...')")`. The `perms` are computed by joining `user_roles` → `role_permissions` → `permissions`.

Entities

```
@Entity @Table(name = "roles")
public class Role {
    @Id @GeneratedValue private UUID id;
    @Column(unique = true, nullable = false) private String name; // e.g. "compliance"

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name = "role_permissions",
        joinColumns = @JoinColumn(name = "role_id"),
        inverseJoinColumns = @JoinColumn(name = "permission_id"))
    private Set<Permission> permissions = new HashSet<>();
    // accessors omitted
}

@Entity @Table(name = "permissions")
public class Permission {
    @Id @GeneratedValue private UUID id;
    @Column(unique = true, nullable = false) private String name; // e.g. "transfer:create"
    // accessors omitted
}

@Entity @Table(name = "user_roles",
    uniqueConstraints = @UniqueConstraint(columnNames = {"userId", "roleId"}))
public class UserRole {
    @Id @GeneratedValue private UUID id;
    @Column(nullable = false) private UUID userId;
    @Column(nullable = false) private UUID roleId;
    // accessors omitted
}
```

Permission resolution

```
@Service
public class DefaultPermissionService implements PermissionService {
```

```

private final EntityManager em;

public DefaultPermissionService(EntityManager em) { this.em = em; }

@Transactional(readOnly = true)
public Set<String> permissionsFor(UUID userId) {
    return new HashSet<>(em.createQuery("""
        select distinct p.name
        from UserRole ur
        join Role r on r.id = ur.roleId
        join r.permissions p
        where ur.userId = :uid
        """, String.class)
        .setParameter("uid", userId)
        .getResultList());
}
}

```

Protected endpoint

```

@RestController
@RequestMapping("/v1/admin/users")
public class UserAdminController {

    private final UserAdminService service;

    public UserAdminController(UserAdminService service) { this.service = service; }

    // default-deny: only principals whose JWT carries this authority pass
    @PostMapping("/{id}/roles")
    @PreAuthorize("hasAuthority('user:assign-role')")
    public void assignRole(@PathVariable UUID id, @RequestBody AssignRoleRequest req) {
        service.assignRole(id, req.role()); // service emits an AuditRecord
    }
}

```

4. Accounts & Wallets

What. An **Account** is the customer's banking relationship; a **Wallet** is a currency-scoped balance container under an account. Both have explicit lifecycle states. A wallet's **balance is never stored** — it is derived by summing ledger entries.

Why. Storing a mutable `balance` column invites drift: a bug, a missed update, or a partial failure leaves the number wrong with no way to detect it.

By deriving balance from an append-only ledger, the balance is always provably

the sum of immutable facts. Lifecycle states let us freeze a wallet (fraud hold) without deleting anything.

How. `AccountStatus` and `WalletStatus` are explicit enums with guarded transitions. The read path sums `ledger_entries`; for hot wallets a materialized balance projection can be added later as a cache that is *rebuildable* from the ledger, never the source of truth.

```
@Entity @Table(name = "accounts")
public class Account {
    @Id @GeneratedValue private UUID id;
    @Column(nullable = false) private UUID ownerUserId;

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private AccountStatus status;          // PENDING, ACTIVE, FROZEN, CLOSED

    @Column(nullable = false) private Instant createdAt;
    // accessors omitted
}

@Entity @Table(name = "wallets",
    uniqueConstraints = @UniqueConstraint(columnNames = {"accountId", "currency"}))
public class Wallet {
    @Id @GeneratedValue private UUID id;
    @Column(nullable = false) private UUID accountId;
    @Column(nullable = false, length = 3) private String currency;    // ISO-4217

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private WalletStatus status;          // ACTIVE, FROZEN, CLOSED
    // accessors omitted
}
```

Balance derived from the ledger

```
public interface LedgerEntryRepository extends JpaRepository<LedgerEntry, UUID> {

    @Query("""
        select coalesce(sum(case when e.direction = 'CREDIT' then e.amount
                                else -e.amount end), 0)
        from LedgerEntry e
        where e.walletId = :walletId
        """)
    BigDecimal deriveBalance(@Param("walletId") UUID walletId);
}
```

REST endpoints

```
@RestController
@RequestMapping("/v1/wallets")
public class WalletController {
```

```

private final WalletService wallets;

public WalletController(WalletService wallets) { this.wallets = wallets; }

@PostMapping
@PreAuthorize("hasAuthority('wallet:create')")
@ResponseStatus(HttpStatus.CREATED)
public WalletView open(@Valid @RequestBody OpenWalletRequest req) {
    return wallets.open(req.accountId(), req.currency());
}

@GetMapping("/{id}/balance")
@PreAuthorize("hasAuthority('wallet:read')")
public BalanceView balance(@PathVariable UUID id) {
    return new BalanceView(id, wallets.balance(id)); // summed from ledger
}
}

```

ENDPOINT	METHOD	PERMISSION	PURPOSE
<code>/v1/accounts</code>	POST	<code>account:create</code>	Open an account (PENDING)
<code>/v1/accounts/{id}/activate</code>	POST	<code>account:manage</code>	PENDING → ACTIVE (post-KYC)
<code>/v1/wallets</code>	POST	<code>wallet:create</code>	Open a currency wallet
<code>/v1/wallets/{id}/balance</code>	GET	<code>wallet:read</code>	Derived balance
<code>/v1/wallets/{id}/freeze</code>	POST	<code>wallet:manage</code>	ACTIVE → FROZEN

5. Ledger, Transactions & Transfers

What. A **double-entry, append-only** ledger. Every money movement writes

balanced entries (total debits == total credits). A **transfer** is a paired

debit + credit committed in a **single @Transactional** unit. Money-moving

POSTs carry an **idempotency key** enforced by a unique DB constraint.

Insufficient funds are rejected. Reversals are done with ****compensating**

entries**, never by mutating or deleting history.

Why.

- **Double-entry** makes every transaction self-balancing and auditable; the whole system's net is always zero, which is a continuously checkable invariant.

- **Append-only** means history is immutable — a correction is itself a recorded event, which is what auditors and regulators require.
- **One ACID transaction** is the simplest correct way to move money: partial states (debit without credit) can never be observed.
- **Idempotency keys** make retries safe. Networks fail after the server committed; the client retries; the unique constraint guarantees the second attempt returns the original result instead of double-charging.
- **BigDecimal** / **NUMERIC** because binary floating point cannot represent decimal money exactly — `0.1 + 0.2 != 0.3` is unacceptable in a ledger.

How. `TransferService.transfer(...)` runs in one transaction: insert an idempotency row (unique constraint), check funds via the derived balance, append two `LedgerEntry` rows in a shared `transactionId`, and write an `OutboxEvent` (see §8) — all committed together or all rolled back.

Entities

```
public enum Direction { DEBIT, CREDIT }

@Entity @Table(name = "ledger_entries") // append-only: no update/delete
public class LedgerEntry {
    @Id @GeneratedValue private UUID id;

    @Column(nullable = false) private UUID transactionId; // groups the paired legs
    @Column(nullable = false) private UUID walletId;

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private Direction direction;

    @Column(nullable = false, precision = 19, scale = 4) // NUMERIC(19,4)
    private BigDecimal amount;

    @Column(nullable = false, length = 3) private String currency;
    @Column(nullable = false) private Instant postedAt;
    @Column private String memo;
    // accessors; amount is always positive, sign comes from direction
}

@Entity @Table(name = "idempotency_keys",
    uniqueConstraints = @UniqueConstraint(name = "uq_idem_key", columnNames = "idemKey"))
public class IdempotencyRecord {
    @Id @GeneratedValue private UUID id;
    @Column(nullable = false) private String idemKey;
    @Column(nullable = false) private UUID transactionId; // the result to replay
    @Column(nullable = false) private Instant createdAt;
}
```

```
// accessors omitted
}
```

TransferService

```
@Service
public class DefaultTransferService implements TransferService {

    private final LedgerEntryRepository ledger;
    private final IdempotencyRepository idempotency;
    private final WalletRepository wallets;
    private final KycGate kycGate;
    private final OutboxWriter outbox;

    // constructor injection omitted

    @Transactional
    public TransferResult transfer(TransferCommand cmd) {

        // 1) Idempotency: insert-first. A duplicate key throws → we replay the prior result.
        try {
            idempotency.saveAndFlush(IdempotencyRecord.of(cmd.idempotencyKey(), cmd.transactionId()));
        } catch (DataIntegrityViolationException dup) {
            UUID existing = idempotency.findByIdemKey(cmd.idempotencyKey())
                .orElseThrow().getTransactionId();
            return TransferResult.replayed(existing);
        }

        Wallet from = wallets.getActive(cmd.fromWalletId());
        Wallet to = wallets.getActive(cmd.toWalletId());

        if (!from.getCurrency().equals(to.getCurrency())) {
            throw new ApiException("CURRENCY_MISMATCH", "Cross-currency not allowed", 422);
        }

        // 2) KYC gate: money-out requires a verified sender.
        kycGate.assertMoneyOutAllowed(from.getAccountId());

        // 3) Insufficient-funds check against the DERIVED balance.
        BigDecimal balance = ledger.deriveBalance(from.getId());
        if (balance.compareTo(cmd.amount()) < 0) {
            throw new ApiException("INSUFFICIENT_FUNDS",
                "Balance %s < amount %s".formatted(balance, cmd.amount()), 422);
        }

        // 4) Double-entry: two balanced legs sharing one transactionId.
        Instant now = Instant.now();
        ledger.save(leg(cmd.transactionId(), from.getId(), Direction.DEBIT, cmd.amount(), from.getCurrency(),
            now, cmd.memo()));
        ledger.save(leg(cmd.transactionId(), to.getId(), Direction.CREDIT, cmd.amount(), to.getCurrency(),
            now, cmd.memo()));

        // 5) Outbox row in the SAME transaction – side-effects fan out after commit.
        outbox.write("transfer.completed", cmd.transactionId(),
            Map.of("from", from.getId(), "to", to.getId(),
                "amount", cmd.amount(), "currency", from.getCurrency()));

        return TransferResult.posted(cmd.transactionId());
    }

    @Transactional
    public TransferResult reverse(UUID originalTxId, String reason) {
        List<LedgerEntry> legs = ledger.findByTransactionId(originalTxId);
        if (legs.isEmpty()) throw new ApiException("TX_NOT_FOUND", "Unknown transaction", 404);
    }
}
```

```

        UUID reversalTxId = UUID.randomUUID();
        Instant now = Instant.now();
        // Compensating entries: each original leg gets an opposite-direction leg.
        for (LedgerEntry e : legs) {
            Direction opp = e.getDirection() == Direction.DEBIT ? Direction.CREDIT : Direction.DEBIT;
            ledger.save(leg(reversalTxId, e.getWalletId(), opp, e.getAmount(),
                e.getCurrency(), now, "reversal:" + reason));
        }
        outbox.write("transfer.reversed", reversalTxId,
            Map.of("original", originalTxId, "reason", reason));
        return TransferResult.posted(reversalTxId);
    }

    private LedgerEntry leg(UUID txId, UUID walletId, Direction dir, BigDecimal amt,
        String ccy, Instant at, String memo) {
        LedgerEntry e = new LedgerEntry();
        e.setTransactionId(txId);
        e.setWalletId(walletId);
        e.setDirection(dir);
        e.setAmount(amt);
        e.setCurrency(ccy);
        e.setPostedAt(at);
        e.setMemo(memo);
        return e;
    }
}

```

Controller — idempotency key from header

```

@RestController
@RequestMapping("/v1/transfers")
public class TransferController {

    private final TransferService transfers;

    public TransferController(TransferService transfers) { this.transfers = transfers; }

    @PostMapping
    @PreAuthorize("hasAuthority('transfer:create')")
    public ResponseEntity<TransferResult> create(
        @RequestHeader("Idempotency-Key") @NotBlank String idemKey,
        @Valid @RequestBody TransferRequest req) {

        var cmd = new TransferCommand(UUID.randomUUID(), req.fromWalletId(),
            req.toWalletId(), req.amount(), idemKey, req.memo());
        TransferResult result = transfers.transfer(cmd);
        return ResponseEntity.status(result.replayed() ? HttpStatus.OK : HttpStatus.CREATED)
            .body(result);
    }
}

```

The matching Flyway constraint that makes idempotency airtight:

```

CREATE TABLE idempotency_keys (
    id                UUID PRIMARY KEY,
    idem_key          TEXT NOT NULL,
    transaction_id    UUID NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_idem_key UNIQUE (idem_key)  -- DB is the source of truth, not app memory
);

```

```
CREATE TABLE ledger_entries (
  id                UUID PRIMARY KEY,
  transaction_id    UUID NOT NULL,
  wallet_id         UUID NOT NULL,
  direction         TEXT NOT NULL CHECK (direction IN ('DEBIT','CREDIT')),
  amount            NUMERIC(19,4) NOT NULL CHECK (amount > 0),
  currency          CHAR(3) NOT NULL,
  posted_at        TIMESTAMPTZ NOT NULL,
  memo             TEXT
);
CREATE INDEX ix_ledger_wallet ON ledger_entries (wallet_id);
CREATE INDEX ix_ledger_tx     ON ledger_entries (transaction_id);
-- No UPDATE/DELETE grants on ledger_entries: append-only at the privilege level.
```

6. KYC

What. Know-Your-Customer onboarding modeled as a **case state machine**.

Customers submit identity documents; an async vendor verifies them; the case advances through states. Money-out is **gated** on a verified case.

Why. KYC is inherently asynchronous — a third-party vendor takes seconds to minutes. We must not block a money transaction on a vendor call, and we must not let unverified customers move funds. A state machine makes the allowed transitions explicit and prevents illegal jumps (e.g. **REJECTED → VERIFIED**).

How. **KycCase** holds a **status**. Document submission stores a reference and fires async vendor orchestration (the document bytes go to object storage; only a pointer is in PostgreSQL). The vendor callback (or polling worker) transitions the case. **KycGate** is consulted by **TransferService** before any money-out.

```
public enum KycStatus { CREATED, DOCS_SUBMITTED, UNDER_REVIEW, VERIFIED, REJECTED }

@Entity @Table(name = "kyc_cases")
public class KycCase {
  @Id @GeneratedValue private UUID id;
  @Column(nullable = false, unique = true) private UUID accountId;

  @Enumerated(EnumType.STRING) @Column(nullable = false)
  private KycStatus status;

  @Column private String vendorRef;      // external verification id
  @Column private String rejectReason;
  @Column(nullable = false) private Instant updatedAt;

  public void transitionTo(KycStatus next) {
```

```

        if (!ALLOWED.get(this.status).contains(next)) {
            throw new ApiException("KYC_ILLEGAL_TRANSITION",
                "%s -> %s not allowed".formatted(status, next), 409);
        }
        this.status = next;
        this.updatedAt = Instant.now();
    }

    private static final Map<KycStatus, Set<KycStatus>> ALLOWED = Map.of(
        KycStatus.CREATED, EnumSet.of(KycStatus.DOCS_SUBMITTED),
        KycStatus.DOCS_SUBMITTED, EnumSet.of(KycStatus.UNDER_REVIEW),
        KycStatus.UNDER_REVIEW, EnumSet.of(KycStatus.VERIFIED, KycStatus.REJECTED),
        KycStatus.VERIFIED, EnumSet.noneOf(KycStatus.class),
        KycStatus.REJECTED, EnumSet.of(KycStatus.DOCS_SUBMITTED)); // allow re-submit
}

```

Document submission + async vendor orchestration

```

@Service
public class DefaultKycService implements KycService {

    private final KycCaseRepository cases;
    private final DocumentStore docStore;
    private final KycVendorClient vendor;

    // constructor injection omitted

    @Transactional
    public void submitDocument(UUID accountId, DocumentUpload upload) {
        KycCase kyc = cases.findById(accountId).orElseThrow();
        String ref = docStore.put(upload.bytes(), upload.contentType()); // object storage
        kyc.transitionTo(KycStatus.DOCS_SUBMITTED);
        // Hand off asynchronously – do NOT block the request on the vendor.
        orchestrateAsync(kyc.getId());
    }

    @Async
    public void orchestrateAsync(UUID caseId) {
        cases.findById(caseId).ifPresent(kyc -> {
            kyc.transitionTo(KycStatus.UNDER_REVIEW);
            cases.save(kyc);
            VendorResult r = vendor.verify(kyc.getAccountId()); // network call off the hot path
            applyVendorResult(caseId, r);
        });
    }

    @Transactional
    public void applyVendorResult(UUID caseId, VendorResult r) {
        KycCase kyc = cases.findById(caseId).orElseThrow();
        kyc.setVendorRef(r.reference());
        kyc.transitionTo(r.approved() ? KycStatus.VERIFIED : KycStatus.REJECTED);
        if (!r.approved()) kyc.setRejectReason(r.reason());
        cases.save(kyc);
    }
}

```

KYC gate consulted by money-out

```

@Component
public class KycGate {

```

```

private final KycCaseRepository cases;

public KycGate(KycCaseRepository cases) { this.cases = cases; }

public void assertMoneyOutAllowed(UUID accountId) {
    KycStatus status = cases.findByAccountId(accountId)
        .map(KycCase::getStatus).orElse(KycStatus.CREATED);
    if (status != KycStatus.VERIFIED) {
        throw new ApiException("KYC_REQUIRED",
            "Account KYC not verified (" + status + ")", 403);
    }
}
}

```

7. Audit Trail

What. An **append-only** `audit_records` table capturing actor, action, before/after snapshots, and the `trace_id` of the request. Privileged actions write an audit record; there is a query endpoint for auditors.

Why. Auditability is a regulatory requirement and an incident-response tool. We want a tamper-evident "who did what, when, and what changed" record that is independent of application logs (which rotate and are mutable). Writing it inside the business transaction means an action and its audit entry commit together.

How. A lightweight AOP aspect intercepts methods annotated `@Audited` and writes an `AuditRecord`; high-stakes paths also call the `AuditService` explicitly with rich before/after payloads. The `trace_id` is pulled from the current OpenTelemetry span.

```

@Entity @Table(name = "audit_records") // append-only
public class AuditRecord {
    @Id @GeneratedValue private UUID id;
    @Column(nullable = false) private String actor; // user id or "system"
    @Column(nullable = false) private String action; // e.g. "user:assign-role"
    @Column(columnDefinition = "jsonb") private String beforeJson;
    @Column(columnDefinition = "jsonb") private String afterJson;
    @Column(nullable = false) private String traceId;
    @Column(nullable = false) private Instant at;
}

```



```
// accessors omitted
}
```

```
@Aspect @Component
public class AuditAspect {

    private final AuditService audit;

    public AuditAspect(AuditService audit) { this.audit = audit; }

    @AfterReturning(pointcut = "@annotation(audited)", returning = "result")
    public void record(JoinPoint jp, Audited audited, Object result) {
        audit.write(currentActor(), audited.action(), null, result);
    }

    private String currentActor() {
        Authentication a = SecurityContextHolder.getContext().getAuthentication();
        return a != null ? a.getName() : "system";
    }
}
```

```
@Service
public class DefaultAuditService implements AuditService {

    private final AuditRecordRepository repo;
    private final ObjectMapper mapper;
    private final Tracer tracer; // OpenTelemetry

    // constructor injection omitted

    @Transactional(propagation = Propagation.MANDATORY) // must join caller's TX
    public void write(String actor, String action, Object before, Object after) {
        AuditRecord r = new AuditRecord();
        r.setActor(actor);
        r.setAction(action);
        r.setBeforeJson(toJson(before));
        r.setAfterJson(toJson(after));
        r.setTraceId(Span.current().getSpanContext().getTraceId());
        r.setAt(Instant.now());
        repo.save(r);
    }

    private String toJson(Object o) {
        try { return o == null ? null : mapper.writeValueAsString(o); }
        catch (JsonProcessingException e) { return "\"<unserializable>\""; }
    }
}
```

Query endpoint

```
@RestController
@RequestMapping("/v1/audit")
public class AuditController {

    private final AuditRecordRepository repo;

    public AuditController(AuditRecordRepository repo) { this.repo = repo; }

    @GetMapping
```

```

@PreAuthorize("hasAuthority('audit:read')") // auditor role only
public Page<AuditRecord> search(
    @RequestParam(required = false) String actor,
    @RequestParam(required = false) String action,
    Pageable pageable) {
    return repo.search(actor, action, pageable);
}
}

```

8. Event-Driven / Transactional Outbox

What. Domain changes that need to fan out (notifications, projections, analytics) are recorded as an `OutboxEvent` written *inside the same DB transaction* as the change. A `@Scheduled` **relay** reads unpublished rows and publishes them to RabbitMQ. Consumers are **idempotent**, deduping by event id.

Why. The classic failure is "dual write": commit the DB row, then publish to the broker — and crash in between, losing the event (or publishing then failing the DB commit, emitting a phantom event). The outbox makes the event part of the ACID transaction, so it is published **exactly when** the change is durable. The relay provides at-least-once delivery; idempotent consumers make at-least-once behave like exactly-once. **Money is never moved by events** — only side-effects.

How. `OutboxWriter.write(...)` is called within `TransferService.transfer`

(§5). The relay polls, marks rows in-flight, publishes, and marks published.

Consumers record processed event ids and skip duplicates.

```

@Entity @Table(name = "outbox_events")
public class OutboxEvent {
    @Id @GeneratedValue private UUID id; // also the dedup key for consumers
    @Column(nullable = false) private String type; // "transfer.completed"
    @Column(nullable = false) private String aggregateId;
    @Column(nullable = false, columnDefinition = "jsonb") private String payload;

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private OutboxStatus status; // NEW, PUBLISHED

    @Column(nullable = false) private Instant createdAt;
    @Column private Instant publishedAt;
    // accessors omitted
}

```

Writing the outbox row inside the money transaction

```
@Component
public class OutboxWriter {

    private final OutboxEventRepository repo;
    private final ObjectMapper mapper;

    public OutboxWriter(OutboxEventRepository repo, ObjectMapper mapper) {
        this.repo = repo; this.mapper = mapper;
    }

    // No @Transactional here: it MUST run inside the caller's transaction (transfer).
    public void write(String type, UUID aggregateId, Map<String, Object> payload) {
        OutboxEvent e = new OutboxEvent();
        e.setType(type);
        e.setAggregateId(aggregateId.toString());
        e.setPayload(serialize(payload));
        e.setStatus(OutboxStatus.NEW);
        e.setCreatedAt(Instant.now());
        repo.save(e); // committed atomically with the ledger entries
    }

    private String serialize(Map<String, Object> p) {
        try { return mapper.writeValueAsString(p); }
        catch (JsonProcessingException ex) { throw new IllegalStateException(ex); }
    }
}
```

The @Scheduled relay

```
@Component
public class OutboxRelay {

    private final OutboxEventRepository repo;
    private final RabbitTemplate rabbit;

    public OutboxRelay(OutboxEventRepository repo, RabbitTemplate rabbit) {
        this.repo = repo; this.rabbit = rabbit;
    }

    @Scheduled(fixedDelay = 1000) // poll every second
    @Transactional
    public void publishBatch() {
        List<OutboxEvent> batch = repo.findTop100ByStatusOrderByCreatedAtAsc(OutboxStatus.NEW);
        for (OutboxEvent e : batch) {
            // event id rides as a message property so consumers can dedup.
            rabbit.convertAndSend("bank.events", e.getType(), e.getPayload(),
                m -> { m.getMessageProperties().setMessageId(e.getId().toString());
                        m.getMessageProperties().setContentType("application/json");
                        return m; });
            e.setStatus(OutboxStatus.PUBLISHED);
            e.setPublishedAt(Instant.now());
        }
    }
}
```

Idempotent consumer

```

@Component
public class TransferNotificationConsumer {

    private final ProcessedEventRepository processed;
    private final NotificationSender notifier;

    // constructor injection omitted

    @RabbitListener(queues = "notifications.transfer-completed")
    @Transactional
    public void onTransferCompleted(Message msg) {
        UUID eventId = UUID.fromString(msg.getMessageProperties().getMessageId());

        // Dedup: insert-first on a unique PK; duplicate => already handled, ack & skip.
        if (processed.existsById(eventId)) return;
        try {
            processed.saveAndFlush(new ProcessedEvent(eventId, Instant.now()));
        } catch (DataIntegrityViolationException dup) {
            return; // concurrent duplicate
        }
        notifier.sendTransferReceipt(new String(msg.getBody(), StandardCharsets.UTF_8));
    }
}

```

RabbitMQ topology declared once at startup:

```

@Configuration
public class RabbitConfig {

    @Bean TopicExchange eventsExchange() { return new TopicExchange("bank.events"); }

    @Bean Queue transferQueue() {
        return QueueBuilder.durable("notifications.transfer-completed")
            .deadLetterExchange("bank.events.dlx") // poison messages park here
            .build();
    }

    @Bean Binding transferBinding(Queue transferQueue, TopicExchange eventsExchange) {
        return BindingBuilder.bind(transferQueue).to(eventsExchange).with("transfer.completed");
    }
}

```

9. API Gateway & Rate Limiting

What. A separate **API gateway** sits in front of the app (TLS termination, routing, coarse rate limiting / WAF). Inside the application we keep a **per-principal rate limiter** (Bucket4j) as defense in depth, a standard **error envelope** `{code, message, trace_id}`, and a global

`@ControllerAdvice` exception handler.

Why. The gateway is the right place for network-edge concerns, but the app should not trust that it is always there — a second, application-aware limiter protects expensive endpoints (e.g. transfers) even if edge config drifts. A single error envelope means every client parses errors the same way, and the `trace_id` ties a client-visible error to server traces and audit records.

How. A `OncePerRequestFilter` keyed by the authenticated subject consumes a token bucket. The exception handler maps `ApiException` and validation errors to the envelope and stamps the current trace id.

Error envelope

```
public record ApiError(String code, String message, String traceId) {
    public static ApiError of(String code, String message) {
        return new ApiError(code, message, Span.current().getSpanContext().getTraceId());
    }
}
```

Global exception handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class)
    public ResponseEntity<ApiError> handleApi(ApiException ex) {
        return ResponseEntity.status(ex.getStatus())
            .body(ApiError.of(ex.getCode(), ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + " " + f.getDefaultMessage())
            .collect(Collectors.joining("; "));
        return ResponseEntity.badRequest().body(ApiError.of("VALIDATION", msg));
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ApiError> handleDenied(AccessDeniedException ex) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(ApiError.of("FORBIDDEN", "Permission denied"));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleUnexpected(Exception ex) {
        // never leak internals; trace_id lets support correlate to logs
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ApiError.of("INTERNAL", "Unexpected error"));
    }
}
```

```
}  
}
```

Bucket4j rate-limit filter

```
@Component  
public class RateLimitFilter extends OncePerRequestFilter {  
  
    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();  
  
    private Bucket newBucket() {  
        return Bucket.builder()  
            .addLimit(Bandwidth.classic(100, Refill.greedy(100, Duration.ofMinutes(1))))  
            .build();    // 100 requests / minute / principal  
    }  
  
    @Override  
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,  
                                   FilterChain chain) throws ServletException, IOException {  
        String key = principalKey(req);  
        Bucket bucket = buckets.computeIfAbsent(key, k -> newBucket());  
        if (bucket.tryConsume(1)) {  
            chain.doFilter(req, res);  
        } else {  
            res.setStatus(429);  
            res.setContentType("application/json");  
            res.getWriter().write("""  
                {"code":"RATE_LIMITED","message":"Too many requests","trace_id":"%s"}""")  
                .formatted(Span.current().getSpanContext().getTraceId());  
        }  
    }  
  
    private String principalKey(HttpServletRequest req) {  
        var auth = SecurityContextHolder.getContext().getAuthentication();  
        return auth != null ? auth.getName() : req.getRemoteAddr();  
    }  
}
```

10. Observability

What. Spring Boot **Actuator** exposes health and Prometheus-format metrics

via **Micrometer**; **OpenTelemetry** provides distributed tracing with the

`trace_id` propagated through logs, the error envelope, and audit records. We
add custom business counters (e.g. transfers posted, idempotent replays).

Why. A money system must be observable in production: we need to know

transfer throughput, failure rates, outbox lag, and be able to follow one

request end-to-end. Business metrics (not just CPU/memory) are what tell us the

platform is healthy.

How. Actuator endpoints are exposed selectively. A `MeterRegistry`-backed counter is incremented in `TransferService`. Trace context is auto-propagated by the OpenTelemetry Spring Boot starter; we just read the current trace id where needed.

application.yml

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      probes:
        enabled: true          # liveness/readiness for Kubernetes
  metrics:
    tags:
      application: bank-core
  tracing:
    sampling:
      probability: 1.0        # sample all in non-prod; lower in prod
```

Custom counter

```
@Service
public class TransferMetrics {

    private final Counter posted;
    private final Counter replayed;
    private final Counter insufficientFunds;

    public TransferMetrics(MeterRegistry registry) {
        this.posted = Counter.builder("bank.transfers.posted")
            .description("Successfully posted transfers").register(registry);
        this.replayed = Counter.builder("bank.transfers.replayed")
            .description("Idempotent replays").register(registry);
        this.insufficientFunds = Counter.builder("bank.transfers.rejected")
            .tag("reason", "insufficient_funds").register(registry);
    }

    public void posted() { posted.increment(); }
    public void replayed() { replayed.increment(); }
    public void insufficientFunds() { insufficientFunds.increment(); }
}
```

Tracing note. Each inbound request begins (or continues) a trace; the gateway injects W3C `traceparent` headers, the OpenTelemetry starter creates a

server span, and downstream RabbitMQ publishes propagate the context so an event consumed minutes later is still linked to the originating transfer. The same `trace_id` is what surfaces in `ApiError`, `AuditRecord`, and structured logs — one id correlates the whole story.

11. Testing Note — Invariants

What. Tests that prove the two non-negotiable properties: ****double-entry balances to zero and idempotency prevents double-posting****.

Why. These are the invariants the whole platform rests on. They must be asserted by automated tests, not assumed. A double-entry test that sums all legs of a transaction and expects zero will catch any future change that breaks balance. An idempotency test proves a retried request never moves money twice.

How. A Spring Boot integration test (Testcontainers PostgreSQL) runs a real transfer, then asserts (1) the two ledger legs net to zero, and (2) a second call with the same idempotency key produces no new entries and returns the original transaction.

```
@SpringBootTest
@Testcontainers
class TransferInvariantTest {

    @Container
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", db::getJdbcUrl);
        r.add("spring.datasource.username", db::getUsername);
        r.add("spring.datasource.password", db::getPassword);
    }

    @Autowired TransferService transfers;
    @Autowired LedgerEntryRepository ledger;
    @Autowired TestFixtures fx; // seeds verified accounts + funded wallets

    @Test
    void doubleEntry_balancesToZero() {
        var cmd = fx.transferCommand("100.00");
        transfers.transfer(cmd);
```



```

List<LedgerEntry> legs = ledger.findByTransactionId(cmd.transactionId());
assertThat(legs).hasSize(2);

BigDecimal net = legs.stream()
    .map(e -> e.getDirection() == Direction.CREDIT ? e.getAmount() : e.getAmount().negate())
    .reduce(BigDecimal.ZERO, BigDecimal::add);

assertThat(net).isEqualByComparingTo("0.00"); // debits == credits
}

@Test
void idempotencyKey_preventsDoublePost() {
    var cmd = fx.transferCommand("50.00");

    TransferResult first = transfers.transfer(cmd);
    TransferResult second = transfers.transfer(cmd); // same idempotency key

    assertThat(first.replayed()).isFalse();
    assertThat(second.replayed()).isTrue();
    assertThat(second.transactionId()).isEqualTo(first.transactionId());

    // Only ONE transaction's worth of legs exists.
    assertThat(ledger.findByTransactionId(cmd.transactionId())).hasSize(2);
}

@Test
void insufficientFunds_isRejected() {
    var cmd = fx.transferCommand("1000000.00"); // exceeds funded balance
    assertThatThrownBy(() -> transfers.transfer(cmd))
        .isInstanceOf(ApiException.class)
        .hasMessageContaining("INSUFFICIENT_FUNDS");
}
}

```

Summary

The backend is a Spring Boot 3 modular monolith where ****money is correct by construction****: every movement is a balanced, append-only, double-entry posting committed in one ACID transaction, protected by DB-enforced idempotency and a KYC gate. **Stateless RS256 JWTs** with rotating, reuse-detecting refresh tokens secure access, and **default-deny RBAC** binds endpoints to permissions.

Side-effects flow through a **transactional outbox** to RabbitMQ with idempotent consumers — never moving money. An **append-only audit trail**, **Actuator/Micrometer metrics**, **OpenTelemetry tracing**, and a standard `{code, message, trace_id}` error envelope make the system auditable and observable end-to-end.

Mobile App — Features & Implementation

Digital Banking Platform · Customer Mobile Client Status: Baseline · Owner: Mobile Lead Runtime:
React Native 0.74 · TypeScript 5 · iOS 15+ / Android 8+ (API 26+)

Overview

The customer-facing app is a single **React Native + TypeScript** codebase that ships to both iOS and Android. It is a **thin client over the** `/v1` **REST API** — it holds **no business logic** of its own. Money correctness, RBAC, the ledger, and KYC decisioning all live in the backend (see `09-backend-api.md`); the app's job is to authenticate securely, render state, and submit well-formed, **idempotent** requests.

Three rules drive every decision in this document:

1. **Secrets live in the OS secure enclave, never in JS.** The long-lived refresh token is stored in the iOS Keychain / Android Keystore; the short-lived access token lives only in memory.
 1. **Every money request is idempotent.** Transfers carry a client-generated `Idempotency-Key`, so a retry over a flaky mobile network can never double-charge.
 1. **The server is the source of truth.** The UI hides actions the user cannot take, but it assumes nothing — it always reflects what the API returns, including the standard error envelope `{code, message, trace_id}`.
- Each feature below is documented as **(a) what** it is, **(b) why** it is designed this way, **(c) how** it works, and **(d) real code**.

1. Project Structure & Architecture

What. A **feature-first** layout: each capability (auth, kyc, wallets, transfers) is a folder owning its screens, hooks, API calls, and types. Shared plumbing (the HTTP client, secure storage, the design system) lives under `lib/` and `ui/`.

Why feature-first. A change to transfers stays inside `features/transfers/` instead of being scattered across global `screens/`, `services/`, and `types/` folders. It mirrors the backend's package-by-feature layout, so the two codebases read the same way.

How — the shape of the tree.

```
src/
├── app/
│   ├── App.tsx           # providers: QueryClient, Auth, Navigation
│   └── navigation.tsx    # native-stack + tab navigators, route guards
├── lib/
│   ├── api/
│   │   ├── client.ts     # fetch wrapper: base URL, auth header, refresh, error envelope
│   │   ├── endpoints.ts  # typed /v1 endpoint helpers
│   │   └── types.ts       # DTOs mirrored from the backend (TokenPair, Wallet, ...)
│   ├── secure/
│   │   └── tokenStore.ts  # Keychain/Keystore-backed refresh-token storage
│   ├── security/
│   │   ├── integrity.ts   # jailbreak/root + debugger detection
│   │   ├── pinning.ts     # certificate pinning config
│   │   └── idempotency.ts  # UUID v4 key generation
│   └── features/
│       ├── auth/         # login, register, silent refresh, biometric unlock
│       ├── kyc/          # document + selfie capture and upload
│       ├── wallets/      # wallet list, balances, transaction history
│       ├── transfers/    # amount entry, confirm, idempotent submit
│       └── notifications/ # push registration + event handling
└── ui/                   # design system: Button, Money, Screen, theme tokens
```

Stack choices (and the honest reasoning).

CONCERN	CHOICE	WHY
Navigation	React Navigation (native-stack)	Native screen transitions; mature deep-link + guard support.
Server state	TanStack Query (React Query)	Caching, retries, and request dedup for a REST API — no Redux boilerplate for server data.
Local/UI state	Zustand	Tiny, hook-based store for session + ephemeral UI state.
Secure storage	<code>react-native-keychain</code>	Wraps iOS Keychain + Android Keystore/EncryptedSharedPreferences.
HTTP	<code>fetch</code> + a thin typed wrapper	No heavy client needed; one place to inject auth + refresh.
Forms/validation	React Hook Form + Zod	Client-side shape validation; the server still re-validates.

We deliberately do **not** reach for Redux, a GraphQL layer, or a native module we don't need — the API is a small typed REST surface (YAGNI).

2. Secure Token Storage & Session

What. The app receives a `TokenPair` — a short-lived **access JWT** and an opaque **refresh token** — from `/v1/auth/login`. The access token is kept in memory; the refresh token is written to the **OS secure enclave**.

Why. A stolen device backup or a compromised JS bundle must not yield a usable long-lived credential. The Keychain/Keystore is hardware-backed and survives app restarts without ever exposing the secret to JavaScript-readable storage (`AsyncStorage` is plain text and is explicitly **not** used for tokens). The access token is intentionally ephemeral — losing it on a cold start just triggers a silent refresh.

How.

- On login/refresh, persist only the refresh token in the enclave.
- Hold the access token in an in-memory Zustand store.
- On cold start, read the refresh token and call `/v1/auth/refresh` to mint a new

access token (silent login) before showing the home screen.

- On logout, delete the enclave entry and clear all cached PII.

Code — secure token store.

```
// lib/secure/tokenStore.ts
import * as Keychain from 'react-native-keychain';

const REFRESH_SERVICE = 'com.bank.platform.refresh';

export const tokenStore = {
  async saveRefreshToken(token: string): Promise<void> {
    await Keychain.setGenericPassword('refresh', token, {
      service: REFRESH_SERVICE,
      accessible: Keychain.ACCESSIBLE.WHEN_UNLOCKED_THIS_DEVICE_ONLY, // not in backups
      accessControl: Keychain.ACCESS_CONTROL.BIOMETRY_CURRENT_SET, // gate read on biometrics
    });
  },

  async readRefreshToken(): Promise<string | null> {
    const creds = await Keychain.getGenericPassword({ service: REFRESH_SERVICE });
    return creds ? creds.password : null;
  },

  async clear(): Promise<void> {
    await Keychain.resetGenericPassword({ service: REFRESH_SERVICE });
  },
};
```

Code — in-memory session (access token never touches disk).

```
// features/auth/session.ts
import { create } from 'zustand';

interface SessionState {
  accessToken: string | null;
  perms: string[]; // mirror of the JWT "perms" claim
  setSession: (accessToken: string, perms: string[]) => void;
  clear: () => void;
}

export const useSession = create<SessionState>((set) => ({
  accessToken: null,
  perms: [],
  setSession: (accessToken, perms) => set({ accessToken, perms }),
  clear: () => set({ accessToken: null, perms: [] }),
}));
```

3. The API Client — Auth, Silent Refresh, Error Envelope

What. A single typed `fetch` wrapper that (a) attaches the `Bearer` access token, (b) transparently refreshes once on a `401`, and (c) decodes the backend's standard error envelope `{code, message, trace_id}` into a typed `ApiError`.

Why. Centralizing auth + refresh in one place means no screen ever deals with token plumbing, and a **single-flight** refresh prevents a thundering herd of parallel refreshes when several requests `401` at once. Surfacing `trace_id` lets a user-reported error be correlated with backend logs.

How. Requests go through `apiFetch`. On `401`, a shared in-progress refresh promise is awaited (or started); on success the original request is retried once; on refresh failure the session is cleared and the user is bounced to login.

Code — typed error + client.

```
// lib/api/client.ts
import { useSession } from '../features/auth/session';
import { tokenStore } from '../secure/tokenStore';
import { decodeJwtPerms } from './jwt';

const BASE_URL = process.env.API_BASE_URL!; // e.g. https://api.bank.example/v1

export class ApiError extends Error {
  constructor(
    readonly code: string,
    message: string,
    readonly status: number,
    readonly traceId?: string,
  ) {
    super(message);
  }
}

let refreshInFlight: Promise<string> | null = null;

async function refreshAccessToken(): Promise<string> {
  if (refreshInFlight) return refreshInFlight; // single-flight
  refreshInFlight = (async () => {
    const refreshToken = await tokenStore.readRefreshToken();
    if (!refreshToken) throw new ApiError('AUTH_NO_SESSION', 'No session', 401);

    const res = await fetch(`${BASE_URL}/auth/refresh`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ refreshToken }),
    });
    if (!res.ok) { // refresh reuse/expiry → hard logout
      await tokenStore.clear();
      useSession.getState().clear();
    }
  })();
}
```

```

        throw new ApiError('AUTH_REFRESH_FAILED', 'Please sign in again', 401);
    }
    const pair = await res.json(); // { accessToken, refreshToken }
    await tokenStore.saveRefreshToken(pair.refreshToken);
    useSession.getState().setSession(pair.accessToken, decodeJwtPerms(pair.accessToken));
    return pair.accessToken as string;
})();
try { return await refreshInFlight; } finally { refreshInFlight = null; }
}

export async function apiFetch<T>(
  path: string,
  init: RequestInit = {},
  retry = true,
): Promise<T> {
  const token = useSession.getState().accessToken;
  const res = await fetch(`${BASE_URL}${path}`, {
    ...init,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...init.headers,
    },
  });

  if (res.status === 401 && retry) {
    await refreshAccessToken();
    return apiFetch<T>(path, init, false); // retry exactly once
  }

  if (!res.ok) {
    const body = await res.json().catch(() => ({})); // { code, message, trace_id }
    throw new ApiError(body.code ?? 'UNKNOWN', body.message ?? 'Request failed',
      res.status, body.trace_id);
  }
  return (res.status === 204 ? undefined : await res.json()) as T;
}

```

4. Login, Registration & Biometric Unlock

What. Email/password login against `/v1/auth/login`, registration via

`/v1/auth/register`, and a **biometric (Face ID / fingerprint) unlock** gate on

app open that authorizes reading the refresh token from the enclave.

Why. Biometrics bind the stored refresh token to the physical device owner — even an unlocked phone left on a table cannot silently resume a banking session.

How. On launch, attempt a biometric-gated read of the refresh token; success

→ silent refresh → home. Failure or no stored token → the login screen.

Code — login mutation + silent resume.

```
// features/auth/useLogin.ts
import { useMutation } from '@tanstack/react-query';
import { apiFetch } from '../../lib/api/client';
import { tokenStore } from '../../lib/secure/tokenStore';
import { useSession } from './session';
import { decodeJwtPerms } from '../../lib/api/jwt';

interface TokenPair { accessToken: string; refreshToken: string; }

export function useLogin() {
  const setSession = useSession((s) => s.setSession);
  return useMutation({
    mutationFn: (vars: { email: string; password: string }) =>
      apiFetch<TokenPair>('/auth/login', { method: 'POST', body: JSON.stringify(vars) }),
    onSuccess: async (pair) => {
      await tokenStore.saveRefreshToken(pair.refreshToken); // biometric-gated enclave
      setSession(pair.accessToken, decodeJwtPerms(pair.accessToken));
    },
  });
}
```

```
// features/auth/resumeSession.ts – runs once on cold start
import { tokenStore } from '../../lib/secure/tokenStore';
import { apiFetch } from '../../lib/api/client';
import { useSession } from './session';
import { decodeJwtPerms } from '../../lib/api/jwt';

export async function resumeSession(): Promise<boolean> {
  const refreshToken = await tokenStore.readRefreshToken(); // prompts Face ID / fingerprint
  if (!refreshToken) return false;
  try {
    const pair = await apiFetch<{ accessToken: string; refreshToken: string }>({
      '/auth/refresh', { method: 'POST', body: JSON.stringify({ refreshToken }) });
    await tokenStore.saveRefreshToken(pair.refreshToken);
    useSession.getState().setSession(pair.accessToken, decodeJwtPerms(pair.accessToken));
    return true;
  } catch {
    return false; // expired/reused → show login
  }
}
```

5. KYC — Document & Selfie Capture

What. A guided flow that captures an identity document and a selfie, uploads

them, and then **polls / waits for a push** on the KYC decision. Maps to the

backend `/v1/kyc` endpoints and the `KYC_CASES` lifecycle

(`PENDING` → `VERIFIED` | `REJECTED`).

Why. The app **never decides** KYC — it only collects evidence and reflects

status. Capture quality (glare, framing) is validated on-device to cut rejections,

but the verdict is the backend's (and a compliance officer's) call.

How. Camera capture → client-side quality check → multipart upload → status

screen that subscribes to a `KycDecided` push and falls back to polling

`/v1/kyc/status` .

Code — KYC status hook.

```
// features/kyc/useKycStatus.ts
import { useQuery } from '@tanstack/react-query';
import { apiFetch } from '../../lib/api/client';

type KycStatus = 'NOT_STARTED' | 'PENDING' | 'VERIFIED' | 'REJECTED';

export function useKycStatus() {
  return useQuery({
    queryKey: ['kyc', 'status'],
    queryFn: () => apiFetch<{ status: KycStatus; reason?: string }>('/kyc/status'),
    // poll while a decision is outstanding; push will also invalidate this key
    refetchInterval: (q) => (q.state.data?.status === 'PENDING' ? 15_000 : false),
  });
}
```

6. Wallets & Transaction History

What. The home surface: a list of the customer's wallets with ****server-derived**

balances** (`GET /v1/wallets/{id}/balance`) and paginated transaction history.

Why. Balances are computed by the backend as `SUM(ledger_entries)` — the app displays them but never computes or caches a stale balance. History is read-only and paginated to keep memory flat on long-lived accounts.

How. TanStack Query caches per-wallet balance and history; pull-to-refresh and push-driven invalidation keep them current.

Code — wallets + balance.

```
// features/wallets/useWallets.ts
import { useQuery } from '@tanstack/react-query';
import { apiFetch } from '../../lib/api/client';

export interface Wallet {
  id: string; currency: string; status: 'ACTIVE' | 'FROZEN' | 'CLOSED';
}
```

```
export function useWallets() {
  return useQuery({ queryKey: ['wallets'], queryFn: () => apiFetch<Wallet[]>('/wallets') });
}

export function useWalletBalance(walletId: string) {
  return useQuery({
    queryKey: ['wallets', walletId, 'balance'],
    queryFn: () => apiFetch<{ amount: string; currency: string }>(
      `/wallets/${walletId}/balance`),
    staleTime: 10_000,
  });
}
```

Money is a string, never a `number` . Amounts come over the wire as decimal strings (mirroring the backend's `BigDecimal` / `NUMERIC(20,4)`). The app formats with `Intl.NumberFormat` and never does floating-point arithmetic on balances.

7. Transfers — On-Device Idempotency

What. The money-moving flow: choose source wallet, enter amount + destination, confirm, and submit `POST /v1/transfers` with a client-generated `Idempotency-Key` header.

Why — the core safety property. Mobile networks drop responses. If the app resends a transfer after a timeout, the backend must treat the **second** request as the **same** operation, not a new one. The backend enforces this with a unique constraint on the idempotency key; the app's job is to generate the key ****once per user intent**** and reuse it across every retry of that intent.

How.

- Generate a UUID v4 key the moment the user taps **Confirm** (not per network attempt).
- Send it as `Idempotency-Key` . On a network failure, retry with the **same** key.
- A `201` returns the completed transfer; a `422` carries a typed reason (`frozen` , `unverified` , `insufficient_funds`) shown to the user.

Code — idempotent transfer mutation.

```
// features/transfers/useTransfer.ts
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { apiFetch, ApiError } from '../../lib/api/client';
import { newIdempotencyKey } from '../../lib/idempotency';

interface TransferInput { fromWalletId: string; toWalletId: string; amount: string; }
interface TransferResult { id: string; status: 'COMPLETED'; transactionId: string; }

export function useTransfer() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (input: TransferInput) => {
      // key is bound to THIS confirmed intent; React Query retries reuse it
      const idempotencyKey = newIdempotencyKey();
      return apiFetch<TransferResult>('/transfers', {
        method: 'POST',
        headers: { 'Idempotency-Key': idempotencyKey },
        body: JSON.stringify(input),
      });
    },
    retry: (count, err) => {
      // retry only transient network/5xx – never a 4xx business rejection
      count < 2 && !(err instanceof ApiError && err.status < 500),
    },
    onSuccess: (_res, input) => {
      qc.invalidateQueries({ queryKey: ['wallets', input.fromWalletId, 'balance'] });
      qc.invalidateQueries({ queryKey: ['wallets', input.toWalletId, 'balance'] });
    },
  });
}
```

```
// lib/idempotency.ts
import 'react-native-get-random-values';
import { v4 as uuidv4 } from 'uuid';
export const newIdempotencyKey = (): string => uuidv4();
```

8. Push Notifications (Event-Fed)

What. Device push for `TransferCompleted`, `KycDecided`, and security events.

These are downstream consumers of the backend's **outbox events** — the app is a subscriber, not a poller, for real-time updates.

Why. Push closes the loop opened by event-driven design: a money event commits in the backend, the outbox relay fans it out, a notification consumer turns it into a device push. The user sees "transfer completed" without holding a screen open.

How. Register the FCM/APNs token after login; on receipt, invalidate the

relevant React Query keys so the UI refetches authoritative state (the push is a **signal**, never the source of truth for balances).

Code — token registration + handler.

```
// features/notifications/register.ts
import messaging from '@react-native-firebase/messaging';
import { apiFetch } from '../../lib/api/client';
import { queryClient } from '../../lib/api/queryClient';

export async function registerPush(): Promise<void> {
  await messaging().requestPermission();
  const deviceToken = await messaging().getToken();
  await apiFetch('/notifications/devices', {
    method: 'POST', body: JSON.stringify({ deviceToken, platform: 'mobile' }),
  });

  messaging().onMessage(async (msg) => {
    switch (msg.data?.event) {
      case 'TransferCompleted':
        queryClient.invalidateQueries({ queryKey: ['wallets'] });
        break;
      case 'KycDecided':
        queryClient.invalidateQueries({ queryKey: ['kyc', 'status'] });
        break;
    }
  });
}
```

9. Client-Side Hardening

What. Defense-in-depth on the device: TLS **certificate pinning**, ****jailbreak**

/ root + debugger detection**, screen-capture protection on sensitive screens, and auto-logout on background timeout.

Why. A banking client runs on hostile, sometimes-compromised hardware. These controls raise the cost of MITM and tampering — but they are **belt-and-suspenders**: the server still authenticates and authorizes every request, so a bypass on the device never grants money-moving power it didn't already have.

How.

- **Pinning:** pin the API's leaf/intermediate public-key hashes; a mismatch aborts

the request before any token is sent.

- **Integrity:** on launch, check for jailbreak/root and an attached debugger; in production, degrade to a restricted/blocked state and report.
- **PII hygiene:** mark balance/transaction screens as secure (`FLAG_SECURE` on Android, blur on iOS app switcher), and clear caches on logout.

Code — integrity gate.

```
// lib/security/integrity.ts
import JailMonkey from 'jail-monkey';

export interface IntegrityResult { safe: boolean; reasons: string[]; }

export function checkDeviceIntegrity(): IntegrityResult {
  const reasons: string[] = [];
  if (JailMonkey.isJailBroken()) reasons.push('jailbroken_or_rooted');
  if (JailMonkey.isDebuggedMode?.()) reasons.push('debugger_attached');
  if (JailMonkey.hookDetected?.()) reasons.push('hooking_framework');
  return { safe: reasons.length === 0, reasons };
}
```

In production a failed integrity check blocks money-moving actions and logs the reasons (with `trace_id`) to the backend; in development it only warns, so the simulator stays usable.

10. Testing Strategy (Client)

What. A pyramid matching the backend's: many fast unit tests, a focused band of component tests, and a thin layer of end-to-end device tests.

LAYER	TOOL	WHAT IT COVERS
Unit	Jest	<code>apiFetch</code> refresh/retry logic, idempotency-key generation, money formatting, reducers.
Component	React Native Testing Library	Screens render correct state for loading/error/success; RBAC-hidden actions stay hidden.
End-to-end	Detox	Login → KYC → transfer happy path on a simulator/emulator; retry-after-timeout does not double-charge.
Contract	Shared DTO types + MSW mocks	App DTOs stay in lockstep with the backend <code>/v1</code> contract.

Critical test — a retried transfer charges once.

```
// features/transfers/__tests__/idempotency.test.ts
it('reuses the idempotency key across a network-timeout retry', async () => {
  const seenKeys: string[] = [];
  server.use(
    http.post('*/transfers', ({ request }) => {
      seenKeys.push(request.headers.get('Idempotency-Key')!);
      return seenKeys.length === 1
        ? HttpResponse.error() // first attempt: network drop
        : HttpResponse.json({ id: 't1', status: 'COMPLETED', transactionId: 'x1' },
          { status: 201 });
    }),
  );

  const { result } = renderHook(() => useTransfer(), { wrapper });
  await act(() => result.current.mutateAsync(
    { fromWalletId: 'w1', toWalletId: 'w2', amount: '25.00' }));

  expect(seenKeys).toHaveLength(2);
  expect(new Set(seenKeys).size).toBe(1); // SAME key on retry
});
```

11. Decision Summary

DECISION	CHOICE
Framework	React Native + TypeScript (single iOS + Android codebase)
Navigation	React Navigation (native-stack)
Server state / cache	TanStack Query (retries, dedup, invalidation)
Refresh-token storage	iOS Keychain / Android Keystore (biometric-gated)
Access-token storage	In memory only — never persisted
Auth model	Access JWT (<code>Bearer</code>) + opaque rotating refresh token
Money safety	Client <code>Idempotency-Key</code> per confirmed transfer intent
Money representation	Decimal strings , <code>Intl.NumberFormat</code> — never <code>number</code>
Real-time updates	Push (outbox-fed) signals → query invalidation
Hardening	TLS pinning, jailbreak/root + debugger detection, FLAG_SECURE
Testing	Jest + RN Testing Library + Detox + contract mocks (MSW)

Bottom line: the mobile app is a secure, idempotent, server-authoritative client. It protects the refresh token in hardware, makes every money request safe to retry, formats money exactly, and treats push as a hint to refetch the truth — never as the truth itself.

Admin Dashboard — Features & Implementation

Digital Banking Platform · Operator Web Console Status: Baseline · Owner: Frontend Lead Runtime: React 18 · TypeScript 5 · Vite · Static build (S3 + CloudFront)

Overview

The admin dashboard is the **operator console** — the internal web app that support, operations, and compliance staff use to search customers, inspect entities, and take **audited, RBAC-gated actions** (suspend an account, freeze a wallet, reverse a transaction, override a KYC decision). It is built with **React 18 + TypeScript, bundled by Vite, and shipped as a static build** to S3 + CloudFront (see `08-deployment-infra.md`) — it runs no server of its own. Like the mobile app, it is a **thin, typed client over the `/v1` REST API** and holds **no business logic**. Three rules govern it:

1. **RBAC is a server guarantee; the UI only mirrors it.** The console hides or disables actions the operator's role lacks — purely for usability — but the backend independently enforces every permission via `@PreAuthorize`. UI hiding is **convenience, not security**.

1. **Every operator action is audited.** Sensitive actions require a typed **reason** and produce an `AuditRecord` (actor + reason + before/after + `trace_id`) on the backend. The UI is built around capturing that reason.

1. **The API is the source of truth.** All state is server state; the console caches and renders it, and surfaces the standard error envelope `{code, message, trace_id}` for support correlation.

Each feature is documented **(a) what, (b) why, (c) how, (d) code**.

1. Project Structure & Architecture

What. A **feature-first** React app: each capability (auth, search, customers, wallets, transactions, kyc, audit) owns its routes, components, hooks, and types.

Shared plumbing lives in `lib/` ; the design system in `ui/` .

Why. It mirrors the backend's package-by-feature and the mobile app's layout, so an engineer moving between the three codebases finds the same shape. A change to "reverse a transaction" lives entirely in `features/transactions/` .

How — the tree.

```
src/
├── app/
│   ├── main.tsx          # React root, providers (QueryClient, Router, Auth)
│   ├── router.tsx        # createBrowserRouter + permission-guarded routes
│   └── AppShell.tsx       # nav, health panel, role-aware menu
├── lib/
│   ├── api/
│   │   ├── client.ts     # fetch wrapper: auth header, refresh, error envelope
│   │   └── types.ts       # DTOs mirrored from /v1
│   └── auth/
│       ├── session.ts     # in-memory access token + perms
│       ├── usePermission.ts # has('wallet:manage') helper
│       └── RequirePermission.tsx # route + element guard
├── features/
│   ├── auth/              # login, silent refresh
│   ├── search/            # global entity search
│   ├── customers/         # detail, suspend/close
│   ├── wallets/           # detail, freeze/unfreeze
│   ├── transactions/      # detail, reversal (compensating entries)
│   ├── kyc/               # case review, compliance override
│   └── audit/             # per-entity audit trail viewer
└── ui/                    # Button, DataTable, Money, ReasonDialog, theme
```

Stack choices.

CONCERN	CHOICE	WHY
Build	Vite	Fast dev server + optimized static bundle for S3/CloudFront.
Routing	React Router (data router)	Nested routes + loader/guard hooks for RBAC gating.
Server state	TanStack Query	Caching, background refetch, mutation invalidation.
Forms	React Hook Form + Zod	Reason capture + client validation (server re-validates).
Tables	TanStack Table	Sortable/paginated operator data grids.
Styling	CSS Modules + design tokens	Scoped styles; tokens shared with the brand.

No Redux, no SSR framework — this is an internal SPA over a small REST surface (KISS / YAGNI). Auth, refresh, and the error envelope reuse the **same client pattern** documented for the mobile app ([10-mobile-app.md](#) §3).

2. Authentication & In-Memory Session

What. Operators log in via [/v1/auth/login](#) ; the access JWT is held **in memory and the refresh token in a** [Secure; HttpOnly; SameSite](#) cookie set by the backend. Silent refresh on [401](#) mirrors the mobile client.

Why — why a cookie here, not Keychain. A browser has no secure enclave. The safest place for a long-lived refresh token in a web app is an **HttpOnly cookie** the JavaScript cannot read (mitigating XSS token theft), scoped and [SameSite](#) to blunt CSRF. The short-lived access token stays in memory and dies on tab close.

How. Login returns the access token (JSON) and sets the refresh cookie. The API client attaches [Authorization: Bearer <accessToken>](#) ; on a [401](#) it calls [/v1/auth/refresh](#) (the browser sends the cookie automatically), single-flight,

then retries once.

Code — session + perms.

```
// lib/auth/session.ts
import { create } from 'zustand';

interface SessionState {
  accessToken: string | null;
  perms: Set<string>; // decoded from the JWT "perms" claim
  operator: { id: string; email: string } | null;
  set: (accessToken: string, perms: string[], operator: SessionState['operator']) => void;
  clear: () => void;
}

export const useSession = create<SessionState>((set) => ({
  accessToken: null,
  perms: new Set(),
  operator: null,
  set: (accessToken, perms, operator) =>
    set({ accessToken, perms: new Set(perms), operator }),
  clear: () => set({ accessToken: null, perms: new Set(), operator: null }),
}));
```

3. RBAC in the UI (Mirror, Not Gate)

What. A `usePermission` hook and a `RequirePermission` wrapper drive what the operator can **see**. Both route-level guards and inline action buttons consult the same permission set decoded from the JWT.

Why. Showing a "Reverse transaction" button to someone who will get a `403` is bad UX; hiding it is good UX. But the button being hidden is **not** what keeps money safe — the backend's `@PreAuthorize("hasAuthority('transaction:reverse')")` is. The UI and the server read the **same permission names**, so they agree without the UI ever being trusted.

How. `usePermission` checks the in-memory perm set. `RequirePermission` guards routes (redirect/403 page) and can wrap any element to hide it.

Code — permission helpers.

```
// lib/auth/usePermission.ts
import { useSession } from './session';
export function usePermission() {
```

```
const perms = useSession((s) => s.perms);
return {
  has: (perm: string) => perms.has(perm),
  hasAny: (...ps: string[]) => ps.some((p) => perms.has(p)),
};
}
```

```
// lib/auth/RequirePermission.tsx
import { ReactNode } from 'react';
import { Navigate } from 'react-router-dom';
import { usePermission } from './usePermission';

export function RequirePermission(
  { perm, children, mode = 'route' }:
  { perm: string; children: ReactNode; mode?: 'route' | 'element' },
) {
  const { has } = usePermission();
  if (has(perm)) return <{children}>{/>;
  return mode === 'route' ? <Navigate to="/403" replace /> : null; // element: render nothing
}
```

```
// usage – the button simply isn't rendered without the permission
<RequirePermission perm="transaction:reverse" mode="element">
  <Button variant="danger" onClick={openReverseDialog}>Reverse transaction</Button>
</RequirePermission>
```

4. Global Search

What. A single search surface over customers, wallets, transactions, and KYC

cases, routing to typed detail screens.

Why. Operators start from "find this entity," not from a menu. One fast search beats four siloed lookups.

How. Debounced query against the backend search endpoint; results are a tagged union rendered by type; selecting a result navigates to its detail route.

Code — search hook.

```
// features/search/useSearch.ts
import { useQuery } from '@tanstack/react-query';
import { apiFetch } from '../../lib/api/client';

type Hit =
  | { type: 'customer'; id: string; email: string; status: string }
  | { type: 'wallet'; id: string; currency: string; status: string }
  | { type: 'transaction'; id: string; amount: string; status: string }
```

```

    | { type: 'kyc'; id: string; customerId: string; status: string };

export function useSearch(term: string) {
  return useQuery({
    queryKey: ['search', term],
    queryFn: () => apiFetch<Hit[]>(`/search?q=${encodeURIComponent(term)}`),
    enabled: term.trim().length >= 2,
  });
}

```

5. Entity Detail & Audit Trail

What. Each entity (customer, wallet, transaction, KYC case) has a detail screen

showing current state **plus its append-only audit trail** from `/v1/audit`.

Why. In a regulated platform, "who did what, when, and why" is a first-class view. The audit trail is read-only and reflects the backend's immutable

`audit_records` (actor, action, before/after, `trace_id`).

How. Two queries per detail screen — the entity and its audit records — shown side by side. Audit rows render the diff (`before` → `after`) and the captured reason.

Code — audit trail.

```

// features/audit/useAuditTrail.ts
import { useQuery } from '@tanstack/react-query';
import { apiFetch } from '../../lib/api/client';

export interface AuditRecord {
  id: string; actorId: string; action: string;
  entityType: string; entityId: string;
  before: unknown; after: unknown; traceId: string; createdAt: string;
}

export function useAuditTrail(entityType: string, entityId: string) {
  return useQuery({
    queryKey: ['audit', entityType, entityId],
    queryFn: () => apiFetch<AuditRecord[]>(
      `/audit?entityType=${entityType}&entityId=${entityId}`),
  });
}

```

6. Operator Actions (Audited Mutations)

What. The console's reason-for-being: state-changing operations, each of which maps to a guarded backend endpoint and **requires a typed reason**:

ACTION	ENDPOINT	PERMISSION
Suspend / close account	POST /v1/accounts/{id}/...	account:manage
Freeze / unfreeze wallet	POST /v1/wallets/{id}/freeze	wallet:manage
Reverse transaction	POST /v1/transactions/{id}/reverse	transaction:reverse
KYC compliance override	POST /v1/kyc/{id}/decision	kyc:override

Why. Every one of these moves real-world risk. The shared **ReasonDialog** makes the reason mandatory at the UI layer, and the backend records it into the audit trail — so an action is never takeable without an accountable "why."

How. A reusable `useOperatorAction` mutation wraps `ReasonDialog`, submits the action with its reason, then invalidates the affected entity + its audit trail so the screen reflects the new state and the new audit row immediately.

Code — freeze wallet with mandatory reason.

```
// features/wallets/useFreezeWallet.ts
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { apiFetch } from '../../lib/api/client';

export function useFreezeWallet(walletId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (reason: string) => {
      apiFetch(`/wallets/${walletId}/freeze`, {
        method: 'POST',
        body: JSON.stringify({ reason }), // reason → AuditRecord on the backend
      }),
    },
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ['wallet', walletId] });
      qc.invalidateQueries({ queryKey: ['audit', 'wallet', walletId] });
    },
  });
}
```

```
// features/wallets/FreezeWalletButton.tsx
import { useState } from 'react';
import { ReasonDialog } from '../../ui/ReasonDialog';
import { RequirePermission } from '../../lib/auth/RequirePermission';
import { useFreezeWallet } from './useFreezeWallet';

export function FreezeWalletButton({ walletId }: { walletId: string }) {
  const [open, setOpen] = useState(false);
  const freeze = useFreezeWallet(walletId);
  return (
    <RequirePermission perm="wallet:manage" mode="element">
      <button onClick={() => setOpen(true)}>Freeze wallet</button>
      <ReasonDialog
        open={open}
        title="Freeze wallet"
        confirmLabel="Freeze"
        onCancel={() => setOpen(false)}
        onConfirm={async (reason) => { await freeze.mutateAsync(reason); setOpen(false); }}
      />
    </RequirePermission>
  );
}
```

Transaction reversal is not a delete. The backend reverses by posting **compensating ledger entries** (a new [REVERSAL](#) transaction linked via [reverses_id](#)), preserving the immutable ledger. The UI presents it as "reverse" but never implies the original record is erased.

7. Health Panel & Observability Links

What. A dashboard home panel showing system health and ****deep links into Grafana**** for metrics, rather than re-implementing charts.

Why. The platform already exposes Prometheus + Grafana (see [08-deployment-infra.md](#)). Rebuilding those dashboards in React would duplicate effort and drift from the source of truth — so the console **links out** to the canonical observability stack and only shows a lightweight liveness summary.

How. A small health query hits the backend's health summary; Grafana links are environment-configured URLs.

```
// features/home/useHealth.ts
import { useQuery } from '@tanstack/react-query';
import { apiFetch } from '../../lib/api/client';

export function useHealth() {
```

```

return useQuery({
  queryKey: ['health'],
  queryFn: () => apiFetch<{ status: 'UP' | 'DEGRADED' | 'DOWN'; broker: string; db: string }>(
    '/admin/health'),
  refetchInterval: 30_000,
});
}

```

8. Build & Deployment

What. Vite produces a hashed static bundle deployed to **S3 + CloudFront**;

there is **no app server**. The console talks to the API origin over HTTPS only.

Why. A static SPA is the cheapest, most cacheable, most attack-surface-minimal way to ship an internal console. CloudFront handles TLS, caching, and (with WAF) edge filtering; no compute to patch.

How.

- `vite build` → `dist/` with content-hashed assets and an `index.html` entry.
- Upload to S3; CloudFront serves it; SPA fallback routes 404s to `index.html`.
- Runtime config (API base URL, Grafana URLs) injected via build-time env

(`import.meta.env`) per environment.

```

// build pipeline (conceptual)
// 1. npm ci && npm run build          → dist/
// 2. aws s3 sync dist/ s3://bank-admin-dashboard --delete
// 3. aws cloudfront create-invalidation --paths "/index.html"

```

The dashboard is deployed **outside** Kubernetes — it is static hosting, not a pod. This is called out explicitly in `08-deployment-infra.md` and `12-gitops-deployment.md`.

9. Testing Strategy (Console)

LAYER	TOOL	WHAT IT COVERS
Unit	Vitest	API client refresh/retry, permission helpers, money/diff formatting.
Component	React Testing Library	RBAC-gated elements hide without permission; ReasonDialog enforces a non-empty reason.
Integration	MSW (mock <code>/v1</code>)	Search → detail → audited action → trail refresh happy path.
E2E	Playwright	Operator login → freeze wallet → audit row appears.

Critical test — an action button is hidden without its permission.

```
// features/wallets/__tests__/rbac.test.tsx
it('hides Freeze wallet when operator lacks wallet:manage', () => {
  seedSession({ perms: ['wallet:read'] }); // no wallet:manage
  render(<FreezeWalletButton walletId="w1" />, { wrapper });
  expect(screen.queryByText('Freeze wallet')).toBeNull(); // not rendered
});

it('requires a reason before confirming a freeze', async () => {
  seedSession({ perms: ['wallet:manage'] });
  render(<FreezeWalletButton walletId="w1" />, { wrapper });
  await userEvent.click(screen.getByText('Freeze wallet'));
  expect(screen.getByRole('button', { name: 'Freeze' })).toBeDisabled(); // empty reason
});
```

10. Decision Summary

DECISION	CHOICE
Framework	React 18 + TypeScript
Build / bundler	Vite → static bundle
Hosting	S3 + CloudFront (no app server, outside k8s)
Routing	React Router (data router) with permission guards
Server state / cache	TanStack Query
Access-token storage	In memory only
Refresh-token storage	<code>Secure; HttpOnly; SameSite</code> cookie (set by backend)
RBAC in UI	Mirror of server perms — UI hides, server enforces
Operator actions	Mandatory typed reason → audited (<code>AuditRecord</code>)
Transaction reversal	Compensating ledger entries, never a delete
Observability	Links out to Grafana; no re-implemented charts
Testing	Vitest + RTL + MSW + Playwright

Bottom line: the dashboard is a static, server-authoritative operator console. It mirrors RBAC for usability while the backend enforces it for real, makes every sensitive action carry an audited reason, and links out to the existing observability stack instead of duplicating it.

GitOps Deployment on AWS EKS

Digital Banking Platform · Continuous Delivery with Argo CD Status: Baseline · Owner: Platform / DevOps

This document describes how the modular-monolith backend, the supporting event-relay process, and the surrounding platform are built into images, declared as Kubernetes manifests, and continuously reconciled onto **AWS EKS** using a **GitOps** workflow driven by **Argo CD**. It complements the cloud infrastructure document (file O8) and the backend API document (file O9). For each piece we cover **what** it is, **why** it is designed this way (reasoning and trade-offs), **how** it works, and the **real config/code** that implements it.

1. Overview & GitOps Principles

What. GitOps is an operating model where the *desired state* of the whole runtime system is declared in Git, and an in-cluster controller (Argo CD) continuously reconciles the live cluster to match Git. You never run `kubectl apply` against production by hand; instead you open a pull request, get it reviewed, merge it, and the controller rolls the change out.

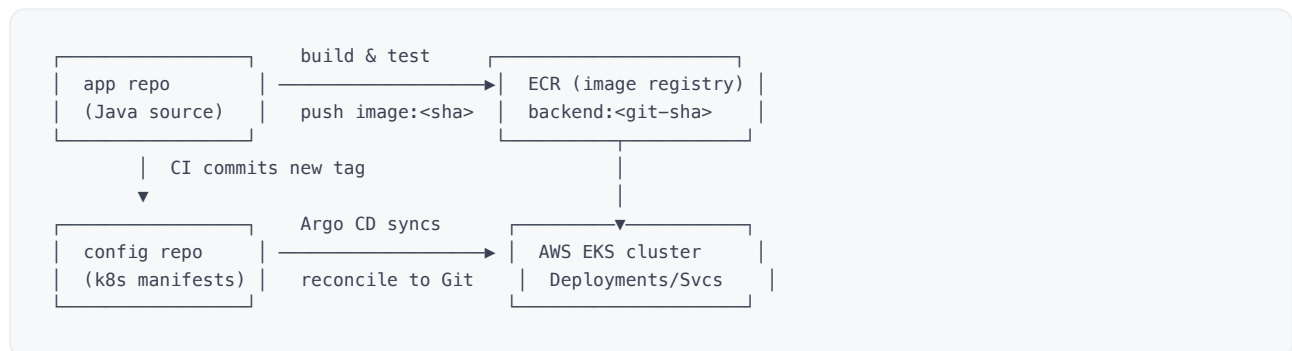
Why. A digital banking platform needs auditability, reproducibility, and safe rollback. Git already gives us reviewed change history, signed commits, and `git revert`. By making Git the single source of truth we get:

- **Auditability** — every production change is a reviewed, attributable commit.
- **Reproducibility** — the cluster can be rebuilt from the repo at any SHA.
- **Drift correction** — if someone hand-edits a live object, the controller

reverts it back to the declared state (self-heal).

- **Trivial rollback** — `git revert` of the offending commit returns the system to a known-good state through the same reviewed path.

How — the two-repo model. We separate *application code* from **deployment state** so that an automated tag bump never triggers a code review and a code change never silently mutates production:



1. A merge to the **app repo** triggers CI: build the JAR, run tests, build the Docker image, push it to **Amazon ECR** tagged with the immutable git SHA.
1. CI then opens (or commits) a tag bump in the **config repo** — changing the image tag in the environment overlay.
1. **Argo CD**, running inside EKS, notices the config repo changed and syncs the cluster to the new desired state.

Why EKS over ECS for GitOps. ECS is a fine container runtime, but GitOps tooling (Argo CD, Flux, External Secrets Operator, Prometheus Operator) is built around the **Kubernetes declarative API**. Kubernetes gives us a single uniform object model that Argo CD can diff and reconcile; ECS task definitions and services are managed through AWS APIs that Argo cannot natively reconcile. EKS also gives us portable manifests, mature autoscaling primitives (HPA), and a rich ecosystem — at the cost of more moving parts than ECS. For a regulated platform where reconciliation, drift detection, and a uniform control plane matter, that trade-off is worth it.

Note: stateful services (RDS PostgreSQL, Amazon MQ / RabbitMQ, S3, Secrets Manager) stay **outside** the cluster as managed AWS services. Kubernetes runs only the **stateless** backend replicas and the single event relay. The database is **never** run inside Kubernetes.

2. Containerizing the Backend

What. A single Docker image for the Spring Boot 3 modular monolith, built with a multi-stage Dockerfile and run as N stateless replicas.

Why a multi-stage build. The build stage needs the full JDK and Maven; the runtime needs only a slim JRE. Multi-stage keeps the final image small (faster pulls, smaller attack surface) and ensures build tooling never ships to prod.

We run as a **non-root** user (defense in depth — a compromised process cannot write outside its sandbox or escalate easily) and bake in a **HEALTHCHECK** that hits Actuator so the container runtime and orchestrator agree on liveness.

How. Stage 1 resolves dependencies (cached layer) then builds the JAR.

Stage 2 copies only the JAR onto a distroless-style JRE base, drops privileges, and exposes the Actuator port.

```
# ---- Stage 1: build ----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /workspace

# Cache dependencies first for faster incremental builds
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

# Build the application JAR
COPY src ./src
RUN mvn -B -q clean package -DskipTests

# ---- Stage 2: runtime ----
FROM eclipse-temurin:17-jre-jammy AS runtime

# Create an unprivileged user
RUN groupadd --system --gid 10001 app \
    && useradd --system --uid 10001 --gid app --no-create-home app

WORKDIR /app
COPY --from=build /workspace/target/*.jar /app/app.jar

# Drop root
```

```

USER 10001:10001
EXPOSE 8080

# Container-level health check against Spring Boot Actuator
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
  CMD wget -qO- http://127.0.0.1:8080/actuator/health/liveness || exit 1

ENV JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+UseG1GC -Djava.security.egd=file:/dev/./urandom"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]

```

`.dockerignore` keeps the build context tiny and prevents secrets or local

crufft from leaking into image layers:

```

target/
.git/
.gitignore
.idea/
*.iml
*.md
.env
.env.*
docker-compose*.yaml
**/node_modules
**/.DS_Store
k8s/
README*

```

3. CI Pipeline (GitHub Actions)

What. A workflow in the **app repo** that builds, tests, pushes the image to ECR tagged with the git SHA, and then bumps that tag in the **config repo**.

Why tag with the git SHA. Immutable, content-addressable tags make every deploy traceable to an exact commit and prevent the "mutable `latest`" anti-pattern where two clusters running `:latest` are actually different. The config repo edit is the *only* thing that triggers a deploy, so build and deploy stay decoupled and the deploy is itself a reviewable Git change.

How. OIDC federation lets GitHub assume an AWS role with no long-lived keys. After push, a small step checks out the config repo and rewrites the image tag in the prod overlay via `kustomize edit set image`, then commits it. (For stricter control, replace the direct commit with a PR — shown in the comment.)

```

# .github/workflows/ci-cd.yml (app repo)
name: ci-cd
on:
  push:
    branches: [main]

permissions:
  id-token: write      # required for AWS OIDC
  contents: read

env:
  AWS_REGION: eu-west-1
  ECR_REPOSITORY: banking/backend

jobs:
  build-test-push:
    runs-on: ubuntu-latest
    outputs:
      image_tag: ${ steps.meta.outputs.sha }
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"
          cache: maven

      - name: Build & unit test
        run: mvn -B verify

      - name: Compute short SHA
        id: meta
        run: echo "sha=${GITHUB_SHA:12}" >> "$GITHUB_OUTPUT"

      - name: Configure AWS credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gha-ecr-push
          aws-region: ${ env.AWS_REGION }

      - name: Login to Amazon ECR
        id: ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build & push image
        env:
          REGISTRY: ${ steps.ecr.outputs.registry }
          TAG: ${ steps.meta.outputs.sha }
        run: |
          docker build -t "$REGISTRY/$ECR_REPOSITORY:$TAG" .
          docker push "$REGISTRY/$ECR_REPOSITORY:$TAG"

  bump-config:
    needs: build-test-push
    runs-on: ubuntu-latest
    steps:
      - name: Checkout config repo
        uses: actions/checkout@v4
        with:
          repository: acme/banking-config
          token: ${ secrets.CONFIG_REPO_PAT }

      - name: Install kustomize
        run: |
          curl -sSL https://raw.githubusercontent.com/kubernetes-
          sigs/kustomize/master/hack/install_kustomize.sh | bash

```

```

sudo mv kustomize /usr/local/bin/

- name: Bump image tag in staging overlay
  env:
    TAG: ${ needs.build-test-push.outputs.image_tag }
  run: |
    cd overlays/staging
    kustomize edit set image \
      backend=123456789012.dkr.ecr.eu-west-1.amazonaws.com/banking/backend:${TAG}

- name: Commit the bump
  env:
    TAG: ${ needs.build-test-push.outputs.image_tag }
  run: |
    git config user.name "ci-bot"
    git config user.email "ci-bot@users.noreply.github.com"
    git commit -am "deploy(staging): backend ${TAG}"
    git push
    # For prod: open a PR instead of pushing, e.g. with `gh pr create`,
    # so a human approves promotion before Argo CD syncs prod.

```

4. Kubernetes Manifests for the Backend

What. The stateless backend declared as a `Deployment`, fronted by a `Service`, scaled by a `HorizontalPodAutoscaler`, and exposed through an `Ingress` backed by the AWS Load Balancer Controller (ALB).

Why these choices.

- **Resource requests/limits** give the scheduler accurate bin-packing information and protect noisy-neighbour isolation; requests also drive HPA and cluster-autoscaler decisions.
- **Liveness vs readiness probes** are split: liveness restarts a wedged process; readiness gates traffic until the app (and its DB pool) is warm. Spring Boot Actuator exposes both groups natively.
- **securityContext non-root + read-only root FS** matches the Dockerfile and satisfies banking hardening baselines.
- **RollingUpdate with maxUnavailable: 0** guarantees no capacity dip during deploys — new pods must be Ready before old ones drain.

How. Probes hit `/actuator/health/liveness` and `/readiness`; env comes from

the k8s Secret that External Secrets Operator populates (§8); the ALB Ingress terminates TLS via ACM.

```
# base/backend-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels: { app: backend }
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels: { app: backend }
  template:
    metadata:
      labels: { app: backend }
    spec:
      serviceAccountName: backend          # IRSA-bound (see §9)
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        fsGroup: 10001
        seccompProfile: { type: RuntimeDefault }
      containers:
        - name: backend
          image: backend                    # tag set by kustomize/CI
          ports:
            - { name: http, containerPort: 8080 }
          envFrom:
            - secretRef: { name: backend-secrets } # from ExternalSecret
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
          resources:
            requests: { cpu: "500m", memory: "768Mi" }
            limits: { cpu: "1500m", memory: "1536Mi" }
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: http }
            initialDelaySeconds: 40
            periodSeconds: 15
            failureThreshold: 3
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: http }
            initialDelaySeconds: 20
            periodSeconds: 10
            failureThreshold: 3
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: ["ALL"] }
          volumeMounts:
            - { name: tmp, mountPath: /tmp }
      volumes:
        - name: tmp
          emptyDir: {}
```

```
# base/backend-service.yaml
apiVersion: v1
```

```
kind: Service
metadata:
  name: backend
spec:
  type: ClusterIP
  selector: { app: backend }
  ports:
    - { name: http, port: 80, targetPort: http }
```

```
# base/backend-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 3
  maxReplicas: 12
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 65 }
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

```
# base/backend-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: backend
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:eu-west-1:123456789012:certificate/abcd-efgh
    alb.ingress.kubernetes.io/healthcheck-path: /actuator/health/readiness
    alb.ingress.kubernetes.io/ssl-redirect: '443'
spec:
  rules:
    - host: api.bank.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: backend
                port: { number: 80 }
```

5. The Outbox Relay Deployment

What. A *separate* Deployment running the same image but with a profile that activates only the **transactional outbox relay** — the background worker that reads unpublished rows from the `outbox` table and publishes them to Amazon MQ (RabbitMQ).

Why `replicas: 1`. The relay must publish each event **exactly once** (at least, never *double-publish*). If two replicas polled the same outbox rows concurrently, a banking event (e.g. a transfer-completed message) could be emitted twice, causing duplicate downstream processing. Running a single active replica is the simplest correct design. The trade-off is availability: if the single pod dies, publishing pauses until Kubernetes reschedules it (seconds). Because the outbox is durable in PostgreSQL, no events are *lost* — only briefly delayed. We therefore set `strategy: Recreate` so we never have two relays overlapping during a rollout.

Alternative — leader election. For higher availability, run multiple replicas that elect a single leader (Spring Integration `LockRegistryLeader` backed by a JDBC/Redis lock, or a Kubernetes `Lease` via the `leader-election` API). Only the leader polls and publishes; on leader loss a standby takes over within the lease TTL. This adds complexity for faster failover — appropriate once publish latency SLAs tighten.

```
# base/outbox-relay-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: outbox-relay
  labels: { app: outbox-relay }
spec:
  replicas: 1                # exactly one active publisher
  strategy:
    type: Recreate          # never overlap two relays during rollout
  selector:
    matchLabels: { app: outbox-relay }
  template:
    metadata:
      labels: { app: outbox-relay }
    spec:
      serviceAccountName: backend
      securityContext:
```

```

runAsNonRoot: true
runAsUser: 10001
containers:
- name: outbox-relay
  image: backend          # same image, different role
  envFrom:
  - secretRef: { name: backend-secrets }
  env:
  - name: SPRING_PROFILES_ACTIVE
    value: prod,relay      # enables @Scheduled outbox poller only
  resources:
    requests: { cpu: "200m", memory: "512Mi" }
    limits:   { cpu: "500m", memory: "768Mi" }
  livenessProbe:
    httpGet: { path: /actuator/health/liveness, port: 8080 }
    initialDelaySeconds: 40
    periodSeconds: 20

```

6. Kustomize Structure

What. A `base/` holding environment-agnostic manifests, plus `overlays/` per environment (`dev` , `staging` , `prod`) that patch the base.

Why Kustomize. It is template-free and built into `kubectl` ; overlays are plain YAML patches, so diffs are readable in PRs and Argo CD renders them natively. Per-environment differences — replica counts, image tags, resource sizes, host names — live as small overrides instead of duplicated full manifests (DRY). The trade-off vs Helm: no loops/conditionals, but for a fixed set of environments that simplicity is a feature.

How. The base lists every resource and a common image name; each overlay sets the concrete image tag (bumped by CI) and patches the deltas.

```

banking-config/
├── base/
│   ├── kustomization.yaml
│   ├── backend-deployment.yaml
│   ├── backend-service.yaml
│   ├── backend-hpa.yaml
│   ├── backend-ingress.yaml
│   ├── outbox-relay-deployment.yaml
│   ├── external-secret.yaml
│   └── service-account.yaml
└── overlays/
    ├── dev/          { kustomization.yaml, patches... }

```

```
├─ staging/ { kustomization.yaml, patches... }
└─ prod/   { kustomization.yaml, patches... }
```

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - service-account.yaml
  - external-secret.yaml
  - backend-deployment.yaml
  - backend-service.yaml
  - backend-hpa.yaml
  - backend-ingress.yaml
  - outbox-relay-deployment.yaml
images:
  - name: backend
    newName: 123456789012.dkr.ecr.eu-west-1.amazonaws.com/banking/backend
    newTag: PLACEHOLDER # overridden per overlay by `kustomize edit set image`
commonLabels:
  app.kubernetes.io/part-of: banking-platform
```

```
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: banking-prod
resources:
  - ../../base
images:
  - name: backend
    newName: 123456789012.dkr.ecr.eu-west-1.amazonaws.com/banking/backend
    newTag: "9f2c1a7b3e0d" # immutable git SHA, bumped via reviewed PR
patches:
  - path: replicas-and-resources.yaml
    target: { kind: Deployment, name: backend }
```

```
# overlays/prod/replicas-and-resources.yaml (strategic-merge patch)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 6
  template:
    spec:
      containers:
        - name: backend
          resources:
            requests: { cpu: "1", memory: "1536Mi" }
            limits: { cpu: "2", memory: "2560Mi" }
```

Overlays for `dev` / `staging` use the same shape with smaller numbers (e.g. dev:

```
replicas: 1 , cpu: 250m ).
```

7. Argo CD Setup

What. Argo CD runs inside EKS and continuously reconciles each environment's overlay path in the config repo onto its namespace. We use the **app-of-apps** pattern: one root Application points at a directory of child Applications, one per environment.

Why app-of-apps + automated sync. A single root app bootstraps the entire platform and keeps the *set* of environments itself under GitOps. Automated sync with `selfHeal` reverts manual drift; `prune` deletes objects removed from Git so the cluster never accumulates orphans. Prod typically keeps automated sync but *gates what reaches its branch/path* behind PR review (promotion is a Git action, §12).

How — install (note). Install via the official manifests or Helm:

```
kubectl create namespace argocd
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Root app (app-of-apps):

```
# argocd/root-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: banking-root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/acme/banking-config.git
    targetRevision: main
    path: argocd/apps          # directory of child Application manifests
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

Per-environment child Application (prod shown):

```
# argocd/apps/prod.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: banking-prod
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/acme/banking-config.git
    targetRevision: main
    path: overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: banking-prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ApplyOutOfSyncOnly=true
  retry:
    limit: 5
    backoff: { duration: 10s, factor: 2, maxDuration: 3m }
```

`overlays/dev` and `overlays/staging` get analogous Applications pointing at their paths and namespaces.

8. Secrets Management

What. External Secrets Operator (ESO) pulls secret material from **AWS**

Secrets Manager at runtime and projects it into native Kubernetes `Secret`

objects. A `SecretStore` defines *where* secrets come from; an `ExternalSecret`

defines *which* keys to fetch and *how* to shape the resulting `Secret`.

Why never commit raw secrets. Anything in Git is permanent and visible to

every repo reader and CI job — unacceptable for DB credentials or JWT signing

keys. ESO keeps the **source of truth in Secrets Manager** (encrypted, IAM-gated, rotatable) while Git holds only a *reference*. The alternative, **Sealed**

Secrets, encrypts the secret so the *ciphertext* can be committed safely; we

prefer ESO here because Secrets Manager already holds our DB/MQ credentials and

supports automatic rotation.

How. ESO authenticates to AWS via IRSA (the operator's ServiceAccount is bound to an IAM role allowing `secretsmanager:GetSecretValue`), reads the named secrets every `refreshInterval`, and writes a `Secret` the backend consumes via `envFrom`.

```
# base/secret-store.yaml
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: aws-secrets
spec:
  provider:
    aws:
      service: SecretsManager
      region: eu-west-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets      # IRSA-bound SA
```

```
# base/external-secret.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: backend-secrets
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets
    kind: SecretStore
  target:
    name: backend-secrets      # the k8s Secret that envFrom reads
    creationPolicy: Owner
  data:
    - secretKey: SPRING_DATASOURCE_URL
      remoteRef: { key: banking/prod/db, property: jdbc_url }
    - secretKey: SPRING_DATASOURCE_USERNAME
      remoteRef: { key: banking/prod/db, property: username }
    - secretKey: SPRING_DATASOURCE_PASSWORD
      remoteRef: { key: banking/prod/db, property: password }
    - secretKey: SPRING_RABBITMQ_ADDRESSES
      remoteRef: { key: banking/prod/mq, property: amqps_uri }
    - secretKey: JWT_SIGNING_KEY
      remoteRef: { key: banking/prod/jwt, property: private_key }
```

The only thing in Git is the *reference* (`banking/prod/db`), never the value.

9. Connecting to Managed AWS Services

What. Pods reach **RDS PostgreSQL, Amazon MQ (RabbitMQ), S3,** and

Secrets Manager — all managed, all outside the cluster.

Why this split. Stateful services demand backup, PITR, Multi-AZ failover and patching that AWS operates far more reliably than we could inside Kubernetes.

The database in particular is **never** run in-cluster (see infra doc). Pods stay stateless and disposable.

How.

- **RDS / Amazon MQ:** reached over the network using connection strings injected as env vars from the ESO-managed Secret (§8). EKS worker nodes and the RDS/MQ instances live in **private subnets**; **security groups** allow only the node security group to reach the DB port (5432) and the broker port (5671/AMQPS).
- **S3:** accessed with **IRSA** (IAM Roles for Service Accounts) — no static keys. The pod's ServiceAccount is annotated with an IAM role granting scoped S3 access; the AWS SDK picks up temporary credentials automatically.

```
# base/service-account.yaml – IRSA binding for the backend pods
apiVersion: v1
kind: ServiceAccount
metadata:
  name: backend
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/banking-backend-irsa
```

A short Terraform sketch for the EKS cluster, the IRSA role, and Multi-AZ RDS

(full networking lives in the infra doc):

```
module "eks" {
  source      = "terraform-aws-modules/eks/aws"
  cluster_name = "banking-prod"
  cluster_version = "1.30"
  vpc_id      = var.vpc_id
  subnet_ids  = var.private_subnet_ids # nodes in private subnets

  eks_managed_node_groups = {
    default = {
      instance_types = ["m6i.large"]
      min_size       = 3
      max_size       = 12
      desired_size    = 4
    }
  }
}
```

```

    enable_irsas = true
  }

  # IRSA role granting the backend ServiceAccount scoped S3 access
  module "backend_irsas" {
    source      = "terraform-aws-modules/iam/aws//modules/iam-role-for-service-accounts-eks"
    role_name   = "banking-backend-irsas"
    oidc_providers = {
      main = {
        provider_arn           = module.eks.oidc_provider_arn
        namespace_service_accounts = ["banking-prod:backend"]
      }
    }
  }

  resource "aws_db_instance" "postgres" {
    identifier      = "banking-prod"
    engine          = "postgres"
    engine_version  = "16"
    instance_class  = "db.m6g.large"
    allocated_storage = 100
    multi_az        = true          # synchronous standby
    backup_retention_period = 14      # PITR window
    storage_encrypted = true
    db_subnet_group_name = aws_db_subnet_group.private.name
    vpc_security_group_ids = [aws_security_group.rds.id]
    deletion_protection = true
  }

```

10. Observability In-Cluster

What. **Prometheus** scrapes metrics, **Grafana** visualizes them, and

Alertmanager routes alerts — typically installed via the

`kube-prometheus-stack` Helm chart. A `ServiceMonitor` tells Prometheus to scrape the backend's Actuator `/actuator/prometheus` endpoint.

Why the Operator/ServiceMonitor approach. It is itself declarative and

GitOps-friendly: a `ServiceMonitor` is a CRD committed to Git, so monitoring config follows the same review-and-reconcile flow as everything else, instead of hand-edited Prometheus config files.

How — install note.

```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kube-prom prometheus-community/kube-prometheus-stack -n monitoring --create-namespace

```

```
# base/servicemonitor.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: backend
  labels: { release: kube-prom } # matches the Prometheus selector
spec:
  selector:
    matchLabels: { app: backend }
  endpoints:
    - port: http
      path: /actuator/prometheus
      interval: 15s
```

```
# base/prometheus-rules.yaml - example alert rule
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: backend-alerts
  labels: { release: kube-prom }
spec:
  groups:
    - name: backend.rules
      rules:
        - alert: BackendHighErrorRate
          expr: |
            sum(rate(http_server_requests_seconds_count{status=~"5..",app="backend"}[5m]))
            /
            sum(rate(http_server_requests_seconds_count{app="backend"}[5m])) > 0.02
          for: 5m
          labels: { severity: critical }
          annotations:
            summary: "Backend 5xx error rate >2% for 5m"
        - alert: OutboxRelayDown
          expr: up{app="outbox-relay"} == 0
          for: 2m
          labels: { severity: critical }
          annotations:
            summary: "Outbox relay is not being scraped - publishing may be stalled"
```

11. Rollout, Rollback & Disaster Recovery

Rolling update behavior. With `maxUnavailable: 0` / `maxSurge: 1`, a deploy brings up a new pod, waits for its readiness probe to pass, then terminates an old one — repeating until all replicas run the new SHA. Capacity never dips, and a pod that fails readiness halts the rollout automatically.

Rollback = `git revert`. Because the live state equals Git, rolling back is a Git operation, not a cluster operation:

```
# Identify the bad deploy commit in the config repo
git log --oneline overlays/prod
# Revert it – restores the previous image tag / manifest
git revert <bad-sha>
git push
# Argo CD detects the change and syncs prod back to the known-good state.
```

This is auditable (the revert is itself a reviewed commit) and uniform with how every other change ships. Argo CD also retains revision history, so

`argocd app rollback banking-prod <revision>` is available for emergencies, but the Git-first path is preferred so Git never drifts from the cluster.

Canary / blue-green (optional). For higher-risk releases, **Argo Rollouts**

replaces the Deployment with a `Rollout` resource supporting canary steps (shift 10% → analyze metrics → 50% → 100%) or blue-green (full parallel stack, instant switch, instant abort). This is optional; the default RollingUpdate is sufficient for most backend changes.

Disaster recovery. Cluster and app state are reproducible from Git, and ECR holds every image by SHA — so the *compute* tier is disposable. ****Data durability and DR live in the infrastructure document: RDS runs Multi-AZ**** with a synchronous standby and **point-in-time recovery** from continuous backups, targeting **RPO ≤ 5 min** and **RTO ≤ 1 hr**. The database is never in Kubernetes, so a full cluster loss costs no data — rebuild the cluster from Git and reconnect to the surviving RDS instance.

12. Promotion Flow & Summary

Promotion flow (dev → staging → prod). Promotion is just moving a tested image SHA up the overlay ladder via reviewed PRs:

```
build image:<sha> → overlays/dev      (auto-bumped by CI, Argo syncs dev)
|
```



The *same image artifact* is promoted unchanged — only the tag reference moves between overlays — so what you tested is exactly what ships. Each promotion is a small, reviewable Git diff, and rollback is the revert of that diff.

Summary — component → tool.

COMPONENT / CONCERN	TOOL / MECHANISM
Container runtime / orchestr.	AWS EKS (managed Kubernetes)
Backend packaging	Multi-stage Dockerfile → single image (slim JRE)
Image registry	Amazon ECR, tagged by immutable git SHA
CI (build/test/push/bump)	GitHub Actions + OIDC to AWS
Deployment config / templating	Kustomize (base + per-env overlays)
Continuous delivery / sync	Argo CD (app-of-apps, automated sync, self-heal)
Rollback	<code>git revert</code> in config repo → Argo CD reconciles
Backend workload	Deployment (N stateless replicas) + Service + HPA
Event publishing	Outbox relay Deployment (replicas:1 / leader-elect)
External traffic / TLS	Ingress → AWS Load Balancer Controller (ALB) + ACM
Autoscaling	HorizontalPodAutoscaler (CPU) + cluster autoscaler
Secrets	External Secrets Operator ← AWS Secrets Manager
Cloud IAM for pods	IRSA (IAM Roles for Service Accounts)
Relational data	Amazon RDS PostgreSQL (Multi-AZ, PITR) — outside k8s
Messaging	Amazon MQ (RabbitMQ) — outside k8s
Object storage	Amazon S3 (accessed via IRSA)
Metrics / dashboards / alerts	Prometheus + Grafana + Alertmanager (ServiceMonitor)
Dashboard (React)	S3 + CloudFront static hosting (not in k8s)
Mobile (React Native)	App stores (not deployed via k8s)

GitOps gives this platform a single auditable source of truth, automated drift correction, and one-commit rollback — exactly the controls a regulated banking system needs, while keeping all durable state in managed AWS services outside the

cluster.